# Python Programming

## Roadmap: [Python Roadmap]

---

## Chapter 1: Introduction to Python

1.1 History of Python
1.2 What is Python?
1.3 Why Learn Python?
1.4 Main Features of Python
1.5 Python Version
1.6 How Python Code Runs
1.7 Compiler vs Interpreter Basics

---

## Chapter 2: Environment Setup

2.1 Installing Python
2.2 Checking Python Version
2.3 Installing VS Code
2.4 Terminal and Command Line Basics
2.5 Python Interpreter
2.6 Python REPL
2.7 Creating and Running .py Files
2.8 Virtual Environments Using venv
2.9 Installing Packages Using pip
2.10 requirements.txt

---

## Chapter 3: Python Syntax Basics

3.1 First Python Code
3.2 Python File Structure
3.3 Comments
3.4 Indentation Rules
3.5 Variables
3.6 Naming Conventions
3.7 Keywords
3.8 Print Statements
3.9 Input from Users
3.10 Basic Program Structure
3.11 Common Syntax Errors
3.12 Quick Revision Table

CodeWithNishchal

# Chapter 4: Data Types

4.1 Numbers
  4.1.1 int
  4.1.2 float
  4.1.3 complex
4.2 Strings
  4.2.1 String Creation
  4.2.2 String Indexing
  4.2.3 String Slicing
  4.2.4 String Immutability
4.3 Boolean Data Type
  4.3.1 True
  4.3.2 False
4.4 None Type
4.5 Type Checking with type()
4.6 Type Conversion
4.7 Mutable vs Immutable Data Types
4.8 Simple Memory Understanding
4.9 Quick Revision

---

# Chapter 5: Operators

5.1 Arithmetic Operators
5.2 Comparison Operators
5.3 Logical Operators
5.4 Assignment Operators
5.5 Membership Operators
5.6 Identity Operators
5.7 Bitwise Operators
5.8 Operator Precedence

---

# Chapter 6: String Operations

6.1 String Creation
6.2 String Indexing
6.3 String Slicing
6.4 String Methods
6.5 String Formatting
  6.5.1 f-strings
  6.5.2 format() Method
  6.5.3 Old % Formatting
6.6 String Concatenation
6.7 Escape Characters
6.8 Raw Strings
6.9 String Multiplication

CodeWithNishchal

# Chapter 7: Control Flow

7.1 if Statement
7.2 if-else Statement
7.3 elif Statement
7.4 Nested Conditions
7.5 Ternary Operator
7.6 match-case
7.7 Truthy and Falsy Values

---

# Chapter 8: Loops and Iteration

8.1 for Loop
8.2 while Loop
8.3 break
8.4 continue
8.5 pass
8.6 else with Loops
8.7 Nested Loops
8.8 range()
8.9 enumerate()
8.10 zip()
8.11 Iterator Protocol Basics
8.12 StopIteration

# Chapter 9: Data Structures

## 9.1 Lists

- 9.1.1 Creating Lists
- 9.1.2 Accessing List Values
- 9.1.3 Updating List Values
- 9.1.4 List Methods
- 9.1.5 List Slicing
- 9.1.6 List Comprehensions
- 9.1.7 Nested Lists
- 9.1.8 Sorting and Reversing Lists
- 9.1.9 Copying Lists
- 9.1.10 Shallow Copy vs Deep Copy

## 9.2 Tuples

- 9.2.1 Creating Tuples
- 9.2.2 Tuple Immutability
- 9.2.3 Tuple Packing
- 9.2.4 Tuple Unpacking

CodeWithNishchal

9.2.5 Named Tuples
9.2.6 Tuple vs List
9.2.7 Tuple methods
9.2.8 Tuple operations
9.2.9 Tuple packing
9.2.10 Tuple unpacking
9.2.11 Extended Tuple Unpacking
9.2.12 Named Tuples
9.2.13 Tuple vs List

## 9.3 Dictionaries

9.3.1 Creating Dictionaries
9.3.2 Accessing Dictionary Values
9.3.3 Updating Dictionary Values
9.3.4 Dictionary Methods
9.3.5 get() Method
9.3.6 Nested Dictionaries
9.3.7 Dictionary Comprehensions
9.3.8 Merging Dictionaries
9.3.9 Real-World Dictionary Use Cases
9.3.10 Nested Dictionaries
9.3.11 Dictionary Comprehensions
9.3.12 Merging Dictionaries
9.3.13 Copying Dictionaries
9.3.14 Dictionary vs List
9.3.15 Important Dictionary Operations

## 9.4 Sets

9.4.1 Creating Sets
9.4.2 Unique Values
9.4.3 Set Elements Must Be Immutable
9.4.4 Accessing Set Values
9.4.5 Checking Membership
9.4.6 Adding Set Items
9.4.7 Removing Set Items
9.4.8 Set Operations
9.4.9 Union
9.4.10 Intersection
9.4.11 Difference
9.4.12 Symmetric Difference
9.4.13 Subset, Superset, & Disjoint Sets
9.4.14 Set Methods
9.4.15 Set Operators
9.4.16 Updating Sets with Operations
9.4.17 Copying Sets
9.4.18 Frozen Sets
9.4.19 Set Comprehensions

CodeWithNishchal

9.4.20 Set Conversion
9.4.21 Set vs List vs Tuple

**9.5 Collections Module**

9.5.1 Counter
9.5.2 defaultdict
9.5.3 OrderedDict
9.5.4 deque
9.5.5 ChainMap

---

# Chapter 10: Functions

10.1 What is a Function?
10.2 Defining Functions
10.3 Calling Functions
10.4 Parameters
10.5 Arguments
10.6 Positional Arguments
10.7 Return Values
10.8 Returning Multiple Values
10.9 Default Parameters
10.10 Keyword Arguments
10.11 *args
10.12 **kwargs
10.13 *args vs **kwargs
10.14 Function Parameter Order
10.15 Docstrings
10.16 Function Naming Rules

---

# Chapter 11: Advanced Functions

11.1 Functions as First-Class Objects
11.2 Lambda Functions
11.3 map()
11.4 filter()
11.5 reduce()
11.6 Recursion
11.7 Nested Functions
11.8 Closures
11.9 Function Annotations
11.10 Higher-Order Functions

---

CodeWithNishchal

# Chapter 12: Scope and Namespaces

12.1 Local Scope
12.2 Global Scope
12.3 Local vs Global Scope
12.4 Variable Shadowing
12.5 global Keyword
12.6 Enclosing Scope
12.7 nonlocal Keyword
12.8 Built-in Scope
12.9 LEGB Rule
12.10 Namespace Concept
12.11 locals() and globals()
12.12 NameError & UnboundLocalError
12.13 Important Scope Summary Table

# Chapter 13: Modules and Packages

13.1 Importing Modules
13.2 Different Ways to Import
13.3 Standard Library Modules
13.4 Creating Your Own Modules
13.5 Importing Specific Code from Your Own Module
13.6 Module Search Path
13.7 Exploring a Module with dir()
13.8 __name__ == "__main__"
13.9 Package Structure
13.10 __init__.py
13.11 Absolute Imports
13.12 Relative Imports
13.13 Package Management Basics
13.14 pip Basics
13.15 Virtual Environment Basics
13.16 requirements.txt
13.17 Third-Party Packages
13.18 Circular Import Warning

# Chapter 14: Object-Oriented Programming

## 14.1 OOP Fundamentals

- 14.1.1 Classes
- 14.1.2 Objects
- 14.1.3 Attributes
- 14.1.4 Methods
- 14.1.5 self
- 14.1.6 __init__ Constructor
- 14.1.7 Instance Variables

CodeWithNishchal

14.1.8 Class Variables
14.1.9 Methods vs Functions

**14.2 OOP Principles**

14.2.1 Encapsulation
14.2.2 Inheritance
14.2.3 Method Overriding
14.2.4 Polymorphism
14.2.5 Duck Typing
14.2.6 Abstraction
14.2.7 Composition

**14.3 Special Methods**

14.3.1 __str__
14.3.2 __repr__
14.3.3 __len__
14.3.4 __getitem__
14.3.5 __add__
14.3.6 __sub__
14.3.7 __call__
14.3.8 __enter__ & __exit__
14.3.9 Context Manager Real Use
14.3.10 Operator Overloading
14.3.11 Magic Methods / Dunder Methods
14.3.12 Reverse and In-place Operator Methods
14.3.13 __bool__
14.3.14 __iter__ and __next__
14.3.15 __contains__

# Chapter 15: Advanced OOP in Python

15.1 @classmethod
15.2 Class Method as Alternate Constructor
15.3 @staticmethod
15.4 Instance Method vs Class Method vs Static Method
15.5 @property
15.6 @property Setter
15.7 Read-only Property
15.8 Abstract Base Classes
15.9 Why Abstract Base Classes Are Used
15.10 Mixins
15.11 Mixin vs Normal Parent Class
15.12 Metaclasses
15.13 Data Classes
15.14 Normal Class vs Data Class
15.15 Default Values in Data Classes
15.16 Mutable Defaults in Data Classes
15.17 Frozen Data Classes

CodeWithNishchal

# Chapter 16: File Handling and Error Management

## 16.1 File Operations

16.1.1 File Handling
16.1.2 Opening Files
16.1.3 Closing Files
16.1.4 Reading Files
16.1.5 Reading Line by Line
16.1.6 Writing Files
16.1.7 Appending Files
16.1.8 File Modes
16.1.9 with Statement
16.1.10 File Object Methods
16.1.11 Binary Files
16.1.12 File Paths
16.1.13 Basic os Module
16.1.14 Basic pathlib Module
16.1.15 Encoding

## 16.2 Exception Handling

16.2.1 try-except
16.2.2 Catching Exception Object
16.2.3 Multiple except Blocks
16.2.4 Handling Multiple Exceptions Together
16.2.5 else in Exception Handling
16.2.6 finally
16.2.7 try-except-else-finally Flow
16.2.8 Raising Exceptions
16.2.9 Re-raising Exceptions
16.2.10 Custom Exceptions
16.2.11 Exception Hierarchy
16.2.12 Bare except Warning
16.2.13 File Handling with Exception Handling

## 15.3 File Formats

16.3.1 Text Files
16.3.2 CSV Files
16.3.3 csv.reader vs csv.DictReader
16.3.4 JSON Files
16.3.5 json.load() vs json.loads()
16.3.6 XML Files

CodeWithNishchal

# Chapter 17: Advanced Python Concepts

## 17.1 Decorators

17.1.1 Decorators
17.1.2 Function Decorators
17.1.3 Decorators with Function Arguments
17.1.4 functools.wraps
17.1.5 Decorators with Arguments
17.1.6 Multiple Decorators
17.1.7 Class Decorators
17.1.8 Built-in and Standard Decorators

## 17.2 Generators and Iterators

17.2.1 Generators and Iterators
17.2.2 Iterable vs Iterator
17.2.3 Iterator Protocol
17.2.4 __iter__
17.2.5 __next__
17.2.6 Generator Functions
17.2.7 yield
17.2.8 Generator Expressions
17.2.9 yield from
17.2.10 One-time Consumption of Iterators
17.2.11 itertools
17.2.12 Memory Efficiency

## 16.3 Context Managers

16.3.1 __enter__
16.3.2 __exit__
16.3.3 Custom Context Manager Using Class
16.3.4 contextlib
16.3.5 @contextmanager
16.3.6 Resource Management

## 16.4 Regular Expressions

16.4.1 re Module
16.4.2 Pattern Matching
16.4.3 Regex Metacharacters
16.4.4 Common Regex Character Classes
16.4.5 Groups and Capturing
16.4.6 Non-capturing Groups
16.4.7 search()
16.4.8 match()
16.4.9 fullmatch()
16.4.10 findall()
16.4.11 finditer()
16.4.12 sub()
16.4.13 split()
16.4.14 re.compile()

CodeWithNishchal

# Chapter 17: Concurrent and Asynchronous Programming

## 17.1 Multithreading

17.1.1 threading Module
17.1.2 Creating Threads
17.1.3 Managing Threads
17.1.4 Race Conditions
17.1.5 Locks
17.1.6 Synchronization

## 17.2 Multiprocessing

17.1.1 Managing Processes
17.1.2 Process Pools
17.1.3  Memory Sharing Basics
17.1.4 Concurrent Futures

## 17.3 Concurrent Futures

17.3.1ThreadPoolExecutor
17.3.2 ProcessPoolExecutor
17.3.3Futures
17.3.4Async Programming

## 17.4 Async Programming

17.4.1 asyncio
17.4.2 async and Coroutines
17.4.3 await
17.4.4 Event Loop
17.4.5 Tasks
17.4.6 asyncio.gather()
17.4.7 Async Futures
17.4.8 Async Context Managers
17.4.9 Async vs Threading vs Multiprocessing

CodeWithNishchal

# Chapter 1: Python Overview

## 1.1 History of Python

Python was created by **Guido van Rossum**.

He started working on Python in the late 1980s, and Python was first released in **1991**.

The language was designed to be simple, readable, and easy to use. Guido wanted Python code to look clean and understandable, so programmers could focus more on solving problems instead of writing complicated syntax.

Python's name does not come from the snake. It was inspired by a British comedy show called **"Monty Python's Flying Circus."**

Over time, Python became popular because it was easy for beginners and powerful for professionals.

Today, Python is used in many fields like:

- Web development
- Automation
- Data science
- Artificial intelligence
- Machine learning
- Testing
- Scripting
- Backend development

## 1.2 What is Python?

Python is a high-level, general-purpose programming language.

It is used to write programs for automation, web development, data handling, artificial intelligence, machine learning, scripting, testing, and many other tasks.Python is popular because its syntax is simple and easy to read.

Example:

```
PYTHON CODE
print("Hello, World!")
```

Output:  Hello, World!

Python files are saved with the .py extension. Example: main.py, app.py, hello.py

CodeWithNishchal

## 1.3 Why Learn Python?

Python is easy to start with and powerful enough for real-world projects.

Python is useful because:

- It is beginner-friendly.
- It is used in many industries.
- It has simple syntax.
- It has many libraries.
- It is good for automation.
- It is widely used in AI, data science, and backend development.

## 1.4 Main Features of Python

| Feature | Meaning |
|---|---|
| Easy syntax | Python code is simple and readable |
| High-level | We do not need to deal directly with machine-level details |
| Interpreted | Python code runs using the Python interpreter |
| Dynamically typed | No need to declare variable type manually |
| Object-oriented | Supports classes and objects |
| Cross-platform | Runs on Windows, macOS, and Linux |
| Large library support | Many built-in and external libraries are available |
| Open-source | Free to use |

## 1.5 Python Version

For modern learning, use **Python 3**. Python 2 is old and should not be used for new learning.

To check Python version, use:

python --version or:  python3 --version

Example output: Python 3.12.5

CodeWithVishchal

# 1.6 How Python Code Runs

**Basic flow:**

**Python Code → Python Interpreter → Output**

More accurate flow:

**Source Code → Bytecode → Python Virtual Machine → Output**

For now, remember:

- Python code is written in .py files.
- Python interpreter reads and runs the code.
- Python runs code from top to bottom.
- Output is shown on the screen.

# 1.7 Compiler vs Interpreter Basics

Before code runs, it must be translated into a form the computer can understand.

There are two common translators:

- Compiler
- Interpreter

## Compiler

A compiler translates the whole program before running it.

Flow: **Source Code → Compiler → Executable File → Run Program**

Examples of compiled languages:

- C
- C++
- Go
- Rust

## Interpreter

An interpreter runs code more directly.

Flow: **Source Code → Interpreter → Output**

Python is commonly called an interpreted language.

## Compiler vs Interpreter

| **Point** | **Compiler** | **Interpreter** |
|---|---|---|
| **Translation** | Translates full code before running | Runs code more directly |
| **Speed** | Usually faster after compilation | Usually slower |
| **Error checking** | Many errors found before running | Errors can appear while running |
| **Examples** | C, C++, Go, Rust | Python, JavaScript, Ruby |

CodeWithNishchal

# Chapter 2: Environment Setup

Environment setup means preparing your computer so you can write, run, and manage Python programs properly.

For Python development, we mainly need:

1. Python installed on the system
2. A code editor like VS Code
3. Terminal/Command Prompt basics
4. Package manager pip
5. Virtual environment setup using venv

**Basic setup flow:**

**Install Python**
↓
**Check Python Version**
↓
**Install VS Code**
↓
**Write Python Code**
↓
**Run Python File**
↓
**Use venv and pip for Projects**

## 2.1 Installing Python

The safest way is to download Python from the official Python website. The official Python downloads page provides the latest stable Python release and installers for different operating systems.

### For Windows

1. Go to the official Python website.
2. Download the latest stable Python 3 installer.
3. Open the installer.
4. Select **Add python.exe to PATH**.
5. Click **Install Now**.
6. Wait for installation to complete.
7. Open Command Prompt and check the version.

**Important:** Always tick: Add python.exe to PATH

This option allows you to run Python from the terminal.

CodeWithNishchal

### For macOS

Steps:

1. Download the macOS installer from the official Python website.
2. Open the .pkg file.
3. Follow the installation steps.
4. Open Terminal.
5. Check Python version.

Command: python3 --version

### For Linux

Many Linux systems already come with Python installed.

Check version: python3 --version

For Ubuntu/Debian-based systems, Python can usually be installed with:

sudo apt update
sudo apt install python3 python3-pip python3-venv

## 2.2 Checking Python Version

### Windows

python --version or: py --version

### macOS/Linux

python3 --version

## 2.3 Installing VS Code

VS Code is a code editor used to write and run Python code. Python itself runs the code, but VS Code helps us write code easily.

### Steps to Install VS Code

1. Download VS Code from the official website.
2. Install it.
3. Open VS Code.
4. Go to Extensions.
5. Search for **Python**.
6. Install the official Python extension by Microsoft.
7. Open a Python file.
8. Select Python interpreter if VS Code asks.

CodeWithNishchal

### Creating First Python File in VS Code

**Create a file:** hello.py

**Write:** print("Hello CodeWithNishchal")

Run the file.

**Output**: Hello CodeWithNishchal

### Ways to Run Python in VS Code

You can run Python code by:

1. Clicking the **Run** button.
2. Right-clicking and selecting **Run Python File in Terminal**.
3. Opening terminal and running:

python hello.py or: python3 hello.py

VS Code's Python extension provides multiple ways to run Python files, including the "Run Python File in Terminal" option.

**Bad file names:** random.py, math.py, sys.py
**Better file names:** number_game.py, student_report.py, calculator.py

## 2.4 Terminal and Command Line Basics

The terminal / command prompt is a place where we type commands to interact with the computer.

| Task | Windows Command | macOS/Linux Command | Meaning |
|---|---|---|---|
| Check current folder | pwd | pwd | Shows the current folder/location in terminal |
| List files and folders | dir | ls | Shows files and folders inside the current folder |
| Change folder | cd folder_name | cd folder_name | Moves into a folder |
| Go back one folder | cd .. | cd .. | Moves one level back |
| Create folder | mkdir python-notes | mkdir python-notes | Creates a new folder named python-notes |
| Clear terminal | cls | clear | Clears the terminal screen |

CodeWithNishchal

### Running a Python File from Terminal

Suppose your file name is:

app.py

Run it using: python app.py or: python3 app.py

Example file:

```
PYTHON CODE

print("Terminal is running Python")
```

Output: Terminal is running Python

## 2.5 Python Interpreter

The Python interpreter is the program that reads and runs Python code.

When we write: print("Hello")

the computer does not understand this directly.

The Python interpreter reads this code and executes it.

Basic flow: **Python Code → Python Interpreter → Output**

## 2.6 Python REPL

REPL stands for:

- **Read**
- **Evaluate**
- **Print**
- **Loop**

Python REPL allows us to run Python code line by line. It is useful for quick testing.

### Opening Python REPL

Windows: python or: py

macOS/Linux: python3

You may see something like: **>>>**

This means Python REPL is ready.

CodeWithNishchal

### REPL Example

```
>>> 10 + 20
30
>>> print("Hello")
Hello
>>> name = "Aman"
>>> name
'Aman'
```

### Exiting Python REPL

Use: exit()

or press:

Ctrl + Z then Enter on Windows
Ctrl + D on macOS/Linux

### REPL vs Python File

| REPL | Python File |
|---|---|
| Used for quick testing | Used for complete programs |
| Code is not saved automatically | Code is saved in .py file |
| Runs line by line | Runs full file |
| Good for experiments | Good for projects |

### When to Use REPL

Use REPL for:

- Testing small calculations
- Checking syntax
- Trying functions
- Quick experiments

## 2.7 Creating and Running .py Files

A Python file is a file that contains Python code. Python files use the .py extension.

Examples:

main.py, hello.py, calculator.py, student_app.py

CodeWithNishchal

## Creating a Python File

Steps:

1. Create a project folder.
2. Open it in VS Code.
3. Create a new file.
4. Save it with the .py extension.
5. Write Python code.
6. Run the file.

## Example File

File name: hello.py

Code:

```
PYTHON CODE

print("Hello, Python")
print("This is my first Python file")
```

**Run:**

python hello.py  OR  python3 hello.py

**Output:**

Hello, Python
This is my first Python file

## Python Runs File Top to Bottom

```
PYTHON CODE

print("Start")
name = "Nishchal"
print("Name:", name)
print("End")
```

Output:

Start
Name: Nishchal
End

CodeWithNishchal

### Common Mistakes

- Saving file as hello.py.txt.
- Forgetting .py extension.
- Running the wrong file.
- Writing Python code in the terminal instead of a file.
- Not saving file before running.
- Giving file names with spaces.

**Bad:**my first python file.py

**Good:** first_python_file.py

### Best Practices

- Use lowercase file names.
- Use underscores for multiple words.
- Keep one project in one folder.
- Save before running.
- Keep file names meaningful.

## 2.8 Virtual Environments Using venv

A virtual environment is an isolated environment for a Python project. It keeps project packages separate from other projects.

Python's official documentation says venv is the standard tool for creating virtual environments, and each virtual environment has its own independent set of installed Python packages.

### Why Virtual Environment is Needed

Suppose you have two projects:

Project A needs package version 1.0
Project B needs package version 2.0

If both projects use the same global Python environment, package conflicts can happen.

Virtual environments solve this problem.

Computer Python
↓
Project A → venv → packages for Project A
Project B → venv → packages for Project B
Project C → venv → packages for Project C

CodeWithNishchal

# 2.9 Installing Packages Using pip

pip is Python's package installer. It is used to install external libraries.

Python's official installing guide describes PyPI as a public repository of open-source packages, and pip is used to install packages from it.

## What is a Package?

A package is reusable code created by someone else.Instead of writing everything from scratch, we can install packages.

Example packages:

| Package | Use |
|---|---|
| requests | Working with APIs |
| pandas | Data analysis |
| numpy | Numerical computing |
| flask | Web development |
| django | Web development |
| pytest | Testing |

## Check pip Version

pip --version or: python -m pip --version

On macOS/Linux: python3 -m pip --version

## Install a Package

**Example:** pip install requests

Better command: python -m pip install requests

On macOS/Linux: python3 -m pip install requests

## Why python -m pip is Better

This makes sure pip installs the package for the same Python interpreter you are using. This is helpful when multiple Python versions are installed.

## List Installed Packages

pip list or: python -m pip list

CodeWithNishchal

# 2.10 requirements.txt

requirements.txt is a file that stores the list of packages used in a Python project.

It helps other people install the same packages easily.

Example:
requests==2.32.3
pandas==2.2.2
numpy==2.0.1

## Why requirements.txt is Important

Suppose you create a project and install many packages. Later, you share the project with another person. Instead of telling them every package manually, you give them requirements.txt. They can install everything using one command.

## Create requirements.txt

After installing packages, run:

pip freeze > requirements.txt

**or:**

python -m pip freeze > requirements.txt

This creates a file like:

requests==2.32.3
urllib3==2.2.2
certifi==2024.7.4

## Install Packages from requirements.txt

Use: pip install -r requirements.txt

**or:**

python -m pip install -r requirements.txt

## Example Project Structure

weather_app/
│
├── app.py
├── requirements.txt
└── venv/

CodeWithNishchal

# Chapter 3. Python Syntax Basics

Python syntax means the set of rules used to write Python programs.

Just like English has grammar, Python also has grammar. If we do not follow Python syntax rules, the program gives an error Python is beginner-friendly because its syntax is simple, readable, and close to normal English.

**Example:**

```
print("Hello, World!")
```

Output: Hello, World!

## 3.1 First Python Code

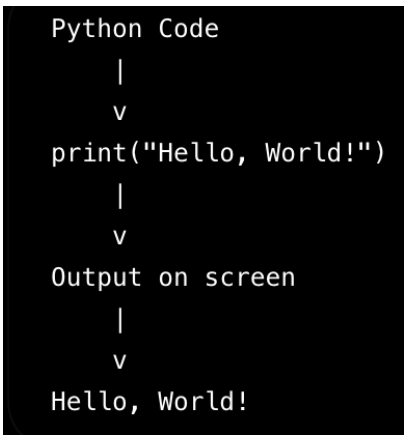
The first Python program usually starts with printing a message on the screen.

```
print("Hello, World!")
```

**Output:** Hello, World!

| Part | Meaning |
|---|---|
| **print** | Built-in Python function used to display output |
| **()** | Parentheses used to pass data to the function |
| **"Hello, World!"** | Text value, also called a string |

**Diagram:**




CodeWithNishchal

**Another example:**

```
print("Welcome to Python")
print("Python is easy to learn")
```

**Output:**

Welcome to Python
Python is easy to learn

Each print() statement displays output on a new line by default.

# 3.2 Python File Structure

Python programs are usually saved with the .py extension.

Example: hello.py

Inside hello.py:

```
print("This is my first Python file")
```

A Python file can be very simple. It may contain only one line of code. But in larger programs, we usually follow a clean structure.

**Basic file structure:**

```
# 1. Comments or program description

# 2. Import statements
import math

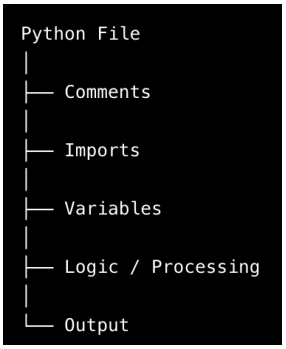
# 3. Variables
radius = 5

# 4. Logic
area = math.pi * radius * radius

# 5. Output
print("Area of circle:", area)
```

LearnWithNishchal

**Diagram:**




**Important point:**

Python does not force a fixed file structure for small programs. But writing code in a clean order makes it easier to read and maintain.

# 3.3 Comments

Comments are notes written inside a program. Python ignores comments while running the code Comments are useful for explaining what the code does.

## Single-Line Comment

```
# This program prints a message
print("Hello Python")
```

Output: Hello Python

The line starting with # is ignored by Python.

## Inline Comment

```
age = 20  # storing user's age
print(age)
```

Output: 20

CodewithNishchal

## Multi-Line Comments

Python does not have a special multi-line comment symbol. The recommended way is to use # on each line.

```
# This program calculates
# the sum of two numbers
# and prints the result

a = 10
b = 20
print(a + b)
```

Output: 30

Triple quotes can also be used for multi-line text, but technically they create a string.

```
"""
This is a multi-line string.
It is sometimes used like a comment,
but it is not an actual comment.
"""

print("Program started")
```

Output: Program started
Best practice for beginners: use # for comments.

# 3.4 Indentation Rules

Indentation means spaces at the beginning of a line. Python uses indentation to define code blocks. In many languages, curly braces {} are used to define blocks. Python uses indentation instead.

Example:

```
if 10 > 5:
    print("10 is greater than 5")
```

Output: 10 is greater than 5
Here, the print() line is indented because it belongs to the if block.

```
if condition:
    statement inside block
    statement inside block

statement outside block
```

CodeWithNishchal

```
age = 18

if age >= 18:
    print("You are eligible to vote")
    print("Please carry your ID")

print("Program finished")
```

**Output:**

You are eligible to vote
Please carry your ID
Program finished

**Explanation:**

if age >= 18: This checks the condition.
  print("You are eligible to vote")
  print("Please carry your ID")

**These two lines are indented,** so they are inside the if block.

print("Program finished") : **This line is not indented, so it is outside the if block.**

### Wrong Indentation

```
if 10 > 5:
print("Correct condition")
```

**Error:** IndentationError: expected an indented block

**Correct code:**

```
if 10 > 5:
    print("Correct condition")
```

# Important Indentation Rules

Use 4 spaces for indentation.

```
if True:
    print("Use 4 spaces")
```

Avoid mixing tabs and spaces.
**Wrong style:**

```
if True:
    print("This uses a tab")
```

Python may give an error if tabs and spaces are mixed.

CodewithNishchal

# 3.5 Variables

A variable is a name used to store data.

Example:

```
name = "Rahul"
age = 21
height = 5.8

print(name)
print(age)
print(height)
```

**Output:**

Rahul
21
5.8

**Variable Name Value**
**name ------> "Rahul"**
**age ------> 21**
**height ------> 5.8**

**Basic syntax:** variable_name = value

Example:

```
city = "Delhi"
marks = 85
is_passed = True
```

| Variable | Value | Type |
|---|---|---|
| city | "Delhi" | String |
| marks | 85 | Integer |

CodeWithNishchal

| | | |
|---|---|---|
| is_passed | True | Boolean |

## Python Variables Do Not Need Type Declaration

In Python, we do not need to write the data type before the variable name.

```
x = 10
print(x)

x = "Python"
print(x)
```

Output: 10

Python

In the first line, x stores an integer. Later, x stores a string. This is allowed because Python is dynamically typed.

## Multiple Variable Assignment

```
a, b, c = 10, 20, 30

print(a)
print(b)
print(c)
```

**Output:**

10

20

30

## Same Value to Multiple Variables

```
x = y = z = 100

print(x)
print(y)
print(z)
```

**Output:**

100

100

100

# 3.6 Naming Conventions

Variable names should be meaningful and easy to understand.

**Good variable names:**

```
student_name = "Aman"
total_marks = 450
is_logged_in = True
```

**Bad variable names:**

CodeWithNishchal

```
x = "Aman"
tm = 450
flag = True
```

These names are not always wrong, but they are less clear.

## Rules for Naming Variables

| Rule | Valid Example | Invalid Example |
|---|---|---|
| Can contain letters | name | — |
| Can contain numbers | student1 | 1student |
| Can contain underscore _ | student_name | student-name |
| Cannot contain spaces | first_name | first name |
| Cannot use keywords | course_name | class |
| Case-sensitive | name, Name | — |

**Valid variable names:**

```
name = "Riya"
student_name = "Amit"
marks1 = 90
_total = 500
is_active = True
```

**Invalid variable names:**

```
1name = "Riya"          # starts with number
student-name = "Amit"   # contains hyphen
first name = "Raj"      # contains space
class = "Python"        # keyword
```

## Python Naming Convention

Python commonly uses snake_case for variable names.

```
student_name = "Karan"
total_salary = 50000
is_available = True
```

| Naming Style | Example 1 | Example 2 | Example 3 |
|---|---|---|---|

Code With Nishchal

| | | | |
|---|---|---|---|
| **snake_case** | student_name | total_marks | user_email |
| **camelCase** | studentName | totalMarks | userEmail |
| **PascalCase** | StudentName | TotalMarks | UserEmail |

# 3.7 Keywords

Keywords are reserved words in Python. They already have special meaning, so we cannot use them as variable names. Some common Python keywords:

1. if
2. else
3. for
4. while
5. class
6. def
7. return
8. True
9. False
10. None
11. import
12. try
13. except
14. with
15. as
16. break
17. Continue

**Wrong example:**

```
class = "Python"
```

This gives an error because class is a keyword.

**Correct example:**

```
course_name = "Python"
```

## Checking Python Keywords

Python provides a built-in module called keyword.

```
import keyword

print(keyword.kwlist)
```

This prints the list of Python keywords. To check whether a word is a keyword:

CodeWithNishchal

```
import keyword

print(keyword.iskeyword("class"))
print(keyword.iskeyword("student"))
```

Output: True
False

# 3.8 Print Statements

The print() function is used to display output on the screen.

## Basic Print

```
print("Hello Python")
```

Output: Hello Python

## Printing Numbers

```
print(100)
print(25.75)
```

**Output:**
100
25.75

## Printing Variables

```
name = "Sneha"
age = 22

print(name)
print(age)
```

**Output:**
Sneha
22

## Printing Text with Variables

```
name = "Arjun"
age = 20

print("Name:", name)
print("Age:", age)
```

**Output:** Name: Arjun

CodeWithNishchal

Age: 20

## Printing Multiple Values

```
a = 10
b = 20

print("A =", a, "B =", b)
```

**Output:** A = 10 B = 20

## Using f-strings

f-strings are a clean way to insert variables inside text.

```
name = "Priya"
age = 19

print(f"My name is {name} and I am {age} years old.")
```

**Output:** My name is Priya and I am 19 years old.

**Another example**

```
product = "Laptop"
price = 55000

print(f"The price of {product} is ₹{price}.")
```

**Output:** The price of Laptop is ₹55000.

## Print with Separator

The sep parameter controls how multiple values are separated.

```
print("Python", "Java", "C++", sep=" | ")
```

**Output:** Python | Java | C++

Default separator is a space.

```
print("Python", "Java", "C++")
```

**Output:** Python Java C++

## Print with End Parameter

By default, print() moves to a new line after printing.

```
print("Hello")
print("Python")
```

Code With Nishchal

**Output:**

Hello

Python

**Using end:**

```
print("Hello", end=" ")
print("Python")
```

**Output:** Hello Python

Another example:

```
print("A", end="-")
print("B", end="-")
print("C")
```

Output:

A-B-C

# 3.9 Input from Users

The input() function is used to take input from the user.

**Example:**

```
name = input("Enter your name: ")
print("Hello", name)
```

**Example output:** Enter your name: Aman
Hello Aman

## Important Point

The input() function always returns data as a string.

```
age = input("Enter your age: ")

print(age)
print(type(age))
```

**Example output:**
Enter your age: 21
21
<class 'str'>
Even though the user entered 21, Python stores it as "21".

CodeWithNishchal

## Taking Integer Input

To convert input into an integer, use int().

```
age = int(input("Enter your age: "))

print("Your age is:", age)
print(type(age))
```

**Example output:**
Enter your age: 21
Your age is: 21
<class 'int'>

## Taking Float Input

To convert input into decimal number, use float().

```
price = float(input("Enter product price: "))

print("Price is:", price)
print(type(price))
```

**Example output:**
Enter product price: 99.50
Price is: 99.5
<class 'float'>

## Example: Add Two Numbers

```
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))

result = num1 + num2

print("Sum is:", result)
```

**Example output:** Enter first number: 10
Enter second number: 20
Sum is: 30

## What Happens Without int()?

```
num1 = input("Enter first number: ")
num2 = input("Enter second number: ")

print(num1 + num2)
```

**Example output:**
Enter first number: 10
Enter second number: 20
1020

CodeWithNishchal

**Here, Python joins the two strings.** It does not perform mathematical addition because both values are strings.

**Correct version:**

```
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))

print(num1 + num2)
```

Output: 30

# 3.10 Basic Program Structure

**A basic Python program usually follows this flow:**

Start
 |
 v
Take input / define data
 |
 v
Process data
 |
 v
Display output
 |
 v
End

**Simple structure:**

```
# Input
number1 = int(input("Enter first number: "))
number2 = int(input("Enter second number: "))

# Processing
sum_result = number1 + number2

# Output
print("Sum:", sum_result)
```

Example: Student Marks Program

```
# Program to calculate total and average marks

student_name = input("Enter student name: ")

math_marks = int(input("Enter math marks: "))
science_marks = int(input("Enter science marks: "))
english_marks = int(input("Enter English marks: "))

total_marks = math_marks + science_marks + english_marks
average_marks = total_marks / 3

print("Student Name:", student_name)
print("Total Marks:", total_marks)
print("Average Marks:", average_marks)
```

ithNishchal

**Example output:**
Enter student name: Ravi
Enter math marks: 80
Enter science marks: 90
Enter English marks: 85
Student Name: Ravi
Total Marks: 255
Average Marks: 85.0

# 3.11 Common Syntax Errors

Syntax errors happen when Python cannot understand the code because syntax rules are broken.

## 1. Missing Parentheses in print()

**Wrong:**

```
PYTHON CODE
print "Hello"
```

**Error:** SyntaxError: Missing parentheses in call to 'print'
**Correct:**

```
PYTHON CODE
print("Hello")
```

## 2. Missing Colon

**Wrong:**

```
PYTHON CODE

age = 20

if age >= 18
  print("Eligible")
```

**Correct:**

```
PYTHON CODE

age = 20

if age >= 18:
  print("Eligible")
```

A colon : is required after statements like: if, else, elif, for, while, def, class, try, except, finally

LearnWithNishchal

## 3. Wrong Indentation

**Wrong:**

```
PYTHON CODE

if True:
print("Hello")
```

**Correct:**

```
PYTHON CODE
if True:
    print("Hello")
```

## 4. Using Keyword as Variable Name

**Wrong:**

```
PYTHON CODE
for = 10
```

**Correct:**

```
PYTHON CODE
number = 10
```

## 5. Variable Used Before Assignment

**Wrong:**

```
PYTHON CODE

print(name)
name = "Aman"
```

**Error:** NameError: name 'name' is not defined

**Correct:**

```
PYTHON CODE
name = "Aman"
print(name)
```

Python reads code from top to bottom. So the variable must be created before using it.

CodeWithNishchal

## 6. Missing Quotes Around String

**Wrong:**

```
PYTHON CODE
name = Aman
```

Python thinks Aman is a variable.

**Correct:**

```
PYTHON CODE

name = "Aman"
```

## 7. Mismatched Quotes

**Wrong:**

```
PYTHON CODE
message = "Hello Python'
```

**Correct:**

```
PYTHON CODE
message = "Hello Python"
```

**Also correct:**

```
PYTHON CODE
message = 'Hello Python'
```

## 8. Invalid Variable Name

**Wrong:**

```
PYTHON CODE
student-name = "Nishchal"
```

**Correct:**

```
PYTHON CODE
student_name = "Nishchal"
```

## 9. Type Conversion Error

**Code:**

```
PYTHON CODE
age = int(input("Enter your age: "))
```

If the user enters:
twenty
**Python gives:** ValueError: invalid literal for int()
Correct input should be numeric: 20

CodeWithNishchal

## 10. Unclosed Parentheses

**Wrong:**

```
PYTHON CODE
print("Hello Python"
```

**Correct:**

```
PYTHON CODE
print("Hello Python")
```

## 11. Extra Closing Bracket

**Wrong:**

```
PYTHON CODE
print("Hello"))
```

**Correct:**

```
PYTHON CODE
print("Hello")
```

# 3.12 Quick Revision Table

| Topic | Key Point |
|---|---|
| **First Python code** | Usually starts with print("Hello, World!") |
| **Python file** | Saved with .py extension |
| **Comments** | Written using # |
| **Indentation** | Defines code blocks in Python |
| **Variables** | Used to store data |
| **Naming convention** | Prefer meaningful snake_case names |
| **Keywords** | Reserved words that cannot be used as variable names |
| **print()** | Displays output |
| **input()** | Takes user input as string |
| **int()** | Converts value to integer |
| **float()** | Converts value to decimal number |
| **Syntax errors** | Occur when Python rules are broken |

CodeWithNishchal

# Chapter 4. Data Types in Python

A **data type** tells Python what kind of value a variable is storing.

Example:

```
name = "Rahul"
age = 21
price = 99.50
is_active = True
```

| Variable | Value | Data Type |
|---|---|---|
| name | "Rahul" | String |
| age | 21 | Integer |
| price | 99.50 | Float |
| is_active | True | Boolean |

**Simple meaning: Data type = Type/category of data stored in a variable**

```
Value
 |
 v
Python checks what kind of value it is
 |
 v
Assigns a data type
```

Example:

```
x = 10
print(type(x))
```

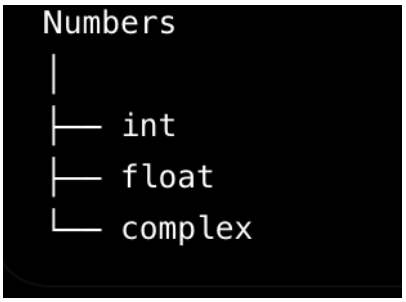
Output: <class 'int'>

This means x is storing an integer value.

WithNishchal

# 4.1 Numbers

Numbers are used to store numeric values. Python mainly has three number types:




## 4.1.1 int

int means **integer**. Integers are whole numbers. They do not have decimal points.

Examples:

```
age = 21
marks = 95
temperature = -5
zero_value = 0

print(age)
print(marks)
print(temperature)
print(zero_value)
```

Output:
21
95
-5
0

All these values are integers.

```
age = 21

print(type(age))
```

Output: <class 'int'>

CodeWithNishchal

## Examples of int

```
student_count = 50
year = 2026
quantity = 10
score = 100
```

**Important point:**
10 is int
10.0 is not int, it is float

```
a = 10
b = 10.0

print(type(a))
print(type(b))
```

**Output:**
<class 'int'>
<class 'float'>

# 4.1.2 float

float means **floating-point number**. Float values contain decimal points.

```
price = 99.50
height = 5.8
temperature = -2.5
percentage = 85.75

print(price)
print(height)
print(temperature)
print(percentage)
```

**Output:**
99.5
5.8
-2.5

WithNishchal

85.75

**Checking type:**

```
price = 99.50

print(type(price))
```

Output: <class 'float'>

## Examples of float

```
salary = 45000.75
distance = 12.5
weight = 60.8
rating = 4.5
```

```
float
 |
 v
Numbers with decimal point
 |
 v
10.5, 0.25, -7.8, 99.99
```

**Important point:**

Python may display 99.50 as 99.5.

Example:

```
price = 99.50
print(price)
```

Output: 99.5

**This does not mean the value is wrong. Python simply removes the unnecessary zero at the end.**

---

CodewithNishchal

# 4.1.3 complex

complex numbers are numbers with two parts:
**Real part + Imaginary part**
In mathematics, complex numbers are usually written like this:
3 + 4i
But in Python, we use j instead of i.

Python complex number:

```
number = 3 + 4j

print(number)
print(type(number))
```

**Output:**
(3+4j)
<class 'complex'>
**Here:** 3  = real part and 4j = imaginary part

```
complex number
 |
 v
3 + 4j
|   |
|   └── imaginary part
|
└── real part
```

**More examples:**

```
a = 2 + 5j
b = 10 + 0j
c = -3 + 7j

print(a)
print(b)
print(c)
```

**Output:**
(2+5j)

CodeWithNishchal

(10+0j)
(-3+7j)
You can access real and imaginary parts like this

```
number = 3 + 4j

print(number.real)
print(number.imag)
```

Output:
3.0
4.0
**Important point:**
Python uses j for complex numbers, not i.
**Wrong:**

```
number = 3 + 4i
```

**Correct:**

```
number = 3 + 4j
```

# 4.2 Strings

A **string** is a sequence of characters.
**Characters can be:** Letters, Numbers, Symbols, Spaces
**Examples:**

```
name = "Rahul"
city = "Delhi"
message = "Hello Python"
phone = "9876543210"
```

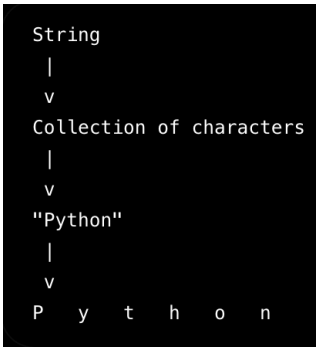
Even though the phone contains numbers, it is inside quotes, so it is a string.

```
phone = "9876543210"

print(type(phone))
```

Output: <class 'str'>

CodeWithNishchal




# 4.2.1 String Creation

Strings can be created using quotes.

**Using Double Quotes**

```
name = "Aman"
print(name)
```

Output: Aman

## Using Single Quotes

```
name = 'Aman'
print(name)
```

**Output:**
Aman
Both are correct.

```
a = "Python"
b = 'Python'

print(a)
print(b)
```

**Output:**
Python
Python

CodeWithNishchal

## Using Triple Quotes

Triple quotes are used for multi-line strings.

```
message = """Python is simple.
Python is powerful.
Python is beginner-friendly."""

print(message)
```

**Output:**
Python is simple.
Python is powerful.
Python is beginner-friendly.

**You can also use triple single quotes:**

```
message = '''Hello
Welcome to Python'''

print(message)
```

**Output:**

Hello

Welcome to Python

## Empty String

A string can also be empty.

CodeWithNishchal

```
text = ""

print(text)
print(type(text))
```

Output:

<class 'str'>

There is no visible text in the first output because the string is empty.

# 4.2.2 String Indexing

Indexing means accessing a single character from a string. Every character in a string has a position number. This position number is called an **index**.

**Python indexing starts from 0.**
**Example:**

```
word = "Python"
```

**Index diagram:**

```
String:   P   y   t   h   o   n
Index:    0   1   2   3   4   5
```

**Code:**

```
word = "Python"

print(word[0])
print(word[1])
print(word[2])
print(word[3])
print(word[4])
print(word[5])
```

**Output:**
P
y
t
h
o
n

## Positive Indexing

Positive indexing starts from the left side.

CodewithNishchal

```
P  y  t  h  o  n
0  1  2  3  4  5
```

**Example:**

```
language = "Python"

print(language[0])
print(language[3])
print(language[5])
```

**Output:**

P
h
n

# Negative Indexing

Negative indexing starts from the right side.

```
String:          P    y    t    h    o    n
Positive index:  0    1    2    3    4    5
Negative index: -6   -5   -4   -3   -2   -1
```

**Example:**

```
language = "Python"

print(language[-1])
print(language[-2])
print(language[-6])
```

Output:

n
o
P

**Important:**

-1 means last character and -2 means second last character

# Index Error

If you try to access an index that does not exist, Python gives an error.

CodeWithNishchal

```
word = "Python"

print(word[10])
```

**Error:** IndexError: string index out of range

**Why?**

"Python" has indexes from 0 to 5 only. Index 10 does not exist.

# 4.2.3 String Slicing

Slicing means taking a part of a string.

**Syntax: string[start:end]**

start index is included. end index is excluded

**Example:**

PYTHON CODE

```
word = "Python"

print(word[0:2])
```

**Output:**

Py

**Explanation:**

```
word[0:2]
 |
 v
Start from index 0
Stop before index 2
```

```
String:  P   y   t   h   o   n
Index:   0   1   2   3   4   5

word[0:2] gives characters from index 0 to before index 2

Result: P y
```

**More examples:**

CodeWithNishchal

**Example:**

```
PYTHON CODE

word = "Python"

print(word[0:4])

print(word[1:4])

print(word[2:6])
```

**Output:**
Pyth
yth
thon

## Leaving Start Empty

If start is empty, Python starts from the beginning.

```
word = "Python"

print(word[:3])
```

**Output:**
Pyt
Meaning:
Start from beginning
Stop before index 3

## Leaving End Empty

If end is empty, Python goes till the end.

```
word = "Python"

print(word[2:])
```

Output: thon
Meaning: Start from index 2 and Go till the end

## Full Slice

```
word = "Python"

print(word[:])
```

Output: Python
This returns the full string.

## Slicing with Negative Index

CodeWithNishchal

```python
word = "Python"

print(word[-3:])
```

```
String:          P    y    t    h    o    n
Negative index: -6   -5   -4   -3   -2   -1

word[-3:] means start from -3 and go till end
Result: h o n
```

Output: hon

## Slicing with Step

Syntax:

```
string[start:end:step]
```

**Example:**

```python
word = "Python"

print(word[0:6:2])
```

```
String: P   y   t   h   o   n
Index:  0   1   2   3   4   5
Pick:   P       t       o

Result: Pto
```

**Output:**
Pto
**Explanation:**
Start at index 0, Go before index 6, Pick every 2nd character

**Reverse string using slicing:**

```python
word = "Python"

print(word[::-1])
```

Output: nohtyP
**Explanation:** [::-1] means read the string from right to left

# 4.2.4 String Immutability

Strings in Python are **immutable**.
**Immutable means:** Once created, it cannot be changed directly.
name = "Ravi"
**Index diagram:**

Prodeep Kishchal

```
String: R   a   v   i
Index:  0   1   2   3
```

Suppose we want to change R to K. This will not work:

```
name = "Ravi"

name[0] = "K"
```

**Error:** TypeError: 'str' object does not support item assignment **Why?** Because strings cannot be changed character by character.

# Correct Way

You can create a new string and store it again.

PYTHON CODE

```
name = "Ravi"
name = "Kavi"
print(name)
```

Output:
Kavi
**Important understanding:**
**Old string:** "Ravi"
**New string:** "Kavi"
Python does not modify the old string. It creates a new string.
Diagram:

```
Before:
name  ------>  "Ravi"

After:
name  ------>  "Kavi"
```

**So this is allowed:**
name = "Ravi"
name = "Kavi"
**But this is not allowed:**
name[0] = "K"
Because changing one character inside the same string is not possible.

CodeWithNishchal

# 4.3 Boolean

Boolean is a data type that has only two possible values:

- True
- False

Boolean type is written as:  bool

```
is_active = True
is_deleted = False

print(is_active)
print(is_deleted)
```

**Output:**
True
False

**Checking type:**

```
is_active = True

print(type(is_active))
```

Output: <class 'bool'>

**Diagram:**

```
Boolean
 |
 v
Only two values
 |
 ├── True
 └── False
```

# 4.3.1 True

True represents yes, correct, active, available, or enabled.

Example:

CodeWithNishchal

```
is_student = True
is_logged_in = True
has_access = True

print(is_student)
print(is_logged_in)
print(has_access)
```

**Output:**
True
True
True
**Important:** True must start with capital T.
**Correct:** is_active = True
**Wrong:** is_active = true
Python will not understand true because Python uses True.

## 4.3.2 False

False represents no, incorrect, inactive, unavailable, or disabled.

```
is_admin = False
is_expired = False
is_completed = False

print(is_admin)
print(is_expired)
print(is_completed)
```

Output:
False
False
False
**Important:** False must start with capital F.
**Correct:** is_active = False
**Wrong:** is_active = false
Python uses False, not false.

# 4.4 None Type

None means **no value** or **empty value**. It is used when a variable exists, but it does not currently store any actual value.

CodeWithNishchal

```
result = None

print(result)
print(type(result))
```

Output:None

<class 'NoneType'>

- None is not 0. None is not an empty string.
- None is not False. **None means no value.**

| Value | Meaning |
|---|---|
| 0 | A number |
| "" | Empty string |
| False | Boolean false |
| None | No value |

```
marks = 0
name = ""
is_passed = False
result = None

print(marks)
print(name)
print(is_passed)
print(result)
```

Output:
0
False
None
There is a blank line after 0 because name is an empty string.
Diagram:

```
Variable created
 |
 v
No actual value yet
 |
 v
Use None
```

**Example:**

```
student_name = None

print(student_name)
```

**Output:** None

odeWithNishchal

**Later, the variable can store a real value:**

```
student_name = None
print(student_name)

student_name = "Aman"
print(student_name)
```

**Output:**
None
Aman
**Important:** None must start with capital N.

Correct:

```
data = None
```

Wrong:

```
data = none
```

# 4.5 Type Checking with type()

The type() function is used to check the data type of a value or variable.
Syntax:

```
type(value)
```

**Example:**

PYTHON CODE

```
age = 21
print(type(age))
```

Output: <class 'int'>

## Checking Different Data Types

devWithNishchal

```
age = 21
price = 99.50
name = "Rahul"
is_active = True
data = None
number = 3 + 4j

print(type(age))
print(type(price))
print(type(name))
print(type(is_active))
print(type(data))
print(type(number))
```

Output:
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
<class 'NoneType'>
<class 'complex'>

```
Variable
 |
 v
type(variable)
 |
 v
Python tells the data type
```

**Example:**

PYTHON CODE
```
x = "100"
print(x)
print(type(x))
```

**Output:**
100
<class 'str'>
Even though it looks like a number, it is inside quotes, so it is a string.

**Example:**

PYTHON CODE
```
x = 100
print(x)
print(type(x))
```

Output: 100
<class 'int'>
Here, 100 is not inside quotes, so it is an integer.

CodeWithNishchal

# 4.6 Type Conversion

Type conversion means changing one data type into another.
Example:
**String "100" ---> Integer 100**
**Integer 10 ---> Float 10.0**
**Number 50 ---> String "50"**

Python provides built-in functions for type conversion.

| Function | Converts Into |
|---|---|
| int() | Integer |
| float() | Float |
| str() | String |
| bool() | Boolean |
| complex() | Complex number |

## 4.6.1 Converting to int

int() converts a value into an integer.
Example:

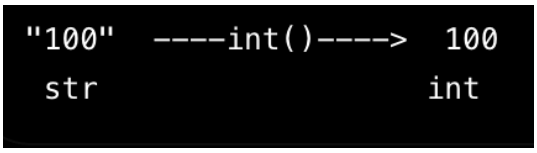
```
x = "100"

y = int(x)

print(y)
print(type(y))
```

Output:
100
<class 'int'>
**Before conversion:** x = "100" type is str
**After conversion:** y = 100 type is int

CodeWithNishchal




## Invalid int() Conversion

This works:

```
PYTHON CODE

number = int("50")
print(number)
```

**Output:** 50

This does not work:

```
PYTHON CODE
number =
int("hello")
print(number)
```

Error: ValueError: invalid literal for int()

**Why?** Because "hello" is not a valid number.

This also does not work:

```
PYTHON CODE
number = int("10.5")
print(number)
```

Error: ValueError: invalid literal for int()

Why? Because "10.5" is a decimal number written as a string. It cannot be converted directly into int.

## 4.6.2 Converting to float

float() converts a value into a decimal number.

```
x = "25.5"

y = float(x)

print(y)
print(type(y))
```

**Output:** 25.5

<class 'float'>

**Integer to float:**

CodeWithNishchal

```
number = 10

converted_number = float(number)

print(converted_number)
print(type(converted_number))
```

**Output:** 10.0
<class 'float'>

**String integer to float:**

```
number = "100"

converted_number = float(number)

print(converted_number)
print(type(converted_number))
```

**Output:**
100.0
<class 'float'>

**Invalid conversion:**

```
value = float("Python")
print(value)
```

**Error:** ValueError: could not convert string to float

## 4.6.3 Converting to str

str() converts a value into a string.

**Example:**

```
age = 21

text_age = str(age)

print(text_age)
print(type(text_age))
```

Output:21
<class 'str'>

```
21  ----str()---->  "21"
int                 str
```

**Checking types:**

PYTHON CODE

price = 99.5
converted_price = str(price)
print(type(price))

CodeWithNishchal

```
print(type(converted_price))
```

**Output:**

<class 'float'>

<class 'str'>

**Important:**

After converting to str, the value becomes text.

Example:

```
PYTHON CODE

x = 100
y = "100"
print(type(x))
print(type(y))
```

**Output:**

<class 'int'>

<class 'str'>

Both look similar when printed, but their data types are different.

## 4.6.4 Converting to bool

bool() converts a value into True or False.

Example:

```
PYTHON CODE

x = 1
y = 0
print(bool(x))
print(bool(y))
```

**Output:** True

False

**Some simple conversions:**

```
PYTHON CODE
print(bool(1))
print(bool(0))
print(bool("Python"))
print(bool(""))
```

**Output:**

True

False

True

False

CodeWithNishchal

```
0              becomes False
Empty string   becomes False
Non-empty text becomes True
```

We will study this more deeply in **Truthy and Falsy Values** in the Control Flow section.

### 4.6.5 Converting to complex

complex() converts a value into a complex number.

**Example:**

```
x = 10

y = complex(x)

print(y)
print(type(y))
```

Output:
(10+0j)
<class 'complex'>

**String to complex:**

```
x = "5"

y = complex(x)

print(y)
print(type(y))
```

**Output:**
(5+0j)
<class 'complex'>

CodeWithNishchal

# 4.7 Mutable vs Immutable Data Types

This is a very important concept in Python. First understand these two words:

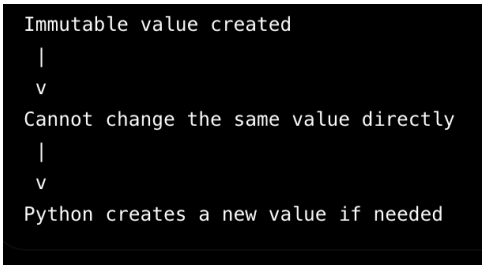
**Mutable = Can be changed after creation**
**Immutable = Cannot be changed after creation**

## Immutable Data Types

Immutable data types cannot be changed directly after they are created. Examples of immutable data types:

- int
- float
- complex
- str
- bool
- NoneType

We have already studied these types in this chapter.




Example with string:

```
PYTHON CODE
name = "Ravi"
name[0] = "K"
```

Error: TypeError: 'str' object does not support item assignment
**Because string is immutable.**
**Correct idea:**

```
PYTHON CODE
name = "Ravi"
name = "Kavi"
print(name)
```

Output: Kavi

Here, Python does not change "Ravi" directly. It creates a new string "Kavi" and makes name refer to that new string.

CodeWithNishchal

## Immutable Number Example

Numbers are also immutable.

Example:

```
age = 20
age = 21

print(age)
```

Output: 21

## Mutable Data Types

Mutable data types can be changed after creation. Some mutable data types in Python are:

- list
- dictionary
- set

These will be studied in detail later in the **Data Structures** section.For now, only remember:

1. Mutable objects can be modified.
2. Immutable objects cannot be modified directly.

| Type | Mutable or Immutable |
|---|---|
| int | Immutable |
| float | Immutable |
| complex | Immutable |
| str | Immutable |
| bool | Immutable |
| NoneType | Immutable |
| list | Mutable |
| dict | Mutable |
| set | Mutable |

CodeWithNishchal

# 4.8 Simple Memory Understanding

A variable is like a name tag.
It points to a value.
Example: x = 10 Means  x  -----> 10
Now:  x = 20 Means  x  -----> 20
The name x now points to 20. It does not mean 10 changed into 20.
This idea is very useful for understanding mutable and immutable data later.

# 4.9 Quick Revision

| Topic | Meaning |
|---|---|
| Data type | Type/category of value |
| int | Whole numbers |
| float | Decimal numbers |
| complex | Number with real and imaginary parts |
| str | Text / sequence of characters |
| Indexing | Accessing one character from a string |
| Slicing | Accessing a part of a string |
| String immutability | Strings cannot be changed directly |
| bool | Stores True or False |
| None | Represents no value |
| type() | Checks data type |
| Type conversion | Converts one data type into another |
| Mutable | Can be changed after creation |
| Immutable | Cannot be changed after creation |

CodeWithNishchal

# Chapter 5. Operators in Python

Operators are special symbols or words used to perform operations on values and variables.

**Simple meaning:**

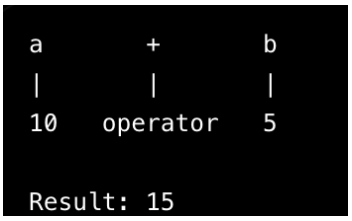
**Operator = Symbol or keyword that performs an operation**

Example:

PYTHON CODE

```
a = 10
b = 5
print(a + b)
```

Output:
15
Here, + is an operator. It adds a and b.
Diagram:




Python operators are mainly divided into these types:

**Operators**
│
├── Arithmetic operators
├── Comparison operators
├── Logical operators
├── Assignment operators
├── Membership operators
├── Identity operators
├── Bitwise operators
└── Operator precedence

---

CodeWithNishchal

# 5.1 Arithmetic Operators

Arithmetic operators are used to perform mathematical operations.

1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Power
6. Remainder

Arithmetic operators work mostly with numbers.

## List of Arithmetic Operators

| Operator | Name | Example |
|---|---|---|
| + | Addition | 10 + 5 |
| - | Subtraction | 10 - 5 |
| * | Multiplication | 10 * 5 |
| / | Division | 10 / 5 |
| // | Floor division | 10 // 3 |
| % | Modulus | 10 % 3 |
| ** | Exponent / Power | 2 ** 3 |

## Addition +

The + operator adds two numbers.

PYTHON CODE

```
a = 10
b = 5
result = a + b
print(result)
```

Output: 15

CodeWithNishchal

## Subtraction -

The - operator subtracts one number from another.

PYTHON CODE

```
a = 10
b = 5
result = a - b
print(result)
```

Output: 5

---

## Multiplication *

The * operator multiplies two numbers.

PYTHON CODE

```
a = 10
b = 5
result = a * b
print(result)
```

Output: 50

---

## Division /

The / operator divides one number by another. Division using / always gives a float result.

PYTHON CODE

```
a = 10
b = 5
result = a / b
print(result)
print(type(result))
```

Output:
2.0
<class 'float'>
Even though 10 / 5 is mathematically 2, Python gives 2.0.
That means the result is a float.

deWithNishchal

## Floor Division //

The // operator divides and gives the whole-number part.
Example:

```
PYTHON CODE

a = 10
b = 3
result = a // b
print(result)
```

Output: 3
Explanation: 10 / 3 = 3.333...
Floor division gives 3

Important point:
/ gives normal division
// gives floor division
Example:

```
PYTHON CODE

print(10 / 3)
print(10 // 3)
```

Output:
3.3333333333333335
3

## Modulus %

The % operator gives the remainder after division.
Example:

```
PYTHON CODE

a = 10
b = 3
result = a % b
print(result)
```

Output:1
Explanation:
10 divided by 3 => 3 goes into 10 three times: 3 × 3 = 9 Remainder: 10 - 9 = 1

CodeWithNishchal

## Exponent / Power **

The ** operator is used to calculate power.

```
PYTHON CODE
result = 2 ** 3
print(result)
```

Output:8

Explanation: 2 ** 3 means 2 raised to the power 3

**Example :**

```
PYTHON CODE
a = 20
b = 6
print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a // b)
print(a % b)
print(a ** 2)
```

Output:

26

14

120

3.3333333333333335

3

2

400

# 5.2 Comparison Operators

Comparison operators are used to compare two values. The result of a comparison is always a Boolean value:

- True
- False

Example:

print(10 > 5)

Output: True

Because 10 is greater than 5.

CodeWithNishchal

## List of Comparison Operators

| Operator | Meaning | Example |
|---|---|---|
| == | Equal to | 10 == 10 |
| != | Not equal to | 10 != 5 |
| > | Greater than | 10 > 5 |
| < | Less than | 5 < 10 |
| >= | Greater than or equal to | 10 >= 10 |
| <= | Less than or equal to | 5 <= 10 |

---

## Equal To ==

The == operator checks whether two values are equal.

```
PYTHON CODE
print(10 == 10)
print(10 == 5)
```

Output:

True

False

## Not Equal To !=

The != operator checks whether two values are not equal.

```
PYTHON CODE
print(10 != 5)
print(10 != 10)
```

Output:

True

False

Explanation:

10 != 5  True because 10 is not equal to 5

10 != 10  False because 10 is equal to 10

---

CodeWithNishchal

## Greater Than >

The > operator checks whether the left value is greater than the right value.

```
PYTHON CODE

print(10 > 5)
print(5 > 10)
```

Output:

True

False

## Less Than <

The < operator checks whether the left value is less than the right value.

```
PYTHON CODE

print(5 < 10)
print(10 < 5)
```

Output:

True

False

---

## Greater Than or Equal To >=

The >= operator checks whether the left value is greater than or equal to the right value.

```
PYTHON CODE

print(10 >= 5)
print(10 >= 10)
print(5 >= 10)
```

Output:

True

True

False

Explanation:

10 >= 5  True because 10 is greater than 5

10 >= 10  True because 10 is equal to 10

5 >= 10  False

---

CodeWithNishchal

## Less Than or Equal To <=

The <= operator checks whether the left value is less than or equal to the right value.

PYTHON CODE

```
print(5 <= 10)
print(10 <= 10)
print(20 <= 10)
```

Output:

True

True

False

Explanation:

5 <= 10  True because 5 is less than 10

10 <= 10  True because 10 is equal to 10

20 <= 10  False

---

## Comparison with Variables

PYTHON CODE

```
a = 15
b = 20
print(a == b)
print(a != b)
print(a > b)
print(a < b)
print(a >= b)
print(a <= b)
```

Output:

False

True

False

True

False

True

---

CodeWithNishchal

# 5.3 Logical Operators

Logical operators are used to combine Boolean values.

Python has three logical operators:

- and
- or
- not

These operators work with True and False.

## List of Logical Operators

| Operator | Meaning |
|---|---|
| and | True when both sides are True |
| or | True when at least one side is True |
| not | Reverses True/False |

## and Operator

The and operator gives True only when both values are True.

PYTHON CODE

```
print(True and True)
print(True and False)
print(False and True)
print(False and False)
```

Output:

True

False

False

False

Truth table:

| Left Value | Right Value | Result |
|---|---|---|
| True | True | True |
| True | False | False |
| False | True | False |
| False | False | False |

CodeWithNishchal

Simple meaning: and means both conditions must be True.

Example:

```
PYTHON CODE
age_valid = True
id_available = True
print(age_valid and id_available)
```

Output:

True

---

## or Operator

The or operator gives True when at least one value is True.

```
PYTHON CODE
print(True or True)
print(True or False)
print(False or True)
print(False or False)
```

**Output:**

True

True

True

False

Truth table:

| Left Value | Right Value | Result |
|---|---|---|
| True | True | True |
| True | False | True |
| False | True | True |
| False | False | False |

or means at least one value must be True.

Example:

```
PYTHON CODE
has_email = True
has_phone = False
print(has_email or has_phone)
```

Output: True

CodeWithNishchal

## not Operator

The not operator reverses a Boolean value.

```
PYTHON CODE
print(not True)
print(not False)
```

**Output:**

False

True

**Simple meaning:**

not True  becomes False

not False becomes True

**Example:**

```
PYTHON CODE
is_logged_in = True
print(not is_logged_in)
```

Output: False

---

## Logical Operators with Comparisons

Comparison operators return Boolean values. So they can be used with logical operators.

**Example:**

```
PYTHON CODE
age = 20
marks = 85
print(age > 18 and marks > 80)
```

Output:

True

Explanation: age > 18  True

marks > 80  True

True and True = True

---

CodeWithNishchal

# 5.4 Assignment Operators

Assignment operators are used to assign values to variables.

The most basic assignment operator is: =

**Example:**

```
PYTHON CODE
x = 10
print(x)
```

Output: 10

Here, 10 is assigned to x.

---

## List of Assignment Operators

| Operator | Example | Same As |
|---|---|---|
| = | x = 10 | x = 10 |
| += | x += 5 | x = x + 5 |
| -= | x -= 5 | x = x - 5 |
| *= | x *= 5 | x = x * 5 |
| /= | x /= 5 | x = x / 5 |
| //= | x //= 5 | x = x // 5 |
| %= | x %= 5 | x = x % 5 |
| **= | x **= 5 | x = x ** 5 |

## Basic Assignment =

```
PYTHON CODE
x = 10
print(x)
```

Output: 10

CodeWithNishchal

## Add and Assign +=

```
PYTHON CODE
x = 10
x += 5
print(x)
```

Output: 15
Explanation:
x += 5

Same as:
x = x + 5

---

## Subtract and Assign -=

```
PYTHON CODE
x = 10
x -= 3
print(x)
```

Output: 7
Explanation: x -= 3
Same as: x = x - 3

---

## Multiply and Assign *=

```
PYTHON CODE
x = 10
x *= 2
print(x)
```

Output:20
Explanation: x *= 2
Same as: x = x * 2

---

CodeWithNishchal

## Divide and Assign /=

```
PYTHON CODE
x = 10
x /= 2
print(x)
```

Output: 5.0

Important: /= gives float result because / gives float result.

---

## Floor Divide and Assign //=

```
PYTHON CODE
x = 10
x //= 3
print(x)
```

Output: 3

Explanation:x //= 3

Same as:x = x // 3

---

## Modulus and Assign %=

```
PYTHON CODE
x = 10
x %= 3
print(x)
```

Output: 1

Explanation: x %= 3

Same as: x = x % 3

---

## Power and Assign **=

```
PYTHON CODE
x = 2
x **= 3
print(x)
```

Output:8

Explanation: x **= 3

Same as: x = x ** 3

CodeWithNishchal

# 5.5 Membership Operators

Membership operators are used to check whether a value exists inside another value.

Python has two membership operators:

1. in
2. not in

For now, we will use membership operators with strings because strings are already covered.

---

## in Operator

The in operator checks whether something is present.

Example:

```
PYTHON CODE
text = "Python"
print("P" in text)
print("Py" in text)
print("Java" in text)
```

**Output:**

True

True

False

**Explanation:**

"P" exists in "Python" True

"Py" exists in "Python" True

"Java" does not exist False

## not in Operator

The not in operator checks whether something is not present.

**Example:**

```
PYTHON CODE

ext = "Python"
print("Java" not in text)
print("Py" not in text)
```

**Output:**

True

False

CodeWithNishchal

## Membership Is Case-Sensitive

Python checks uppercase and lowercase carefully.

Example:

```
PYTHON CODE

text = "Python"
print("P" in text)
print("p" in text)
```

Output:

True

False

Explanation:

"P" and "p" are different in Python.

---

# 5.6 Identity Operators

Identity operators are used to check whether two variables refer to the same object in memory.

Python has two identity operators:

1. is
2. is not

Important:

== checks value

is checks identity/memory object

---

## is Operator

The is operator checks whether two variables point to the same object.

Example:

```
PYTHON CODE

a = None
b = None
print(a is b)
```

Output: **True**

**Explanation:**

a refers to None

b refers to None

Both refer to the same None object.

CodeWithNishchal

## is not Operator

The is not operator checks whether two variables do not point to the same object.

Example:

```
PYTHON CODE
a = None
b = 10
print(a is not b)
```

**Output:**

True

**Explanation:**

a refers to None

b refers to 10

They are not the same object.

---

## Difference Between == and is

== checks whether values are equal.

is checks whether both variables refer to the same object.

```
PYTHON CODE
a = 100
b = 100
print(a == b)
print(a is b)
```

**Possible output:**

True

True

For some simple values, Python may reuse the same object internally.

But beginners should remember this rule: Use == for value comparison. Use is mainly with None.

**Best example:**

```
PYTHON CODE
result = None
print(result is None)
print(result is not None)
```

Output: True

False

CodeWithNishchal

# 5.7 Bitwise Operators

Bitwise operators work on binary numbers.

Binary means numbers written using only:

0 and 1

Computers store numbers internally in binary form.

Example:

Decimal 5 = Binary 101

Decimal 3 = Binary 011

Bitwise operators compare or shift bits.

---

## List of Bitwise Operators

| Operator | Name |
|---|---|
| & | Bitwise AND |
| ` | ` |
| ^ | Bitwise XOR |
| ~ | Bitwise NOT |
| << | Left shift |
| >> | Right shift |

## Bitwise AND &

Bitwise AND compares bits.

Rule:

```
1 & 1 = 1
1 & 0 = 0
0 & 1 = 0
0 & 0 = 0
```

Example:

PYTHON CODE

```
a = 5
b = 3
print(a & b)
```

Output: 1

CodeWithNishchal

```
5 in binary = 101
3 in binary = 011

  101
& 011
-----
  001

001 in decimal = 1
```

So: 5 & 3 = 1

## Bitwise OR |

Bitwise OR compares bits.

```
1 | 1 = 1
1 | 0 = 1
0 | 1 = 1
0 | 0 = 0
```

Example:

```
a = 5
b = 3

print(a | b)
```

Output: 7

Explanation:

```
5 in binary = 101
3 in binary = 011

  101
| 011
-----
  111

111 in decimal = 7
```

deWithNishchal

# Bitwise XOR ^

Bitwise XOR gives 1 when bits are different.

Rule:

```
1 ^ 1 = 0
1 ^ 0 = 1
0 ^ 1 = 1
0 ^ 0 = 0
```

Example:

PYTHON CODE

```
a = 5
b = 3
print(a ^ b)
```

Output: 6

Explanation:

```
5 in binary = 101
3 in binary = 011

  101
^ 011
-----
  110

110 in decimal = 6
```

---

# Bitwise NOT ~

Bitwise NOT flips bits. In Python, the result of ~x is:

```
~x = -(x + 1)
```

Example: x = 5

PYTHON CODE

```
print(~x)
```

Output: -6

Explanation:

~5 = -(5 + 1)

~5 = -6

**For beginners, remember:**

Bitwise NOT does not simply make 5 into -5.

It gives -(number + 1).

CodeWithNishchal

# Left Shift <<

Left shift moves bits to the left.

Simple meaning:

x << n means x multiplied by 2 power n

Example:

```
PYTHON CODE

x = 5
print(x << 1)
```

**Output:**10

**Explanation:**

5 << 1

5 × 2 = 10

**Binary view:**

5 in binary = 101

After left shift by 1:

1010

1010 in decimal = 10

---

# Right Shift >>

Right shift moves bits to the right.

Simple meaning:

x >> n means x divided by 2 power n and gives whole-number result

Example:

```
PYTHON CODE

x = 10
print(x >> 1)
```

Output: 5

Explanation:

10 >> 1

10 // 2 = 5

Binary view: 10 in binary = 1010

After right shift by 1:

101

101 in decimal = 5

CodeWithNishchal

# 5.8 Operator Precedence

Operator precedence means the order in which Python solves operators.

Example:

```
PYTHON CODE
result = 10 + 5 * 2
print(result)
```

**Output:** 20

A beginner may think:

10 + 5 = 15

15 * 2 = 30

But Python does multiplication first.

Correct solving:

10 + 5 * 2

10 + 10

20

So the result is 20.

---

## Why Operator Precedence Matters

Without precedence, Python would not know which operation to do first.

Example:

10 + 5 * 2

There are two possible ways:

**Way 1:** (10 + 5) * 2 = 30

**Way 2:** 10 + (5 * 2) = 20

Python follows fixed precedence rules. So: 10 + 5 * 2 = 20

## and Happens Before or

```
PYTHON CODE
result = True or False and False
print(result)
```

**Output:** True

**Using parentheses:**

result = (True or False) and False

print(result)

**Output:** False

CodeWithNishchal

# Common Operator Precedence Table

Higher operators are solved first.

| Priority | Operators | Meaning |
|---|---|---|
| 1 | () | Parentheses |
| 2 | ** | Power |
| 3 | +x, -x, ~x | Unary plus, unary minus, bitwise NOT |
| 4 | *, /, //, % | Multiplication, division, floor division, modulus |
| 5 | +, - | Addition, subtraction |
| 6 | <<, >> | Bitwise shifts |
| 7 | & | Bitwise AND |
| 8 | ^ | Bitwise XOR |
| 9 | ` | ` |
| 10 | ==, !=, >, <, >=, <= | Comparisons |
| 11 | not | Logical NOT |
| 12 | and | Logical AND |
| 13 | or | Logical OR |

# Parentheses () Have Highest Priority

Parentheses are used to control the order manually.

```
PYTHON CODE
result = (10 + 5) * 2
print(result)
```

Output: 30

Explanation: (10 + 5) * 2 = 15 * 2 = 30

**Without parentheses:**

```
PYTHON CODE
result = 10 + 5 * 2
print(result)
```

Output: 20

# Power Has Higher Priority Than Multiplication

```
PYTHON CODE
result = 2 * 3 ** 2
print(result)
```

Output: 18

CodeWithNishchal

# Chapter 6. String Operations

A **string** is a sequence of characters enclosed in quotes.

name = "Python"

Strings are one of the most used data types in Python because text handling is needed in almost every program.

---

## 6.1 String Creation

Strings can be created using single quotes, double quotes, or triple quotes.

PYTHON CODE

```
s1 = "Hello"
s2 = 'Hello'
s3 = """Hello Python"""
```

All three are valid.

Single quotes and double quotes are commonly used for one-line strings. Triple quotes are used for multi-line strings.

PYTHON CODE

```
message = """Python is simple.
Python is powerful.
Python is widely used."""
```

---

## 6.2 String Indexing

Indexing means accessing one character from a string. Python starts counting from **0**.

**Example:**

PYTHON CODE

```
word = "Python"
print(word[0])
print(word[3])
```

Output:

P

h

For negative indexing, counting starts from the end.

Codewith Nishchal

**Example:**

```
PYTHON CODE
print(word[-1])
print(word[-2])
```

**Output:** n

o

**Index diagram:**

```
P   y   t   h   o   n
0   1   2   3   4   5
-6 -5  -4  -3  -2  -1
```

## 6.3 String Slicing

Slicing means taking a part of a string.

Syntax:

```
string[start:end:step]
```

Important rule: the **start index is included**, and the **end index is excluded**.

**Example:**

```
PYTHON CODE
word = "Python"
print(word[0:3])
print(word[2:])
print(word[:4])
```

Output:

Pyt

thon

Pyth

You can also use step values:

```
PYTHON CODE
print(word[0:6:2])
```

**Output:** Pto

**To reverse a string:**

```
PYTHON CODE
print(word[::-1])
```

Output:

nohtyP

CodeWithNishchal

# 6.4 String Methods

String methods are built-in functions that work on strings.

Common string methods are used for cleaning, checking, changing case, finding text, and replacing text.

## lower()

Converts string to lowercase.

```
PYTHON CODE

text = "Python"
print(text.lower())
```

Output:

python

## upper()

Converts string to uppercase.

```
PYTHON CODE

text = "Python"
print(text.upper())
```

Output:

PYTHON

## strip()

Removes extra spaces from the beginning and end.

```
PYTHON CODE

text = "   Python   "
print(text.strip())
```

Output:

Python

## replace()

Replaces one part of a string with another.

```
PYTHON CODE

text = "I like Java"
print(text.replace("Java", "Python"))
```

Output:

I like Python

CodeWithNishchal

## split()

Splits a string into a list of parts using a separator.

```
PYTHON CODE

text = "apple,banana,mango"
print(text.split(","))
```

Output:

['apple', 'banana', 'mango']

## find()

Finds the position of a substring.

```
PYTHON CODE

text = "Python"
print(text.find("th"))
```

Output:

2

If the text is not found, it returns -1.

## count()

Counts how many times a character or substring appears.

```
PYTHON CODE

text = "banana"
print(text.count("a"))
```

Output: 3

## startswith() and endswith()

```
PYTHON CODE

text = "Python"
print(text.startswith("Py"))
print(text.endswith("on"))
```

**Output:**

True

True

These methods are very useful in text checking and validation.

CodeWithNishchal

# 6.5 String Formatting

String formatting means placing values inside a string in a clean way.

This is better than manually joining many values.

Python has three main ways to format strings:

- f-strings
- format()
- old % formatting

---

## 6.5.1 f-strings

f-strings are the most readable and modern way.

Write f before the string and place variables inside {}.

PYTHON CODE

```
name = "Aman"
age = 20
print(f"My name is {name} and I am {age} years old.")
```

**Output:**

My name is Aman and I am 20 years old.

You can also place expressions inside f-strings.

PYTHON CODE

```
a = 10
b = 5
print(f"Sum is {a + b}")
```

Output:

Sum is 15

## 6.5.2 format()

The format() method inserts values into placeholders.

PYTHON CODE

```
name = "Aman"
age = 20
print("My name is {} and I am {} years old.".format(name, age))
```

Output:

My name is Aman and I am 20 years old.

You can also use position numbers:

print("My name is {0} and I am {1} years old.".format(name, age))

This is useful when you want control over the order of values.

CodeWithNanchal

### 6.5.3 Old % Formatting

This is the older style of formatting.

```
PYTHON CODE

name = "Aman"
age = 20
print("My name is %s and I am %d years old." % (name, age))
```

**Output:** My name is Aman and I am 20 years old.

Here:

- %s is used for string
- %d is used for integer

This style is still found in older code, but f-strings are preferred in modern Python.

---

## 6.6 String Concatenation

Concatenation means joining strings together.

The + operator is used for this.

```
PYTHON CODE

first = "Python"
second = "Programming"
print(first + " " + second)
```

Output:

Python Programming

**Important point:** only strings can be joined directly.

## 6.7 Escape Characters

Escape characters start with a backslash \ and help insert special characters inside strings.

Common escape characters:

| Escape Character | Meaning |
|---|---|
| \n | New line |
| \t | Tab space |
| \\ | Backslash |
| \' | Single quote |
| \" | Double quote |

CodeWithNishchal

## Common Escape Characters

| Escape Character | Meaning | Example | Output |
|---|---|---|---|
| \n | New Line | print("Hello\nPython") | <pre>Hello<br>Python</pre> |
| \t | Horizontal Tab | print("Name\tAge") | <pre>Name Age</pre> |
| \\ | Prints a Backslash | print("C:\\Users\\Admin") | C:\Users\Admin |
| \' | Prints a Single Quote | print('It\'s Python') | It's Python |
| \" | Prints a Double Quote | print("He said \"Hello\"") | He said "Hello" |
| \b | Backspace (removes previous character) | print("ABC\bD") | ABD |
| \r | Carriage Return (moves cursor to beginning of line) | print("Hello\rHi") | Hillo *(behavior may vary by terminal)* |
| \f | Form Feed | print("Hello\fPython") | Inserts a form-feed character |
| \v | Vertical Tab | print("Hello\vPython") | Inserts a vertical tab |
| \a | Alert/Bell | print("\a") | Produces a system beep |

## 6.8 Raw Strings

Raw strings treat backslashes as normal characters.

Write r before the string.

```
PYTHON CODE
path = r"C:\new_folder\test"
print(path)
```

Output: C:\new_folder\test

## 6.9 String Multiplication

A string can be repeated using the * operator.

```
PYTHON CODE

text = "Hi"
print(text * 3)
```

Output: HiHiHi

Prepared with Nishchal

# Common String Methods

| Method | Description | Example | Result |
|---|---|---|---|
| lower() | Converts all characters to lowercase | "Python".lower() | "python" |
| upper() | Converts all characters to uppercase | "Python".upper() | "PYTHON" |
| title() | Converts first letter of every word to uppercase | "python programming".title() | "Python Programming" |
| capitalize() | Capitalizes only the first letter of the string | "python".capitalize() | "Python" |
| swapcase() | Converts uppercase to lowercase and lowercase to uppercase | "PyThOn".swapcase() | "pYtHoN" |
| strip() | Removes spaces from both ends | " Python ".strip() | "Python" |
| lstrip() | Removes spaces from the left side | " Python".lstrip() | "Python" |
| rstrip() | Removes spaces from the right side | "Python ".rstrip() | "Python" |
| replace(old, new) | Replaces one substring with another | "I like Java".replace("Java","Python") | "I like Python" |
| find() | Returns the first index of a substring | "Python".find("th") | 2 |
| index() | Returns the index of a substring (raises error if not found) | "Python".index("th") | 2 |
| count() | Counts occurrences of a substring | "banana".count("a") | 3 |
| startswith() | Checks whether a string starts with a substring | "Python".startswith("Py") | True |
| endswith() | Checks whether a string ends with a substring | "Python".endswith("on") | True |
| split() | Splits a string into a list | "a,b,c".split(",") | ['a', 'b', 'c'] |
| join() | Joins iterable elements into a string | "-".join(["A","B","C"]) | "A-B-C" |
| isalpha() | Returns True if all characters are alphabets | "Python".isalpha() | True |
| isdigit() | Returns True if all characters are digits | "12345".isdigit() | True |
| isalnum() | Returns True if all characters are letters or digits | "Python3".isalnum() | True |
| isspace() | Returns True if all characters are whitespace | " ".isspace() | True |
| center(width) | Centers the string within the specified width | "Python".center(12) | " Python " |
| zfill(width) | Pads the string with leading zeros | "25".zfill(5) | "00025" |

CodeWithNishal

# Chapter 7. Control Flow

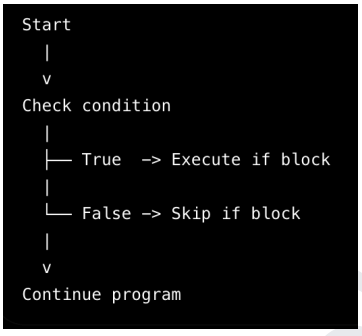
## 7.1 if Statement

### Syntax

```
if condition:
    statement
```

The if statement is used when we want to run some code only when a condition is True.
If the condition is True, Python executes the indented block.
If the condition is False, Python skips the indented block.

### Flow Chart




**Example**

PYTHON CODE
```
age = 20
if age >= 18:
    print("Eligible to vote")
```

**Output:** Eligible to vote

**example:**

PYTHON CODE
```
marks = 35
if marks >= 40:
    print("Passed")
print("Program finished")
```

**Output:** Program finished

CodeWithNishchal

# 7.2 if-else Statement

## Syntax

```
if condition:
    statement_when_true
else:
    statement_when_false
```

## Explanation

The if-else statement is used when we want to run **one block if the condition is True** and **another block if the condition is False**.

Only one block runs.

If the condition is True, the if block runs.

If the condition is False, the else block runs.

## Flow Chart

```
Start
  |
  v
Check condition
  |
  ├── True  -> Execute if block
  |
  └── False -> Execute else block
  |
  v
Continue program
```

**Example**

PYTHON CODE

```
age = 16
if age >= 18:
   print("Eligible to vote")
else:
   print("Not eligible to vote")
```

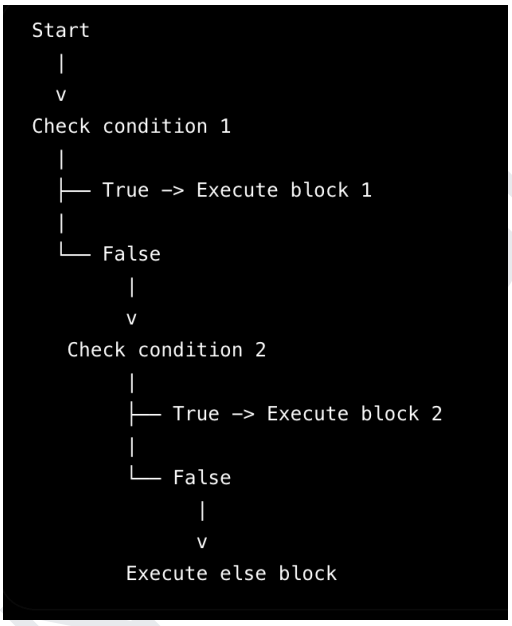
**Output:** Not eligible to vote

CodeWithNishchal

# 7.3 elif Statement

## Syntax

```
if condition1:
    statement1
elif condition2:
    statement2
elif condition3:
    statement3
else:
    default_statement
```




## Explanation

elif means **else if**.

It is used when we need to check **multiple conditions**. Python checks conditions from top to bottom. The first condition that becomes True gets executed. After that, Python skips the remaining conditions.

The else block runs only when all previous conditions are False.

**Example**

PYTHON CODE

```
marks = 75
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
elif marks >= 40:
    print("Grade C")
else:
    print("Fail")
```

**Output:** Grade B

---

# 7.4 Nested Conditions

## Syntax

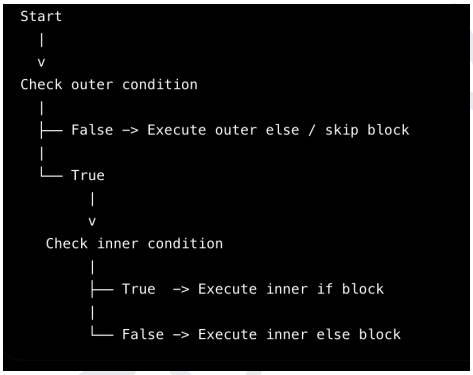
```
if condition1:
    if condition2:
        statement
```

With else:

```
if condition1:
    if condition2:
        statement1
    else:
        statement2
else:
    statement3
```

Nested condition means writing one condition inside another condition. The inner condition is checked only when the outer condition is True. This is useful when one decision depends on another decision.

## Flow Chart




**Example**

```
age = 20
has_id = True

if age >= 18:
    if has_id:
        print("Entry allowed")
    else:
        print("ID required")
else:
    print("Entry not allowed")
```

Output: Entry allowed

Code with Nishchal

# 7.5 Ternary Operator

## Syntax

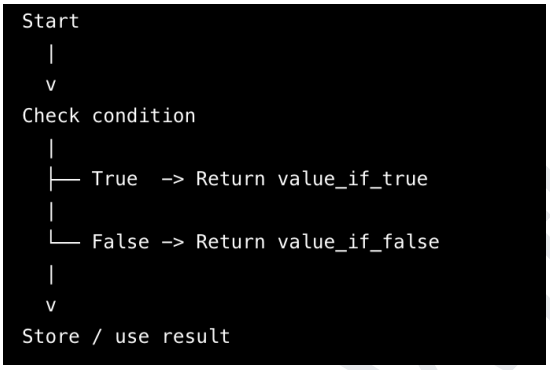
```
value_if_true if condition else value_if_false
```

**Common usage:**

```
variable = value_if_true if condition else value_if_false
```

The ternary operator is a short way to write a simple if-else statement in one line. It is useful when we need to choose between two values. Use it only for simple conditions. For complex logic, normal if-else is better.

## Flow Chart




**Normal if-else:**

```
PYTHON CODE
age = 20
if age >= 18:
    status = "Adult"
else:
    status = "Minor"
print(status)
```

**Output:** Adult

**Same code using ternary operator:**

```
PYTHON CODE
age = 20
status = "Adult" if age >= 18 else "Minor"
print(status)
```

Output: Adult

SwathiNishchal

# 7.6 match-case

## Syntax

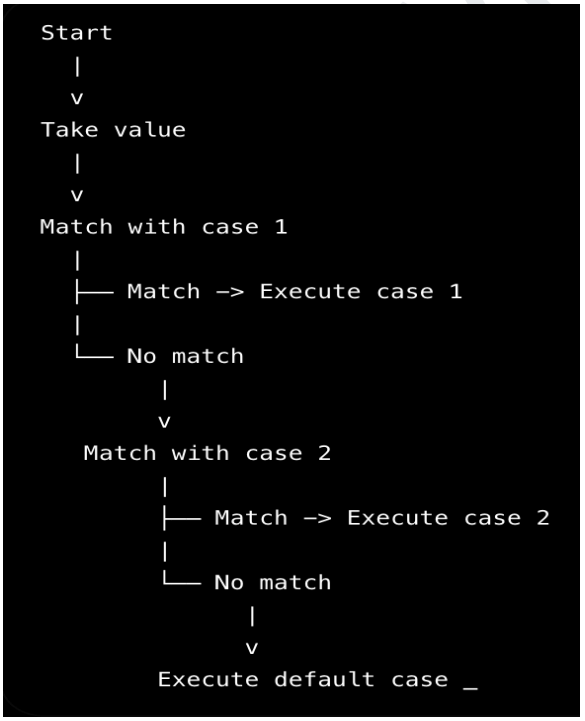
```
match value:
    case pattern1:
        statement1
    case pattern2:
        statement2
    case _:
        default_statement
```

## Explanation

match-case is used to compare one value with multiple possible cases. It is similar to checking many fixed options.

The _ case works like a default case. It runs when no other case matches. match-case was introduced in Python 3.10.

## Flow Chart




ithNishchal

**Example**

```
command = "start"

match command:
    case "start":
        print("Starting program")
    case "stop":
        print("Stopping program")
    case "pause":
        print("Pausing program")
    case _:
        print("Unknown command")
```

**Output:** Starting program

PYTHON CODE

```
day = 3
match day:
   case 1:
      print("Monday")
   case 2:
      print("Tuesday")
   case 3:
      print("Wednesday")
   case _:
      print("Invalid day")
```

Output: Wednesday

# 7.7 Truthy and Falsy Values

## Syntax

```
if value:
    statement
```

## Explanation

In Python, conditions do not always need direct comparison like: if age >= 18:
Python can also treat values as True or False. These are called **truthy** and **falsy** values.
A **truthy value** behaves like True. **A falsy value behaves like False.**

CodeWithNishchal

**Common falsy values:**

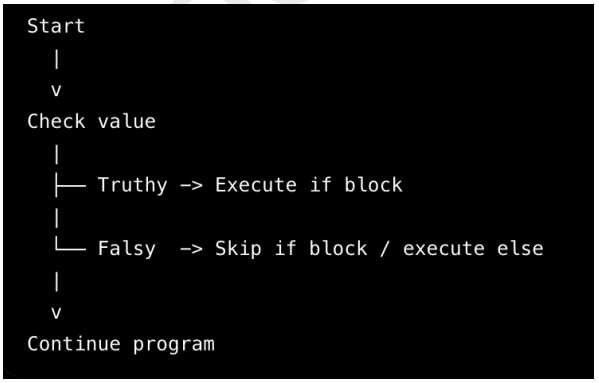
| Value | Meaning |
|---|---|
| False | Boolean false |
| 0 | Zero number |
| 0.0 | Zero float |
| "" | Empty string |
| None | No value |

**Common truthy values:**

| Value | Meaning |
|---|---|
| True | Boolean true |
| 10 | Non-zero number |
| -5 | Non-zero number |
| "Python" | Non-empty string |
| " " | String with a space |

Important: " " is truthy because it contains a space. It is not empty.

# Flow Chart




CodeWithNishchal

Example 1: Non-empty string

```
PYTHON CODE
name = "Rahul"
if name:
    print("Name is available")
else:
    print("Name is missing")
```

**Output:**

Name is available

# Example 2: Empty string

```
PYTHON CODE
name = ""
if name:
    print("Name is available")
else:
    print("Name is missing")
```

**Output:**Name is missing

# Example 3: Number

```
PYTHON CODE

amount = 0
if amount:
    print("Amount available")
else:
    print("Amount is zero")
```

Output: Amount is zero

CodeWithNishchal

# Chapter 8. Loops and Iteration

Loops are used when we want to run the same code multiple times.

**Q: Print numbers from 1 to 5**

Without a loop, we write many print() statements. With loop, we write the logic once and Python repeats it.

# 8.1 for Loop

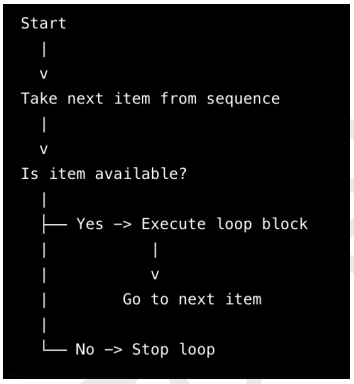
## Syntax

```
for variable in sequence:
    statement
```

## Explanation

1. A for loop is used to repeat code over a sequence.
2. A sequence can be a string, range, list, tuple, etc.
3. In each round, Python takes one value from the sequence.
4. The loop stops automatically when all values are finished.

## Flow Chart




**Example 1: Loop through a string**

```
word = "Python"

for letter in word:
    print(letter)
```

**Output:**

P
y
t
h
o
n

PrepWithNishchal

**Example 2: Loop with range()**

```
for number in range(1, 6):
    print(number)
```

**Output:**

1
2
3
4
5

# 8.2 while Loop

## Syntax

```
while condition:
    statement
```

## Explanation

1. A while loop runs as long as the condition is True.
2. Before every round, Python checks the condition.
3. If the condition is True, the loop block runs.
4. If the condition becomes False, the loop stops.

## Flow Chart

```
Start
  |
  v
Check condition
  |
  ├── True -> Execute loop block
  |           |
  |           v
  |        Check condition again
  |
  └── False -> Stop loop
```

Vishal Lishchal

**Example 1:** Print numbers from 1 to 5

```
PYTHON CODE
number = 1

while number <= 5:
  print(number)
  number += 1
```

**Output:**

**1**
**2**
**3**
**4**
**5**

# for Loop vs while Loop

| Point | for Loop | while Loop |
|---|---|---|
| **Main use** | Used to loop over a sequence | Used to repeat while a condition is True |
| **Best when** | Number of iterations is known | Number of iterations is not fixed |
| **Works with** | String, range(), and other iterable values | Conditions |
| **Stops when** | Sequence ends | Condition becomes False |
| **Risk** | **Usually safer** | **Can create infinite loop if condition never becomes False** |

## When to Use for and while

| Situation | Better Loop |
|---|---|
| Loop through characters in a string | for loop |
| Loop through numbers using range() | for loop |
| Repeat until password is correct | while loop |
| Repeat while balance is available | while loop |
| Repeat fixed number of times | for loop |
| Repeat based on condition | while loop |

CodeWithNishchal

# 8.3 break

**Syntax:**

```
break
```

## Explanation

1. break is used to stop a loop immediately.
2. When Python sees break, it exits the loop.
3. Code after the loop continues normally.

Flow Chart:

```
Loop starts
  |
  v
Condition inside loop
  |
  ├── break found -> Exit loop
  |
  └── no break -> Continue loop
```

## Example 1: Stop loop when number is 4

PYTHON CODE

```
for number in range(1, 8):
if number == 4:
break
print(number)
```

**Output:**

1

2

3

**Explanation:**

1. The loop starts from 1.
2. When number becomes 4, break runs.
3. The loop stops before printing 4.

CodeWithNishchal

# 8.4 continue

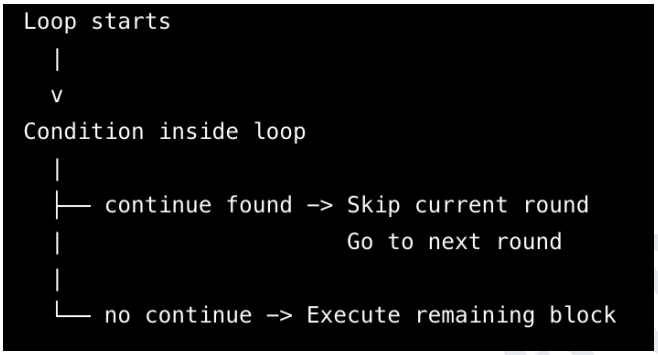
## Syntax

```
continue
```

## Explanation

1. continue skips the current round of the loop.
2. It does not stop the full loop.
3. After continue, Python moves to the next round.

## Flow Chart




Example 1: Skip number 3

```
PYTHON CODE
for number in range(1, 6):
if number == 3:
continue
print(number)
```

**Output:**

1
2
4
5

**Explanation:**

1. When number is 3, continue runs.
2. print(number) is skipped for 3.
3. The loop continues with 4 and 5.

Coded with Nishchal

# 8.5 pass

## Syntax

```
pass
```

1. pass means “do nothing”. It is used when Python needs a statement, but we do not want to write logic yet. It does not stop or skip the loop like break or continue.

## Flow Chart

```
Loop starts
  |
  v
pass found
  |
  v
Do nothing
  |
  v
Continue normal execution
```

### Example 1: Empty loop block

```
PYTHON CODE
for number in range(1, 4):
    pass
```

**Output:** No output appears because pass does nothing.

### Example 2: Placeholder inside condition

```
PYTHON CODE
for number in range(1, 4):
    if number == 2:
        pass
    print(number)
```

**Output:**
1
2
3

## Loop Control Statements Comparison

| Statement | Meaning | Effect on Loop | Common Use |
|---|---|---|---|
| **break** | Stop the loop | Exits the loop completely | Stop when required value is found |
| **continue** | Skip current round | Moves to next iteration | Skip unwanted values |
| **pass** | Do nothing | No effect on loop | Temporary placeholder |

CodeWithNishchal

# 8.6 else with Loops

### Syntax

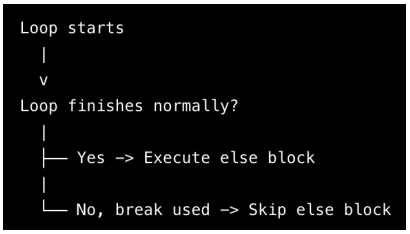
```
for variable in sequence:
    statement
else:
    statement_after_loop
```

```
while condition:
    statement
else:
    statement_after_loop
```

## Explanation

1. A loop can have an else block. The else block runs when the loop finishes normally.
2. If the loop stops because of break, the else block does not run.

## Flow Chart




**Example 1: Loop finishes normally**

```
for number in range(1, 4):
    print(number)
else:
    print("Loop finished")
```

**Output:**

1
2
3
Loop finished

**Example 2: Loop stops with break**

```
PYTHON CODE
for number in range(1, 5):
    if number == 3:
        break
    print(number)
else:
    print("Loop finished")
```

**Output:**

**1**
**2**

CodeWithNishchal

# 8.7 Nested Loops

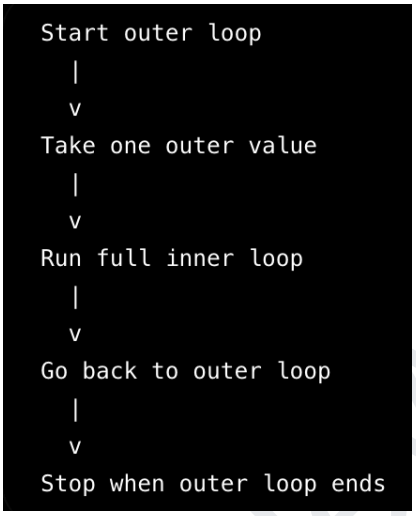
## Syntax

```
for outer_variable in outer_sequence:
    for inner_variable in inner_sequence:
        statement
```

## Explanation

1. Nested loop means one loop inside another loop.
2. The outer loop runs first.
3. For every one round of the outer loop, the inner loop runs completely.
4. Nested loops are useful for patterns, tables, rows and columns.

## Flow Chart




**Example 1: Row and column output**

PYTHON CODE

```
for row in range(1, 3):
    for column in range(1, 4):
        print(row, column)
```

**Output:**

1 1

1 2

1 3

2 1

2 2

2 3

deWithNishchal

## Example 2: Simple pattern

PYTHON CODE

```
for row in range(1, 4):
  print("*" * row)
```

**Output:**

*

**

***

# 8.8 range()

## Syntax

```
range(stop)
range(start, stop)
range(start, stop, step)
```

## Explanation

1. range() creates a sequence of numbers.
2. It is commonly used with for loops.
3. The stop value is excluded.
4. step controls the gap between numbers.

## Flow Chart

```
range() creates numbers
  |
  v
for loop takes one number
  |
  v
Loop block executes
  |
  v
Next number is taken
```

**Example 1: range(stop)**

PYTHON CODE

```
for number in range(5):
print(number)
```

deWithNishchal

**Output:**

**0**

**1**

**2**

**3**

**4**

**Explanation:**

1. range(5) starts from 0.
2. It stops before 5.

## Example 2: range(start, stop)

```
PYTHON CODE
for number in range(1, 6):
    print(number)
```

**Output:**

1
2
3
4
5

## Example 3: range(start, stop, step)

```
PYTHON CODE
for number in range(2, 11, 2):
    print(number)
```

**Output:**

2
4
6
8
10

# range() Forms

| Syntax | Meaning | Example | Output Values |
|---|---|---|---|
| range(stop) | Starts from 0, stops before stop | range(5) | 0, 1, 2, 3, 4 |
| range(start, stop) | Starts from start, stops before stop | range(1, 5) | 1, 2, 3, 4 |
| range(start, stop, step) | Uses step/gap between values | range(2, 10, 2) | 2, 4, 6, 8 |

CodeWithNishchal

# Positive and Negative Step in range()

| Example | Meaning | Output Values |
|---|---|---|
| range(1, 6, 1) | Increase by 1 | 1, 2, 3, 4, 5 |
| range(1, 6, 2) | Increase by 2 | 1, 3, 5 |
| range(5, 0, -1) | Decrease by 1 | 5, 4, 3, 2, 1 |
| range(10, 0, -2) | Decrease by 2 | 10, 8, 6, 4, 2 |

# 8.9 enumerate()

## Syntax

```
for index, value in enumerate(sequence):
    statement
```

## Explanation

1. enumerate() gives both index and value while looping.
2. It is useful when we need the position of each item.
3. The index starts from 0 by default.

## Flow Chart

```
Sequence
  |
  v
enumerate()
  |
  v
Gives index + value
  |
  v
Loop block executes
```

deWithShchal

**Example 1: Enumerate a string**

```
PYTHON CODE

word = "Python"
for index, letter in enumerate(word):
    print(index, letter)
```

**Output:**

0 P

1 y

2 t

3 h

4 o

5 n

## Example 2: Start index from 1

```
PYTHON CODE

word = "Python"


for index, letter in enumerate(word,
start=1):
    print(index, letter)
```

**Output:**

1 P

2 y

3 t

4 h

5 o

6 n

# enumerate() vs Normal Loop

| Normal Loop | enumerate() |
|---|---|
| Gives only value | Gives index and value |
| Good when index is not needed | Good when index is needed |
| Simple for direct looping | Better for position-based output |

CodeWithNishchal

# 8.10 zip()

## Syntax

```
for value1, value2 in zip(sequence1, sequence2):
    statement
```

## Explanation

1. zip() is used to loop over two or more sequences together.
2. It takes one item from each sequence at the same time.
3. The loop stops when the shortest sequence ends.

## Flow Chart

```
Sequence 1: A B C
Sequence 2: 1 2 3
      |
      v
zip()
      |
      v
(A,1), (B,2), (C,3)
      |
      v
Loop runs pair by pair
```

**Example 1: Zip two strings**

PYTHON CODE

```
letters = "ABC"
numbers = "123"
for letter, number in zip(letters, numbers):
    print(letter, number)
```

**Output:**

**A 1**

**B 2**

**C 3**

CodeWithNishchal

## Example 2: Different length sequences

PYTHON CODE

```
letters = "ABCD"
numbers = "12"
for letter, number in zip(letters, numbers):
    print(letter, number)
```

**Output:**

**A 1**

**B 2**

**Explanation:**

1. letters has 4 characters.
2. numbers has 2 characters.
3. zip() stops after the shorter sequence ends.

# 8.11 Iterator Protocol Basics

## Syntax

```
iterator = iter(sequence)
value = next(iterator)
```

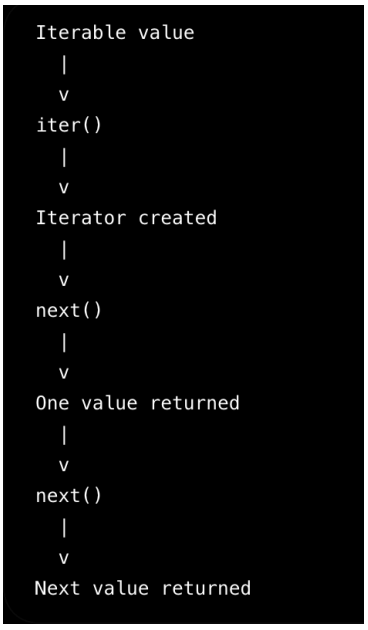
## Explanation

1. An iterator is an object that gives values one by one.
2. iter() creates an iterator from an iterable value.
3. next() gets the next value from the iterator.
4. Loops internally use this idea to get values one by one.

| Term | Meaning |
|---|---|
| Iterable | Something that can be looped over |
| Iterator | Object that gives values one by one |
| iter() | Creates an iterator |
| next() | Gets the next value |

CodewithNishchal

**Flow Chart**




**Example 1: Using iter() and next()**

```
word = "ABC"

iterator = iter(word)

print(next(iterator))
print(next(iterator))
print(next(iterator))
```

**Output:**

A
B
C

## **Example 2:** How for loop thinks internally

**This loop:**

```
PYTHON CODE

for letter in "ABC":
    print(letter)
```

**Output:**

A

B

C

CodeWithNishchal

Internally, the idea is similar to:

```
PYTHON CODE
iterator = iter("ABC")
print(next(iterator))
print(next(iterator))
print(next(iterator))
```

**Output:**
A
B
C

# Iterable vs Iterator

| Term | Meaning | Example |
|---|---|---|
| Iterable | Something we can loop over | String, range() |
| Iterator | Object that gives values one by one | Created using iter() |
| iter() | Creates an iterator | iter("ABC") |
| next() | Gets next value | next(iterator) |

**Example 1: Iterable**

```
word = "ABC"

for letter in word:
    print(letter)
```

Output: A
B
C

Here, "ABC" is iterable because we can loop over it.

## Example 2: Iterator

```
word = "ABC"

iterator = iter(word)

print(next(iterator))
print(next(iterator))
print(next(iterator))
```

**Output:** A
B
C

CodeWithMishchal

# 8.12 StopIteration

**When an iterator has no more values, Python raises StopIteration.**

## Example 1: Iterator ends

```
word = "AB"

iterator = iter(word)

print(next(iterator))
print(next(iterator))
print(next(iterator))
```

**Output:**

**A**

**B**

**StopIteration**

# Infinite Loop

An infinite loop is a loop that never stops. This usually happens when the condition in a while loop never becomes False.

## Example 1: Infinite loop

```
number = 1

while number <= 5:
    print(number)
```

**Problem:**

number starts as 1 => **condition is number <= 5** => number is never increased=> condition always stays True . loop never stops

**Correct version:**

```
number = 1

while number <= 5:
    print(number)
    number += 1
```

**Output:**
**1**
**2**
**3**
**4**
**5**

CodeWithNishchal

# Chapter 9. Data Structure in Python

## 9.1 Lists in Python

A **list** is a data structure used to store multiple values in one variable.

### Syntax

```
list_name = [value1, value2, value3]
```

### Explanation

1. Lists are written using square brackets [].
2. List items are separated by commas.
3. Lists are ordered, so every item has an index.
4. Lists are mutable, meaning we can change them after creation.
5. Lists can store duplicate values.
6. Lists can store different data types together.

### Example 1

```
student = ["Rahul", 21, "Python", 85.5]

print(student)
```

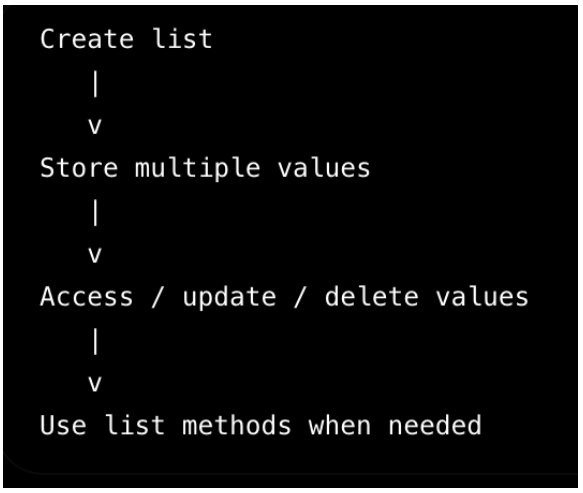
**Output:** ['Rahul', 21, 'Python', 85.5]

### List Properties Table

| Property | Meaning | Example |
|---|---|---|
| **Ordered** | Items have fixed positions | items[0] |
| **Mutable** | Items can be changed | items[1] = "new" |
| **Allows duplicates** | Same value can appear multiple times | [10, 10, 20] |
| **Mixed data allowed** | Can store different data types | ["Aman", 20, True] |
| **Indexed** | Every item has position number | 0, 1, 2... |

CodewithNishchal

Flow Chart




# 9.1.1 Creating Lists

## Syntax

```
list_name = [item1, item2, item3]
```

1. A list can store numbers, strings, Boolean values, or mixed values.
2. An empty list can also be created.
3. Lists are useful when many related values need to be stored together.

## Example 1

```python
numbers = [10, 20, 30, 40]
names = ["Aman", "Riya", "Kabir"]
mixed = ["Python", 100, 99.5, True]
empty_list = []

print(numbers)
print(names)
print(mixed)
print(empty_list)
```

**Output:**

[10, 20, 30, 40]
['Aman', 'Riya', 'Kabir']
['Python', 100, 99.5, True]
[]

eWithNishchal

## Different Ways to Create Lists

| Type | Example |
|---|---|
| **Empty list** | items = [] |
| **Number list** | marks = [80, 90, 75] |
| **String list** | names = ["Aman", "Riya"] |
| **Mixed list** | data = ["Aman", 20, True] |
| **Nested list** | matrix = [[1, 2], [3, 4]] |
| **Using list()** | letters = list("ABC") |

# 9.1.2 Accessing Values

## Syntax

```
list_name[index]
```

## Explanation

1. List items are accessed using index numbers.
2. Python indexing starts from 0.
3. Positive indexing starts from the left.
4. Negative indexing starts from the right.

```
List:     ["Aman", "Riya", "Kabir", "Neha"]
Index:       0       1       2       3
Negative:   -4      -3      -2      -1
```

Example 1

```
students = ["Aman", "Riya", "Kabir", "Neha"]

print(students[0])
print(students[2])
print(students[-1])
```

**Output:**
Aman
Kabir
Neha

CodeWithNishchal

## Indexing Table

| Code | Meaning | Result |
|---|---|---|
| students[0] | First item | "Aman" |
| students[1] | Second item | "Riya" |
| students[-1] | Last item | "Neha" |
| students[-2] | Second last item | "Kabir" |

# 9.1.3 Updating Values

## Syntax

```
list_name[index] = new_value
```

1. Lists are mutable.
2. We can change an existing item using its index.
3. The index must exist in the list.

**Example 1**

```python
marks = [70, 80, 90]

marks[1] = 85

print(marks)
```

Output: [70, 85, 90]

## Updating Multiple Values

```
marks = [70, 80, 90, 60]

marks[1:3] = [88, 95]

print(marks)
```

**Output:** [70, 88, 95, 60]

CodeWithNishchal

# 9.1.4 List Methods

| Method | Purpose | Example | Result |
|---|---|---|---|
| append() | Adds one item at the end | items.append(40) | Adds 40 |
| extend() | Adds multiple items | items.extend([50, 60]) | Adds 50, 60 |
| insert() | Adds item at specific index | items.insert(1, 15) | Adds 15 at index 1 |
| remove() | Removes first matching value | items.remove(20) | Removes 20 |
| pop() | Removes item by index | items.pop(1) | Removes item at index 1 |
| clear() | Removes all items | items.clear() | Empty list |
| index() | Returns index of value | items.index(30) | Gives position |
| count() | Counts occurrences | items.count(10) | Count of 10 |
| sort() | Sorts list | items.sort() | Ascending order |
| reverse() | Reverses list | items.reverse() | Reverse order |
| copy() | Creates shallow copy | new = items.copy() | New list copy |

## Example 1

PYTHON CODE

```
numbers = [10, 20, 30]

numbers.append(40)
numbers.insert(1, 15)
numbers.remove(20)
print(numbers)
```

**Output:** [10, 15, 30, 40]

## Useful Functions with Lists

| Function | Purpose | Example |
|---|---|---|
| len() | Counts items | len(numbers) |
| sum() | Adds numeric items | sum(numbers) |
| min() | Smallest value | min(numbers) |
| max() | Largest value | max(numbers) |
| sorted() | Returns sorted copy | sorted(numbers) |

CodeWithNisharg

# 9.1.5 Slicing

## Syntax

list_name[start:end:step]

## Explanation

1. Slicing is used to get a part of a list.
2. The start index is included.
3. The end index is excluded.
4. Step controls the gap between selected items.

## Example 1

PYTHON CODE

```
numbers = [10, 20, 30, 40, 50, 60]
print(numbers[1:4])
print(numbers[:3])
print(numbers[3:])
print(numbers[::2])
print(numbers[::-1])
```

Output:
[20, 30, 40]
[10, 20, 30]
[40, 50, 60]
[10, 30, 50]
[60, 50, 40, 30, 20, 10]

## Slicing Table

| Code | Meaning | Result |
|---|---|---|
| numbers[1:4] | Index 1 to before 4 | [20, 30, 40] |
| numbers[:3] | Start to before index 3 | [10, 20, 30] |
| numbers[3:] | Index 3 to end | [40, 50, 60] |
| numbers[::2] | Every second item | [10, 30, 50] |
| numbers[::-1] | Reverse list | [60, 50, 40, 30, 20, 10] |

CodeWithNishchal

# 9.1.6 List Comprehensions

## Syntax

new_list = [expression for item in sequence]

**With condition:**

new_list = [expression for item in sequence if condition]

1. List comprehension is a short way to create a new list.
2. It is commonly used when each item needs to be processed.
3. It can also filter values using if.
4. It makes code shorter and cleaner.

## Example 1

PYTHON CODE

```
numbers = [1, 2, 3, 4, 5]
squares = [number * number for number in numbers]
print(squares)
```

**Output:** [1, 4, 9, 16, 25]

## Example 2: With Condition

PYTHON CODE

```
numbers = [1, 2, 3, 4, 5, 6]

even_numbers = [number for number in numbers if number % 2 == 0]
print(even_numbers)
```

**Output:** [2, 4, 6]

# 9.1.7 Nested Lists

## Syntax

list_name = [[item1, item2], [item3, item4]]

## Explanation

1. A nested list means a list inside another list.
2. It is useful for matrix-like data, rows and columns, or grouped values.
3. To access nested list values, use multiple indexes.

CodeWithNishchal

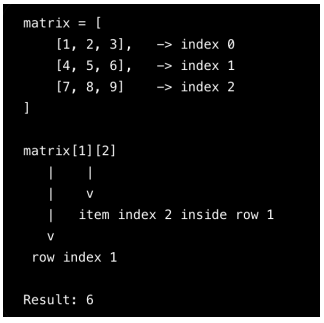
# Example 1

PYTHON CODE

```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
print(matrix[0])
print(matrix[1][2])
```

**Output:** [1, 2, 3]
6

# Index Diagram




### Updating Nested List Value

PYTHON CODE

```
matrix = [
    [1, 2, 3],
    [4, 5, 6]
]

matrix[1][0] = 40
print(matrix)
```

Output: [[1, 2, 3], [40, 5, 6]]

WithNishchal

# 9.1.8 Sorting and Reversing

Sorting means arranging values in order. Reversing means changing the order from last to first.

## Sorting and Reversing Table

| Operation | Code | Changes Original List? | Result |
|---|---|---|---|
| Sort ascending | numbers.sort() | Yes | Small to large |
| Sort descending | numbers.sort(reverse=True) | Yes | Large to small |
| Sorted copy | sorted(numbers) | No | Returns new sorted list |
| Reverse original | numbers.reverse() | Yes | Reverses same list |
| Reverse copy | numbers[::-1] | No | Returns reversed copy |

## Example 1

PYTHON CODE

```
numbers = [40, 10, 30, 20]
numbers.sort()
print(numbers)

numbers.sort(reverse=True)
print(numbers)

numbers.reverse()
print(numbers)
```

**Output:**
[10, 20, 30, 40]
[40, 30, 20, 10]
[10, 20, 30, 40]

WithNishchal

## sort() vs sorted()

| Point | sort() | sorted() |
|---|---|---|
| Type | List method | Built-in function |
| Original list | Changes original list | Does not change original list |
| Return value | Returns None | Returns new sorted list |
| Usage | numbers.sort() | sorted(numbers) |

## Example 2

PYTHON CODE

```
numbers = [30, 10, 20]
new_numbers =
sorted(numbers)
print(numbers)
print(new_numbers)
```

Output:
[30, 10, 20]
[10, 20, 30]

# 9.1.9 Copying Lists

## Syntax

new_list = old_list.copy()

**Other ways:**

new_list = old_list[:]

new_list = list(old_list)

## Explanation

1. Copying means creating another list with the same values.
2. Direct assignment does not create a real copy.
3. Direct assignment makes two variables point to the same list.
4. To create a separate list, use copy(), slicing, or list().

CodeWithNishchal

# Direct Assignment

PYTHON CODE

```
list1 = [10, 20, 30]
list2 = list1

list2.append(40)
print(list1)
print(list2)
```

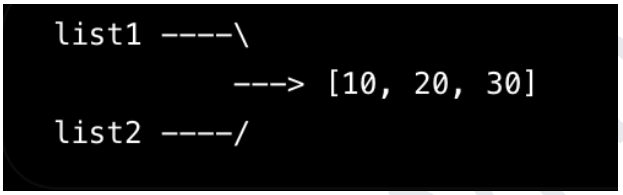
Output:

[10, 20, 30, 40]

[10, 20, 30, 40]

Explanation:

1. list2 = list1 does not create a new list.
2. Both variables refer to the same list.
3. So changing list2 also affects list1.




**Real Copy Example**

Python

```python
list1 = [10, 20, 30]
list2 = list1.copy()

list2.append(40)

print(list1)
print(list2)
```

**Output:**

[10, 20, 30]

[10, 20, 30, 40]

Nishchal

## Copying Methods Table

| Method | Code | Meaning |
|---|---|---|
| **copy()** | new = old.copy() | Creates shallow copy |
| **Slicing** | new = old[:] | Creates shallow copy |
| **list()** | new = list(old) | Creates shallow copy |
| **Direct assignment** | new = old | Not a real copy |

# 9.1.10 Shallow Copy vs Deep Copy

## Explanation

This concept matters when a list contains another list. There are two types of copy:

- **Shallow copy**
- **Deep copy**

## Shallow Copy

A shallow copy creates a new outer list, but nested lists inside are still shared.

## Example 1: Shallow Copy

```
PYTHON CODE

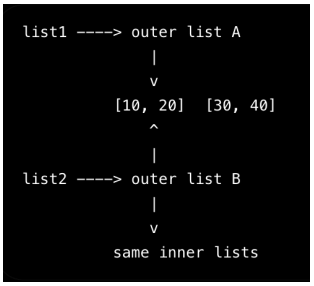
list1 = [[10, 20], [30, 40]]
list2 = list1.copy()
list2[0][0] = 99
print(list1)
print(list2)
```

Output:
[[99, 20], [30, 40]]
[[99, 20], [30, 40]]

1. list1.copy() creates a new outer list.
2. But the inner lists are still shared.
3. So changing an inner list affects both lists.

WithNishchal

## Shallow Copy Diagram




## Deep Copy

A deep copy creates a completely separate copy of both the outer list and inner lists.

## Example 2: Deep Copy

```
PYTHON CODE
import copy
list1 = [[10, 20], [30, 40]]
list2 = copy.deepcopy(list1)

list2[0][0] = 99
print(list1)
print(list2)
```

Output: [[10, 20], [30, 40]]
[[99, 20], [30, 40]]

**Explanation:**

1. copy.deepcopy() creates a fully independent copy.
2. Inner lists are also copied separately.
3. Changing list2 does not affect list1.

## Shallow Copy vs Deep Copy Table

| Point | Shallow Copy | Deep Copy |
|---|---|---|
| **Outer list** | New copy | New copy |
| **Inner nested lists** | Shared | New copy |
| **Affects original nested data?** | Yes | No |
| **Common method** | copy() | copy.deepcopy() |
| **Useful for** | Simple lists | Nested lists |

CodeWithNishchal

# Important List Operations Table

| Operation | Code | Meaning |
|---|---|---|
| **Create list** | items = [10, 20, 30] | Creates list |
| **Access item** | items[0] | Gets first item |
| **Update item** | items[1] = 25 | Changes second item |
| **Add item** | items.append(40) | Adds at end |
| **Insert item** | items.insert(1, 15) | Adds at index |
| **Remove item** | items.remove(20) | Removes value |
| **Remove by index** | items.pop(0) | Removes by index |
| **Slice list** | items[1:3] | Gets part of list |
| **Sort list** | items.sort() | Sorts original list |
| **Reverse list** | items.reverse() | Reverses original list |
| **Copy list** | items.copy() | Creates shallow copy |
| **Length** | len(items) | Counts items |

# List Method Return Behavior

| Method | Changes Original List? | Returns |
|---|---|---|
| append() | Yes | None |
| extend() | Yes | None |
| insert() | Yes | None |
| remove() | Yes | None |
| pop() | Yes | Removed item |
| clear() | Yes | None |
| sort() | Yes | None |
| reverse() | Yes | None |
| copy() | No | New list |

CodeWithNishchal

# 9.2 Tuples in Python

A **tuple** is a data structure used to store multiple values in one variable.

Tuples are similar to lists, but the main difference is:

**List -> mutable -> can be changed**

**Tuple -> immutable -> cannot be changed**

## Syntax

```
tuple_name = (value1, value2, value3)
```

## Explanation

1. Tuples are written using parentheses ().
2. Tuple items are separated by commas.
3. Tuples are ordered, so items have index positions.
4. Tuples are immutable, so items cannot be changed after creation.
5. Tuples allow duplicate values.
6. Tuples can store different data types.

## Example 1

```
student = ("Rahul", 21, "Python", 85.5)

print(student)
```

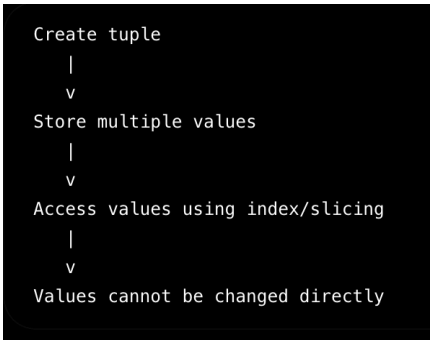
**Output:** ('Rahul', 21, 'Python', 85.5)

## Tuple Properties Table

| Property | Meaning | Example |
|---|---|---|
| Ordered | Items have fixed positions | items[0] |
| Immutable | Items cannot be changed | items[1] = 50 not allowed |
| Allows duplicates | Same value can appear more than once | (10, 10, 20) |
| Mixed data allowed | Can store different data types | ("Aman", 20, True) |
| Indexed | Every item has position number | 0, 1, 2... |

CodewithNishchal

## Flow Chart




# 9.2.1 Creating Tuples

## Syntax

tuple_name = (item1, item2, item3)

1. Tuples can store numbers, strings, Booleans, or mixed values. An empty tuple can also be created.
2. A tuple can be created with or without parentheses, but parentheses are recommended for clarity.
3. A single-item tuple must have a comma.

## Example 1

PYTHON CODE

```
numbers = (10, 20, 30)
names = ("Aman", "Riya", "Kabir")
mixed = ("Python", 100, 99.5, True)
empty_tuple = ()
print(numbers)
print(names)
print(mixed)
print(empty_tuple)
```

**Output:**

```
(10, 20, 30)
('Aman', 'Riya', 'Kabir')
('Python', 100, 99.5, True)
()
```

Vith Nishchal

## Different Ways to Create Tuples

| Type | Example |
|---|---|
| Empty tuple | items = () |
| Number tuple | marks = (80, 90, 75) |
| String tuple | names = ("Aman", "Riya") |
| Mixed tuple | data = ("Aman", 20, True) |
| Nested tuple | matrix = ((1, 2), (3, 4)) |
| Without parentheses | items = 10, 20, 30 |
| Using tuple() | letters = tuple("ABC") |

# 9.2.2 Single-Item Tuple

## Syntax

tuple_name = (value,)

## Explanation

1. A single-item tuple must contain a comma.
2. Without the comma, Python does not treat it as a tuple.
3. Parentheses alone are not enough.

## Example 1

PYTHON CODE

```
a = ("Python")
b = ("Python",)
print(type(a))
print(type(b))
```

**Output:**

```
<class 'str'>
<class 'tuple'>
```

CodeWithNishchal

# 9.2.3 Accessing Tuple Values

## Syntax

tuple_name[index]

1. Tuple values are accessed using index numbers.
2. Python indexing starts from 0.
3. Positive indexing starts from the left.
4. Negative indexing starts from the right.

Index Diagram

```
Tuple:    ("Aman", "Riya", "Kabir", "Neha")
Index:       0       1       2        3
Negative:   -4      -3      -2       -1
```

Example 1

```python
students = ("Aman", "Riya", "Kabir", "Neha")
print(students[0])
print(students[2])
print(students[-1])
```

**Output:**
Aman
Kabir
Neha

# 9.2.4 Tuple Slicing

## Syntax

tuple_name[start:end:step]

## Explanation

1. Slicing is used to get a part of a tuple.
2. The start index is included.
3. The end index is excluded.
4. Step controls the gap between selected values.
5. Slicing returns a new tuple.

CodeWithNishchal

## Example 1

```python
numbers = (10, 20, 30, 40, 50, 60)

print(numbers[1:4])
print(numbers[:3])
print(numbers[3:])
print(numbers[::2])
print(numbers[::-1])
```

**Output:**

(20, 30, 40)
(10, 20, 30)
(40, 50, 60)
(10, 30, 50)
(60, 50, 40, 30, 20, 10)

## Slicing Table

| Code | Meaning | Result |
|---|---|---|
| numbers[1:4] | Index 1 to before 4 | (20, 30, 40) |
| numbers[:3] | Start to before index 3 | (10, 20, 30) |
| numbers[3:] | Index 3 to end | (40, 50, 60) |
| numbers[::2] | Every second item | (10, 30, 50) |
| numbers[::-1] | Reverse tuple | (60, 50, 40, 30, 20, 10) |

# 9.2.5 Tuple Immutability

## Syntax

tuple_name[index] = new_value

This syntax is **not allowed** for tuples.

## Explanation

1. Tuples are immutable.
2. Once a tuple is created, its values cannot be changed directly.
3. We cannot update, add, or remove tuple items directly.
4. If changes are needed, use a list or create a new tuple.

CodeWithNishchal

## Example 1

```
PYTHON CODE
numbers = (10, 20, 30)
numbers[1] = 25
```

**Output:** TypeError: 'tuple' object does not support item assignment

## Correct Way

Create a new tuple:

```
PYTHON CODE
numbers = (10, 20, 30)
numbers = (10, 25, 30)
print(numbers)
```

**Output:** (10, 25, 30)

## Memory Idea

**Before:** numbers ----> (10, 20, 30)
**After:** numbers ----> (10, 25, 30)
The old tuple is not changed. The variable starts pointing to a new tuple.

# 9.2.6 Tuple with Mutable Items

## Syntax

tuple_name = ([item1, item2], value)

1. A tuple itself is immutable.
2. But if a tuple contains a mutable item like a list, the inner list can be changed.
3. This happens because the tuple is still pointing to the same inner list.

## Example 1

```
PYTHON CODE
data = ([10, 20], "Python")
data[0][1] = 99
print(data)
```

Output: ([10, 99], 'Python')

CodeWithNishchal

## Explanation Table

| Part | Mutable or Immutable? |
|---|---|
| Outer tuple | Immutable |
| Inner list | Mutable |
| data[0] = [1, 2] | Not allowed |
| data[0][1] = 99 | Allowed |

# 9.2.7 Tuple Methods

| Method | Purpose | Example | Result |
|---|---|---|---|
| count() | Counts how many times a value appears | items.count(10) | Count of 10 |
| index() | Returns index of first matching value | items.index(20) | Position of 20 |

## Useful Functions with Tuples

| Function | Purpose | Example |
|---|---|---|
| len() | Counts items | len(numbers) |
| sum() | Adds numeric items | sum(numbers) |
| min() | Smallest value | min(numbers) |
| max() | Largest value | max(numbers) |
| sorted() | Returns sorted list | sorted(numbers) |

# 9.2.8 Tuple Operations

| Operation | Code | Meaning |
|---|---|---|
| **Concatenation** | tuple1 + tuple2 | Joins tuples |
| **Repetition** | tuple1 * 3 | Repeats tuple |
| **Membership** | value in tuple1 | Checks value exists |
| **Length** | len(tuple1) | Counts items |
| **Slicing** | tuple1[1:4] | Gets part of tuple |

CodeWithNishchal

# 9.2.9 Tuple Packing

## Syntax

tuple_name = value1, value2, value3

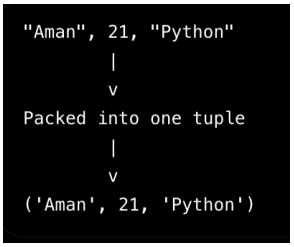
## Explanation

1. Tuple packing means storing multiple values together in a tuple.
2. Python automatically packs comma-separated values into a tuple.
3. Parentheses are optional, but recommended for readability.

## Example 1

```
PYTHON CODE
student = "Aman", 21, "Python"
print(student)
print(type(student))
```

**Output:** ('Aman', 21, 'Python')
<class 'tuple'>




# 9.2.10 Tuple Unpacking

## Syntax

var1, var2, var3 = tuple_name

## Explanation

1. Tuple unpacking means taking values from a tuple and storing them in separate variables.
2. The number of variables should match the number of tuple values.
3. Unpacking makes code cleaner and readable.

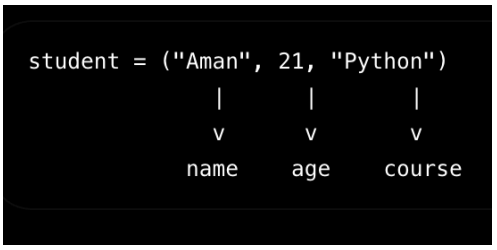
CodeWithNishchal

## Example 1

```
student = ("Aman", 21, "Python")

name, age, course = student

print(name)
print(age)
print(course)
```




Output:
Aman
21
Python

## Important Point

This gives an error:

```
PYTHON CODE
student = ("Aman", 21, "Python")
name, age = student
```

**Output:** ValueError: too many values to unpack

Because the tuple has 3 values, but only 2 variables are given.

# 9.2.11 Extended Tuple Unpacking

## Syntax

first, *middle, last = tuple_name

## Explanation

1. Extended unpacking is used when we do not want to manually create variables for every value.
2. The starred variable collects extra values.
3. The starred variable becomes a list.

CodeWithNishchal

## Example 1

PYTHON CODE

```
numbers = (10, 20, 30, 40, 50)
first, *middle, last = numbers
print(first)
print(middle)
print(last)
```

**Output:**
10
[20, 30, 40]
50

**Important point:** The starred variable stores values in a list, not a tuple.

# 9.2.12 Named Tuples

## Syntax

from collections import namedtuple

**TupleName = namedtuple("TupleName", ["field1", "field2"])**

**object_name = TupleName(value1, value2)**

## Explanation

1. A named tuple is a tuple with named fields.
2. Normal tuple values are accessed by index.
3. Named tuple values can be accessed by name.
4. This makes code more readable.
5. Named tuples are immutable like normal tuples.

## Example 1

```
from collections import namedtuple

Student = namedtuple("Student", ["name", "age", "course"])

student1 = Student("Aman", 21, "Python")

print(student1.name)
print(student1.age)
print(student1.course)
```

**Output:** Aman
21
Python

CodeWithNishchal

## Normal Tuple vs Named Tuple

| Normal Tuple | Named Tuple |
|---|---|
| Access by index | Access by name |
| student[0] | student.name |
| Less readable | More readable |
| Immutable | Immutable |

# 9.2.13 Tuple vs List

| Point | List | Tuple |
|---|---|---|
| **Brackets** | Uses [] | Uses () |
| **Mutability** | Mutable | Immutable |
| **Can update items** | Yes | No |
| **Can add/remove items** | Yes | No |
| **Speed** | Slightly slower | Slightly faster |
| **Best for** | Data that may change | Data that should not change |

# Important Tuple Operations Table

| Operation | Code | Meaning |
|---|---|---|
| Create tuple | items = (10, 20, 30) | Creates tuple |
| Single-item tuple | item = (10,) | Creates tuple with one item |
| Access item | items[0] | Gets first item |
| Slice tuple | items[1:3] | Gets part of tuple |
| Count item | items.count(10) | Counts value |
| Find index | items.index(20) | Finds position |
| Pack tuple | data = "Aman", 21 | Packs values |
| Unpack tuple | name, age = data | Extracts values |
| Concatenate | a + b | Joins tuples |
| Repeat | a * 2 | Repeats tuple |
| Check membership | 10 in items | Returns True or False |
| Convert list to tuple | tuple(my_list) | Creates tuple |
| Convert tuple to list | list(my_tuple) | Creates list |

CodeWithNishchal

# 9.3 Dictionaries in Python

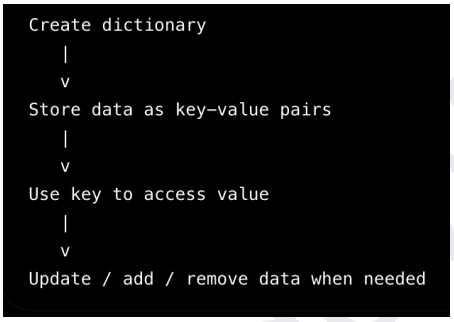
A **dictionary** is a data structure used to store data in **key-value pairs**.

## Syntax

```
dictionary_name = {
    key1: value1,
    key2: value2,
    key3: value3
}
```

1. A dictionary stores data using **keys** and **values**.
2. Each key is connected to one value.
3. Keys are used to access values.
4. Dictionaries are mutable, so values can be changed.
5. Dictionary keys must be unique.
6. Dictionaries are written using curly braces {}.

## Flow Chart




**Example 1**

PYTHON CODE

```
student = {
"name": "Aman",
"age": 21,
"course": "Python"
}
print(student)
```

**Output:** {'name': 'Aman', 'age': 21, 'course': 'Python'}

Code With Nishchal

## Dictionary Structure

```
Dictionary
   |
   v
{
  "name": "Aman",
   key      value

  "age": 21,
   key   value
}
```

## Dictionary Properties Table

| Property | Meaning | Example |
|---|---|---|
| Key-value based | Stores data as pairs | "name": "Aman" |
| Mutable | Can be changed | student["age"] = 22 |
| Ordered | Keeps insertion order | Python 3.7+ |
| Unique keys | Duplicate keys are not allowed | Last value replaces old value |
| Fast lookup | Values are accessed using keys | student["name"] |
| Mixed values allowed | Values can be any data type | string, int, list, tuple, dict |

# 9.3.1 Creating Dictionaries

## Syntax

```python
dictionary_name = {
    "key1": "value1",
    "key2": "value2"
}
```

## Explanation

1. Dictionaries are created using {}.
2. Keys and values are separated using a colon :.
3. Each pair is separated using a comma.
4. Keys are usually strings, but numbers and tuples can also be used.
5. Values can be any data type.

CodeWithNishchal

PYTHON CODE

```
student = {
    "name": "Rahul",
    "age": 20,
    "marks": 85.5,
    "is_passed": True
}
print(student)
```

**Output:** {'name': 'Rahul', 'age': 20, 'marks': 85.5, 'is_passed': True}

## Different Ways to Create Dictionaries

| Type | Example |
|---|---|
| **Empty dictionary** | data = {} |
| **Normal dictionary** | student = {"name": "Aman", "age": 21} |
| **Using dict()** | student = dict(name="Aman", age=21) |
| **Nested dictionary** | students = {"s1": {"name": "Aman"}} |
| **Dictionary with list value** | data = {"marks": [80, 90, 85]} |
| **Dictionary with tuple key** | points = {(10, 20): "A"} |

# 9.3.2 Dictionary Keys and Values

## Syntax

```
dictionary = {
    key: value
}
```

## Explanation

1. A **key** is used to identify a value.
2. A **value** is the data stored against the key.
3. Keys must be unique.
4. Values can be duplicate.
5. Keys should be immutable types like string, number, or tuple.

Code with Nishchal

```python
student = {
    "name": "Aman",
    "age": 21,
    "course": "Python"
}

print(student["name"])
```

**Output:** Aman

## Key Rules Table

| Rule | Allowed? | Example |
|---|---|---|
| String key | Yes | "name": "Aman" |
| Number key | Yes | 1: "One" |
| Tuple key | Yes | (10, 20): "Point" |
| List key | No | [1, 2]: "Value" |
| Duplicate keys | Not useful | Last value is kept |

## Duplicate Key Example

```python
student = {
    "name": "Aman",
    "name": "Rahul"
}

print(student)
```

**Output:** {'name': 'Rahul'}

The second value replaces the first value because dictionary keys must be unique.

---

CodeWithNishchal

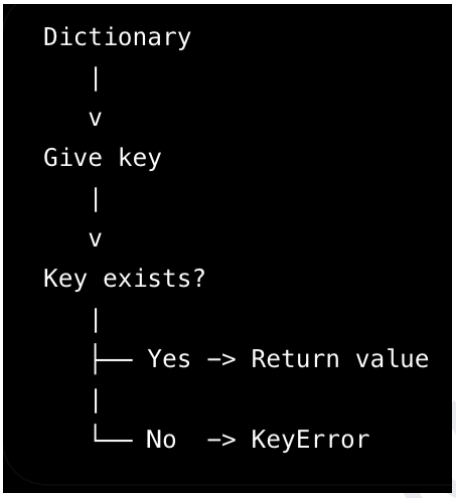
# 9.3.3 Accessing Values

## Syntax

dictionary_name[key]

## Explanation

1. Dictionary values are accessed using keys.
2. Unlike lists and tuples, dictionaries are not accessed mainly by index.
3. If the key exists, Python returns its value.
4. If the key does not exist, Python gives KeyError.




Example 1

```python
student = {
    "name": "Aman",
    "age": 21,
    "course": "Python"
}

print(student["name"])
print(student["age"])
```

**Output:**
Aman
21

CodeWithNishchal

## Accessing Table

| Code | Meaning | Result |
|---|---|---|
| student["name"] | Access name | "Aman" |
| student["age"] | Access age | 21 |
| student["course"] | Access course | "Python" |

## KeyError Example

```
PYTHON CODE
student = {
"name": "Aman",
"age": 21
}
print(student["marks"])
```

**Output:**
KeyError: 'marks'
The key "marks" does not exist.

# 9.3.4 Updating Values

## Syntax

dictionary_name[key] = new_value

1. Dictionaries are mutable.
2. Existing values can be updated using keys.
3. If the key already exists, its value is updated.
4. If the key does not exist, a new key-value pair is added.

```
PYTHON CODE
student = {
    "name": "Aman",
    "age": 21
}
student["age"] = 22
student["course"] = "Python"
print(student)
```

Output: {'name': 'Aman', 'age': 22, 'course': 'Python'}

CodeWithNishchal

# 9.3.5 Dictionary Methods

Dictionary methods are built-in operations used to access, update, remove, copy, and manage dictionary data.

Assume this dictionary:

```python
student = {
    "name": "Aman",
    "age": 21,
    "course": "Python"
}
```

## Dictionary Methods Table

| Method | Purpose | Example | Result / Effect |
|---|---|---|---|
| keys() | Returns all keys | student.keys() | dict_keys(['name', 'age', 'course']) |
| values() | Returns all values | student.values() | dict_values(['Aman', 21, 'Python']) |
| items() | Returns key-value pairs | student.items() | dict_items([('name','Aman'),...]) |
| get() | Returns value safely | student.get("name") | "Aman" |
| update() | Adds or updates data | student.update({"age": 22}) | Updates age |
| pop() | Removes key and returns value | student.pop("age") | Returns 21 |
| popitem() | Removes last inserted pair | student.popitem() | Removes last pair |
| clear() | Removes all items | student.clear() | {} |
| copy() | Creates shallow copy | new = student.copy() | New dictionary |
| setdefault() | Gets value or adds default | student.setdefault("city", "Delhi") | Adds city if missing |
| fromkeys() | Creates dictionary from keys | dict.fromkeys(["a","b"], 0) | {'a': 0, 'b': 0} |

CodeWithNishchal

## Method Behavior Table

| Method | Changes Original Dictionary? | Returns |
|---|---|---|
| keys() | No | View of keys |
| values() | No | View of values |
| items() | No | View of key-value pairs |
| get() | No | Value or default |
| update() | Yes | None |
| pop() | Yes | Removed value |
| popitem() | Yes | Removed key-value pair |
| clear() | Yes | None |
| copy() | No | New shallow copy |
| setdefault() | May change | Existing/default value |
| fromkeys() | Creates new dictionary | New dictionary |

# 9.3.6 get() Method

## Syntax

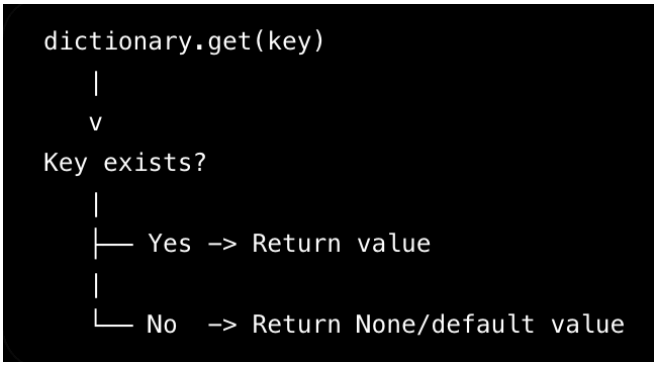
```python
dictionary_name.get(key)
dictionary_name.get(key, default_value)
```

## Explanation

1. get() is used to access dictionary values safely.
2. If the key exists, it returns the value.
3. If the key does not exist, it returns None by default.
4. We can also provide our own default value.
5. get() avoids KeyError.

CodeWithNishchal

## Flow Chart




Example 1

PYTHON CODE

```
student = {
"name": "Aman",
"age": 21
}
print(student.get("name"))
print(student.get("marks"))
print(student.get("marks", "Not available"))
```

**Output:**
Aman
None
Not available

## [] vs get() Table

| Access Method | If Key Exists | If Key Missing |
|---|---|---|
| student["name"] | Returns value | Gives KeyError |
| student.get("name") | Returns value | Returns None |
| student.get("marks", 0) | Returns value | Returns default value |

CodeWithNishchal

# 9.3.7 Removing Dictionary Items

## Syntax

dictionary.pop(key)
dictionary.popitem()
del dictionary[key]
dictionary.clear()

## Explanation

1. pop(key) removes a specific key and returns its value.
2. popitem() removes the last inserted key-value pair.
3. del removes a specific key.
4. clear() removes all items.

## Removing Items Table

| Method / Keyword | Purpose | Example |
|---|---|---|
| pop(key) | Removes specific key | student.pop("age") |
| popitem() | Removes last pair | student.popitem() |
| del | Deletes specific key | del student["age"] |
| clear() | Empties dictionary | student.clear() |

## Example 1

PYTHON CODE

```
student = {
    "name": "Aman",
    "age": 21,
    "course": "Python"
}
student.pop("age")
print(student)
```

Output: {'name': 'Aman', 'course': 'Python'}

deWithNishchal

# 9.3.8 Looping Through Dictionaries

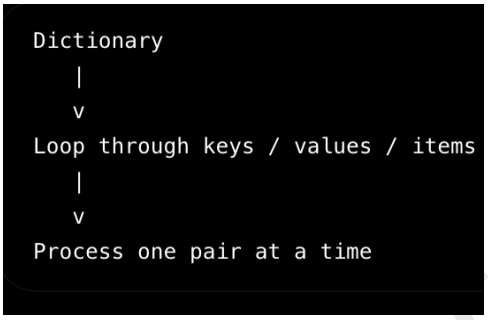
## Syntax

```python
for key in dictionary:
    statement
```

```python
for key, value in dictionary.items():
    statement
```

## Explanation

1. A dictionary can be looped through using for.
2. By default, looping over a dictionary gives keys.
3. Use values() to loop through values.
4. Use items() to loop through both keys and values.




Example 1

PYTHON CODE

```
student = {
    "name": "Aman",
    "age": 21,
    "course": "Python"
}
for key, value in student.items():
    print(key, value)
```

**Output:**

name Aman
age 21
course Python

WithNishchal

# 9.3.9 Checking Key Membership

## Syntax

```
key in dictionary
key not in dictionary
```

## Explanation

1. in checks whether a key exists in the dictionary.
2. It checks keys, not values.
3. It returns True or False.

## Example 1

```python
student = {
    "name": "Aman",
    "age": 21
}

print("name" in student)
print("marks" in student)
```

Output:
True
False

## Membership Table

| Code | Meaning |
|---|---|
| "name" in student | Checks if key exists |
| "marks" not in student | Checks if key does not exist |
| "Aman" in student | Checks key, not value |

CodeWithNishchal

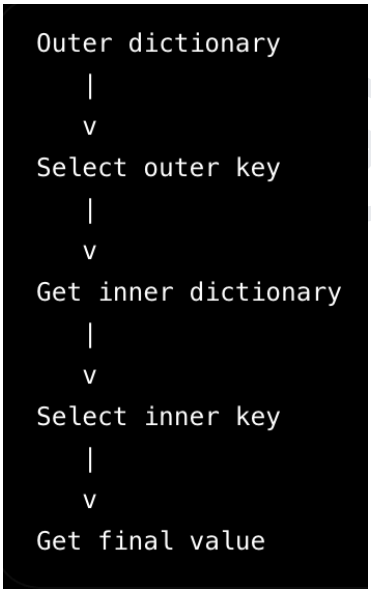
# 9.3.10 Nested Dictionaries

## Syntax

```python
dictionary_name = {
    key1: {
        inner_key1: inner_value1,
        inner_key2: inner_value2
    },
    key2: {
        inner_key1: inner_value1,
        inner_key2: inner_value2
    }
}
```

## Explanation

1. A nested dictionary means a dictionary inside another dictionary.
2. It is useful for storing structured data.
3. To access inner values, use multiple keys.
4. Nested dictionaries are common in real-world data like users, students, products, and API responses.



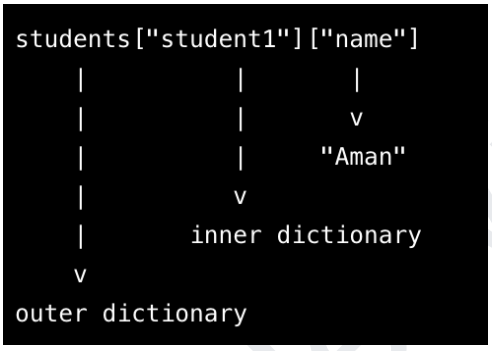

eWith Nishchal

Example 1

PYTHON CODE

```
students = {
    "student1": {
        "name": "Aman",
        "age": 21
    },
    "student2": {
        "name": "Riya",
        "age": 20
    }
}
print(students["student1"]["name"])
print(students["student2"]["age"])
```

Output:
Aman
20




**Updating Nested Dictionary**

Python

```
students = {
    "student1": {
        "name": "Aman",
        "age": 21
    }
}

students["student1"]["age"] = 22

print(students)
```

Output: {'student1': {'name': 'Aman', 'age': 22}}

devWithNishchal

# 9.3.11 Dictionary Comprehensions

## Syntax

```
new_dict = {key_expression: value_expression for item in sequence}
```

**With condition:**

```
new_dict = {key_expression: value_expression for item in sequence if condition}
```

## Explanation

1. Dictionary comprehension is a short way to create a dictionary.
2. It is similar to list comprehension.
3. It creates key-value pairs using a loop.
4. It can also include conditions.

## Flow Chart

Take sequence
|
v
Loop through items
|
v
Create key-value pair
|
v
Store in new dictionary

**Example 1**

PYTHON CODE

```
numbers = [1, 2, 3, 4]
squares = {number: number * number for number in numbers}
print(squares)
```

Output:

{1: 1, 2: 4, 3: 9, 4: 16}

CodeWithNishchal

## Example 2: With Condition

PYTHON CODE

```
numbers = [1, 2, 3, 4, 5, 6]
even_squares = {number: number * number for number in numbers if number % 2 == 0}
print(even_squares)
```

Output: {2: 4, 4: 16, 6: 36}

### Dictionary Comprehension Parts

| Part | Meaning |
|---|---|
| number before : | Key |
| number * number | Value |
| for number in numbers | Loop |
| if number % 2 == 0 | Optional condition |

# 9.3.12 Merging Dictionaries

Merging means combining two or more dictionaries.

## Example 1: Using update()

PYTHON CODE

```
student = {
    "name": "Aman",
    "age": 21
}
extra = {
    "course": "Python",
    "city": "Delhi"
}
student.update(extra)
print(student)
```

Output: {'name': 'Aman', 'age': 21, 'course': 'Python', 'city': 'Delhi'}

With Nishchal

## Example 2: Using |

PYTHON CODE

```
student = {
    "name": "Aman",
    "age": 21
}
extra = {
    "course": "Python",
    "city": "Delhi"
}
result = student | extra
print(result)
```

Output: {'name': 'Aman', 'age': 21, 'course': 'Python', 'city': 'Delhi'}

## Same Key During Merge

If both dictionaries have the same key, the second dictionary value wins.

PYTHON CODE

```
a = {"name": "Aman", "age": 21}
b = {"age": 22, "course": "Python"}
result = a | b
print(result)
```

Output: {'name': 'Aman', 'age': 22, 'course': 'Python'}

# 9.3.13 Copying Dictionaries

## Syntax

new_dict = old_dict.copy()

## Explanation

1. copy() creates a shallow copy of a dictionary.
2. Direct assignment does not create a new dictionary.
3. Direct assignment makes both variables point to the same dictionary.

Coo hNishchal

## Direct Assignment Example

PYTHON CODE

```
student1 = {"name": "Aman", "age": 21}
student2 = student1

student2["age"] = 22
print(student1)
print(student2)
```

Output:
{'name': 'Aman', 'age': 22}
{'name': 'Aman', 'age': 22}

## Real Copy Example

PYTHON CODE

```
student1 = {"name": "Aman", "age": 21}
student2 = student1.copy()

student2["age"] = 22
print(student1)
print(student2)
```

**Output:**

{'name': 'Aman', 'age': 21}

{'name': 'Aman', 'age': 22}

## Copying Table

| Method | Creates New Dictionary? | Notes |
|---|---|---|
| dict2 = dict1 | No | Same dictionary reference |
| dict2 = dict1.copy() | Yes | Shallow copy |
| dict2 = dict(dict1) | Yes | Shallow copy |

Codev Nishchal

# 9.3.14 Dictionary vs List

| Point | List | Dictionary |
|---|---|---|
| **Data storage** | Stores values | Stores key-value pairs |
| **Access by** | Index | Key |
| **Brackets** | [] | {} |
| **Best for** | Ordered collection of values | Labeled/structured data |
| **Example** | ["Aman", 21] | {"name": "Aman", "age": 21} |

# 9.3.15 Important Dictionary Operations

| Operation | Code | Meaning |
|---|---|---|
| Create dictionary | data = {"name": "Aman"} | Creates dictionary |
| Access value | data["name"] | Gets value by key |
| Safe access | data.get("name") | Avoids KeyError |
| Update value | data["name"] = "Rahul" | Changes value |
| Add new pair | data["age"] = 21 | Adds key-value pair |
| Remove key | data.pop("age") | Removes key |
| Check key | "name" in data | Returns True or False |
| Get keys | data.keys() | Returns all keys |
| Get values | data.values() | Returns all values |
| Get pairs | data.items() | Returns key-value pairs |
| Merge dictionaries | `dict1 | dict2` |
| Copy dictionary | data.copy() | Creates shallow copy |
| Empty dictionary | data.clear() | Removes all items |

CodeWithNishchal

# 9.4 Sets in Python

A **set** is a data structure used to store multiple **unique** values.

## Syntax

```python
set_name = {value1, value2, value3}
```

## Explanation

1. Sets are written using curly braces {}.
2. Set items are separated by commas.
3. Sets store only unique values.
4. Sets are unordered, so items do not have fixed positions.
5. Sets are mutable, so we can add or remove items.
6. Set elements must be immutable/hashable values.

### Example 1

PYTHON CODE

```
numbers = {10, 20, 30, 40}
print(numbers)
```

{40, 10, 20, 30} : The output order may look different because sets are unordered.

## Set Properties Table

| Property | Meaning | Example |
|---|---|---|
| Unordered | Items have no fixed index | Cannot use items[0] |
| Unique values | Duplicate values are removed | {10, 10, 20} becomes {10, 20} |
| Mutable | Items can be added/removed | items.add(50) |
| Unindexed | No indexing/slicing | items[1] not allowed |
| Mixed data allowed | Can store different immutable types | {10, "Aman", True} |
| Fast membership check | Good for checking existence | 10 in numbers |

Codevi Nishchal

# 9.4.1 Creating Sets

## Syntax

set_name = {item1, item2, item3}

1. Sets can store numbers, strings, Booleans, and tuples.
2. Sets cannot store mutable values like lists or dictionaries.
3. Empty set must be created using set(). {} creates an empty dictionary, not an empty set.

## Example 1

PYTHON CODE

```
numbers = {10, 20, 30}
names = {"Aman", "Riya", "Kabir"}
mixed = {"Python", 100, 99.5, True}
empty_set = set()

print(numbers)
print(names)
print(mixed)
print(empty_set)
```

Output:
{10, 20, 30}
{'Aman', 'Kabir', 'Riya'}
{True, 99.5, 100, 'Python'}
set()
Order may differ in output.

## Different Ways to Create Sets

| Type | Example |
|---|---|
| Empty set | items = set() |
| Number set | marks = {80, 90, 75} |
| String set | names = {"Aman", "Riya"} |
| Mixed set | data = {"Aman", 20, True} |
| From list | items = set([10, 20, 10]) |
| From string | letters = set("ABC") |
| From tuple | items = set((10, 20, 30)) |

CodeWithNishchal

## Empty Set Important Point

```
PYTHON CODE

a = {}
b = set()
print(type(a))
print(type(b))
```

**Output:**
<class 'dict'>
<class 'set'>

# 9.4.2 Unique Values

## Syntax

set_name = {value1, value2, value1}

1. Sets automatically remove duplicate values.
2. Each value appears only once.
3. This makes sets useful for removing duplicates from data.
4. Sets check uniqueness using the value, not position.

## Example 1

```
PYTHON CODE
numbers = {10, 20, 10, 30, 20, 40}
print(numbers)
```

**Output:** {40, 10, 20, 30}  =>  Duplicate 10 and 20 are removed.

## Removing Duplicates from a List

```
PYTHON CODE
numbers = [10, 20, 10, 30, 20, 40]
unique_numbers = set(numbers)
print(unique_numbers)
```

**Output:** {40, 10, 20, 30}

Code With Nishchal

# 9.4.3 Set Elements Must Be Immutable

## Syntax

set_name = {immutable_value1, immutable_value2}

## Explanation

1. Set elements must be hashable.
2. Immutable values like numbers, strings, and tuples can be stored in a set.
3. Mutable values like lists, dictionaries, and sets cannot be stored in a set.
4. This is because sets internally need stable values to check uniqueness.

## Example 1

```
PYTHON CODE
valid_set = {10, "Python", (1, 2)}
print(valid_set)
```

**Output:** {10, 'Python', (1, 2)}

**Invalid example:**

```
PYTHON CODE
invalid_set = {[1, 2], [3, 4]}
```

Output: TypeError: unhashable type: 'list'

## Allowed vs Not Allowed Table

| Value Type | Allowed in Set? | Example |
|---|---|---|
| Integer | Yes | {10, 20} |
| Float | Yes | {10.5, 20.5} |
| String | Yes | {"A", "B"} |
| Boolean | Yes | {True, False} |
| Tuple | Yes, if tuple contains immutable items | {(1, 2)} |
| List | No | {[1, 2]} |
| Dictionary | No | {{"a": 1}} |
| Set | No | {{1, 2}} |

CodeWithNishchal

# 9.4.4 Accessing Set Values

## Syntax

```
for item in set_name:
    statement
```

## Explanation

1. Sets do not support indexing.
2. Sets do not support slicing.
3. Values can be accessed by looping.
4. Membership can be checked using in.

## Example 1

PYTHON CODE

```
languages = {"Python", "Java", "C++"}
for language in languages:
    print(language)
```

**Possible output:**
Java
Python
C++
Output order may differ.

## Invalid Access:

PYTHON CODE

```
languages = {"Python", "Java", "C++"}
print(languages[0])
```

**Output:** TypeError: 'set' object is not subscriptable

## Accessing Table

| Operation | Allowed? | Example |
|---|---|---|
| Looping | Yes | for item in items: |
| Membership check | Yes | "Python" in items |
| Indexing | No | items[0] |
| Slicing | No | items[1:3] |

CodeWithNishchal

# 9.4.5 Checking Membership

## Syntax

```
value in set_name
value not in set_name
```

## Explanation

1. Sets are very useful for checking whether a value exists.
2. in returns True if value exists.
3. not in returns True if value does not exist.
4. Membership checking in sets is usually faster than lists for large data.

**Example:**

```
numbers = {10, 20, 30, 40}

print(20 in numbers)
print(50 in numbers)
print(50 not in numbers)
```

**Output:**
True
False
True

# 9.4.6 Adding Set Items

## Syntax

```
Python

set_name.add(value)
set_name.update(iterable)
```

## Explanation

1. add() adds one item.
2. update() adds multiple items.
3. If an added value already exists, it is not added again.
4. Sets automatically maintain uniqueness.

CodeWithNishchal

## Example 1

```
PYTHON CODE
numbers = {10, 20, 30}
numbers.add(40)
numbers.update([50, 60, 20])
print(numbers)
```

**Output:** {40, 10, 50, 20, 60, 30}

20 was already present, so it is not duplicated.

## add() vs update()

| Method | Adds | Example |
|---|---|---|
| add() | One item | items.add(10) |
| update() | Multiple items | items.update([10, 20]) |

# 9.4.7 Removing Set Items

```
PYTHON CODE
set_name.remove(value)
set_name.discard(value)
set_name.pop()
set_name.clear()
```

1. remove() removes a specific item.
2. discard() also removes a specific item.
3. pop() removes a random item because sets are unordered.
4. clear() removes all items.

**Example:**

```
PYTHON CODE
numbers = {10, 20, 30, 40}
numbers.remove(20)
numbers.discard(50)
print(numbers)
```

**Output:** {40, 10, 30}

discard(50) does not give an error even though 50 is missing.

Code Nishchal

## Removing Methods Table

| Method | Purpose | If Value Missing |
|---|---|---|
| remove(value) | Removes specific value | Gives KeyError |
| discard(value) | Removes specific value | No error |
| pop() | Removes random item | Gives error if set is empty |
| clear() | Removes all items | No error |

# 9.4.8 Set Operations

Set operations are used to compare or combine sets.
**Main set operations:**

1. Union
2. Intersection
3. Difference
4. Symmetric difference
5. Subset
6. Superset
7. Disjoint

Assume:
a = {1, 2, 3}
b = {3, 4, 5}

## Set Operations Table

| Operation | Symbol | Method | Meaning | Result |
|---|---|---|---|---|
| Union | `a | b` | a.union(b) | All unique values from both sets | |
| Intersection | a & b | a.intersection(b) | Common values | {3} |
| Difference | a - b | a.difference(b) | Values in a but not in b | {1, 2} |
| Symmetric difference | a ^ b | a.symmetric_difference(b) | Values not common in both | {1, 2, 4, 5} |
| Subset | a <= b | a.issubset(b) | Checks if all items of a are in b | False |
| Superset | a >= b | a.issuperset(b) | Checks if a contains all items of b | False |
| Disjoint | No symbol | a.isdisjoint(b) | Checks if no common items exist | False |

CodeWithNishchal

## Example 1

```
PYTHON CODE
a = {1, 2, 3}
b = {3, 4, 5}
print(a | b)
print(a & b)
print(a - b)
print(a ^ b)
```

**Output:**
{1, 2, 3, 4, 5}
{3}
{1, 2}
{1, 2, 4, 5}

# 9.4.9 Union

## Syntax

set1 | set2
set1.union(set2)

## Explanation

1. Union combines two sets.
2. It returns all unique values from both sets.
3. Duplicate/common values appear only once.

## Example 1

```
PYTHON CODE

a = {1, 2, 3}
b = {3, 4, 5}
result = a.union(b)
print(result)
```

**Output:** {1, 2, 3, 4, 5}

a = {1, 2, 3}
b = {3, 4, 5}
Union = all unique values = {1, 2, 3, 4, 5}

deWithNishchal

# 9.4.10 Intersection

## Syntax

set1 & set2

set1.intersection(set2)

1. Intersection returns only common values.
2. Values must exist in both sets.

```
PYTHON CODE
a = {1, 2, 3}
b = {3, 4, 5}
result = a.intersection(b)
print(result)
```

Output: {3}

# 9.4.11 Difference

## Syntax

**set1 - set2**
**set1.difference(set2)**

1. Difference returns values present in the first set but not in the second set.
2. a - b and b - a can give different results.

```
PYTHON CODE
a = {1, 2, 3}
b = {3, 4, 5}
print(a - b)
print(b - a)
```

Output:
{1, 2}
{4, 5}
a - b = values in a but not in b  = {1, 2}
b - a = values in b but not in a   = {4, 5}

devWithNishchal

# 9.4.12 Symmetric Difference

## Syntax

set1 ^ set2

set1.symmetric_difference(set2)

1. Symmetric difference returns values that are not common.
2. It removes common values from the final result.

```
PYTHON CODE
a = {1, 2, 3}
b = {3, 4, 5}
result = a.symmetric_difference(b)
print(result)
```

Output: {1, 2, 4, 5}

# 9.4.13 Subset, Superset, & Disjoint Sets

## Syntax

set1.issubset(set2)
set1.issuperset(set2)
set1.isdisjoint(set2)

1. A subset means all values of one set exist inside another set.
2. A superset means one set contains all values of another set.
3. Disjoint sets have no common values.

```
PYTHON CODE
a = {1, 2}
b = {1, 2, 3, 4}
c = {5, 6}
print(a.issubset(b))
print(b.issuperset(a))
print(a.isdisjoint(c))
```

**Output:**
True
True
True

devWithNishchal

## Comparison Table

| Concept | Meaning | Example | Result |
|---|---|---|---|
| **Subset** | All items of a are in b | {1, 2} <= {1, 2, 3} | True |
| **Superset** | b contains all items of a | {1, 2, 3} >= {1, 2} | True |
| **Disjoint** | No common items | {1, 2}.isdisjoint({3, 4}) | True |

# 9.4.14 Set Methods

| Method | Purpose | Example | Result / Effect |
|---|---|---|---|
| add() | Adds one item | s.add(10) | Adds 10 |
| update() | Adds multiple items | s.update([10, 20]) | Adds all items |
| remove() | Removes item | s.remove(10) | Error if missing |
| discard() | Removes item safely | s.discard(10) | No error if missing |
| pop() | Removes random item | s.pop() | Returns removed item |
| clear() | Removes all items | s.clear() | Empty set |
| copy() | Creates shallow copy | new = s.copy() | New set |
| union() | Combines sets | a.union(b) | New set |
| intersection() | Common items | a.intersection(b) | New set |
| difference() | Items in first but not second | a.difference(b) | New set |
| symmetric_difference() | Items not common | a.symmetric_difference(b) | New set |
| intersection_update() | Keeps only common items | a.intersection_update(b) | Changes a |
| difference_update() | Removes items found in other set | a.difference_update(b) | Changes a |
| symmetric_difference_update() | Keeps non-common items | a.symmetric_difference_update(b) | Changes a |
| issubset() | Checks subset | a.issubset(b) | True/False |
| issuperset() | Checks superset | a.issuperset(b) | True/False |
| isdisjoint() | Checks no common items | a.isdisjoint(b) | True/False |

CodeWithNishchal

# 9.4.15 Set Operators

Set operations can also be done using operators.

| Operator | Meaning | Same As |
|---|---|---|
| ` | ` | Union |
| & | Intersection | a.intersection(b) |
| - | Difference | a.difference(b) |
| ^ | Symmetric difference | a.symmetric_difference(b) |
| <= | Subset check | a.issubset(b) |
| < | Proper subset check | a < b |
| >= | Superset check | a.issuperset(b) |
| > | Proper superset check | a > b |

# 9.4.16 Updating Sets with Operations

## Syntax

set1.intersection_update(set2)
set1.difference_update(set2)
set1.symmetric_difference_update(set2)

1. Normal set operation methods return a new set.
2. Update methods change the original set.
3. These are useful when we do not need the old set.

PYTHON CODE

```
a = {1, 2, 3}
b = {3, 4, 5}
a.intersection_update(b)
print(a)
```

Output: {3}

CodeWithNishchal

# 9.4.17 Copying Sets

## Syntax

new_set = old_set.copy()

1. copy() creates a shallow copy of a set. Direct assignment does not create a new set.
2. Direct assignment makes both variables point to the same set.

```
PYTHON CODE
a = {10, 20, 30}
b = a.copy()
b.add(40)
print(a)
print(b)
```

**Output:** {10, 20, 30}
{40, 10, 20, 30}

## Copying Table

| Code | Creates New Set? | Meaning |
|---|---|---|
| b = a | No | Same set reference |
| b = a.copy() | Yes | Shallow copy |
| b = set(a) | Yes | New set |

# 9.4.18 Frozen Sets

A **frozenset** is an immutable version of a set.

## Syntax

frozenset_name = frozenset(iterable)

1. A normal set is mutable. A frozenset is immutable.
2. Values cannot be added or removed from a frozenset. Frozensets can be used as dictionary keys or set elements.
3. Frozensets support read-only set operations like union, intersection, and difference.

```
PYTHON CODE
numbers = frozenset([10, 20, 30])
print(numbers)
print(type(numbers))
```

**Output:** frozenset({10, 20, 30})
<class 'frozenset'>

CodeWithNishchal

## Invalid Operation

```
PYTHON CODE
numbers = frozenset([10, 20, 30])
numbers.add(40)
```

**Output:** AttributeError: 'frozenset' object has no attribute 'add'

## Set vs Frozen Set Table

| Point | Set | Frozenset |
|---|---|---|
| Mutable | Yes | No |
| Add items | Allowed | Not allowed |
| Remove items | Allowed | Not allowed |
| Unique values | Yes | Yes |
| Unordered | Yes | Yes |
| Can be dictionary key | No | Yes |
| Can be set element | No | Yes |
| Syntax | {1, 2, 3} | frozenset([1, 2, 3]) |

## Frozenset Methods Table

| Method | Available in Frozenset? | Reason |
|---|---|---|
| union() | Yes | Does not modify original |
| intersection() | Yes | Does not modify original |
| difference() | Yes | Does not modify original |
| symmetric_difference() | Yes | Does not modify original |
| issubset() | Yes | Only checks |
| issuperset() | Yes | Only checks |
| isdisjoint() | Yes | Only checks |
| add() | No | Modifies set |
| remove() | No | Modifies set |
| discard() | No | Modifies set |
| clear() | No | Modifies set |
| update() | No | Modifies set |

CodeWithNishchal

# 9.4.19 Set Comprehensions

## Syntax

new_set = {expression for item in sequence}

With condition:

new_set = {expression for item in sequence if condition}

1. Set comprehension is a short way to create a set.
2. It is similar to list comprehension.
3. It automatically keeps only unique values.
4. It can include conditions.

## Example 1

PYTHON CODE

```
numbers = [1, 2, 2, 3, 4, 4]
squares = {number * number for number in numbers}
print(squares)
```

**Output:** {16, 1, 4, 9}  :Duplicate input values do not create duplicate set values.

## Example 2: With Condition

PYTHON CODE

```
numbers = [1, 2, 3, 4, 5, 6]
even_numbers = {number for number in
numbers if number % 2 == 0}
print(even_numbers)
```

Output: {2, 4, 6}

## List Comprehension vs Set Comprehension

| Point | List Comprehension | Set Comprehension |
|---|---|---|
| Brackets | [] | {} |
| Allows duplicates | Yes | No |
| Ordered | Yes | No fixed order |
| Result type | List | Set |
| Example | [x for x in data] | {x for x in data} |

# 9.4.20 Set Conversion

## Syntax

set(iterable)

list(set_name)

tuple(set_name)

1. set() converts an iterable into a set.
2. This is often used to remove duplicates.
3. A set can be converted back to a list or tuple.
4. Order may change after converting to a set.

## Example 1

PYTHON CODE

```
numbers = [10, 20, 10, 30, 20]
unique_numbers = set(numbers)
print(unique_numbers)
print(list(unique_numbers))
```

Output:
{10, 20, 30}
[10, 20, 30]
Order may differ.

## Conversion Table

| Conversion | Code | Result Type |
|---|---|---|
| List to set | set([1, 2, 2]) | Set |
| Tuple to set | set((1, 2, 2)) | Set |
| String to set | set("banana") | Set of unique characters |
| Set to list | list({1, 2, 3}) | List |
| Set to tuple | tuple({1, 2, 3}) | Tuple |

CodeWithNishchal

# 9.4.21 Set vs List vs Tuple

| Point | List | Tuple | Set |
|---|---|---|---|
| Brackets | [] | () | {} |
| Ordered | Yes | Yes | No fixed order |
| Mutable | Yes | No | Yes |
| Allows duplicates | Yes | Yes | No |
| Indexing | Yes | Yes | No |
| Slicing | Yes | Yes | No |
| Best for | Changeable ordered data | Fixed ordered data | Unique unordered data |
| Example | [10, 20] | (10, 20) | {10, 20} |

# Important Set Operations Table

| Operation | Code | Meaning |
|---|---|---|
| Create set | items = {10, 20, 30} | Creates set |
| Empty set | items = set() | Creates empty set |
| Add one item | items.add(40) | Adds one value |
| Add multiple items | items.update([40, 50]) | Adds many values |
| Remove safely | items.discard(20) | Removes without error |
| Remove with error | items.remove(20) | Error if missing |
| Random remove | items.pop() | Removes random item |
| Empty set | items.clear() | Removes all items |
| Check value | 10 in items | Membership check |
| Union | `a | b` |
| Intersection | a & b | Common values |
| Difference | a - b | Values in a, not in b |
| Symmetric difference | a ^ b | Non-common values |
| Subset | a <= b | Checks subset |
| Superset | a >= b | Checks superset |
| Copy set | items.copy() | Creates shallow copy |
| Frozen set | frozenset(items) | Immutable set |

CodeWithNishal

# 9.5 Collections Module in Python

The collections module provides special data structures that are more powerful than normal lists, tuples, dictionaries, and sets.

## Syntax

```python
from collections import Counter
from collections import defaultdict
from collections import OrderedDict
from collections import deque
from collections import ChainMap
```

1. collections is a built-in Python module.
2. It gives ready-made advanced data structures.
3. These structures help solve common problems easily.
4. They are useful for counting, grouping, ordering, fast queue operations, and combining dictionaries.

## Collections Module Overview

| Tool | Main Use |
|---|---|
| Counter | Counting frequency |
| defaultdict | Dictionary with default values |
| OrderedDict | Dictionary that manages order |
| deque | Fast queue and stack operations |
| ChainMap | Combines multiple dictionaries as one view |

# 9.5.1 Counter

Counter is used to count how many times each value appears.

## Syntax

```python
from collections import Counter

counter_name = Counter(iterable)
```

CodeWithNishchal

# Explanation

1. Counter counts repeated values.
2. It returns a dictionary-like object.
3. Items become keys.
4. Their counts become values.
5. It is commonly used for frequency counting.

# Flow Chart

Input data
  |
  v
Counter checks each item
  |
  v
Counts repeated items
  |
  v
Returns item-count pairs

# Example 1

PYTHON CODE

```
from collections import Counter
letters = "banana"
count = Counter(letters)
print(count)
```

**Output:** Counter({'a': 3, 'n': 2, 'b': 1})

# Example 2

PYTHON CODE

```
from collections import Counter
items = ["apple", "banana", "apple", "mango", "banana", "apple"]
count = Counter(items)
print(count)
```

**Output:** Counter({'apple': 3, 'banana': 2, 'mango': 1})

Code with Nishchal

## Common Counter Methods

| Method | Meaning | Example |
|---|---|---|
| most_common() | Returns most frequent items | count.most_common() |
| most_common(n) | Returns top n frequent items | count.most_common(2) |
| elements() | Returns items repeated by count | count.elements() |
| update() | Adds more counts | count.update(data) |
| subtract() | Subtracts counts | count.subtract(data) |

# 9.5.2 defaultdict

defaultdict is a dictionary that gives a default value when a key does not exist.

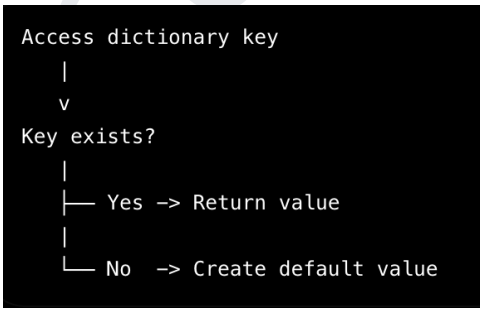
## Syntax

```
from collections import defaultdict
dictionary_name = defaultdict(default_type)
```

## Explanation

1. Normal dictionaries give KeyError if a key does not exist.
2. defaultdict avoids this problem.
3. It automatically creates a default value for missing keys.
4. Common default types are int, list, and set.

## Flow Chart




CodeWithNishchal

## Example 1: Using int

```
PYTHON CODE
from collections import defaultdict
marks = defaultdict(int)
marks["math"] += 10
marks["science"] += 20
print(marks)
```

**Output:** defaultdict(<class 'int'>, {'math': 10, 'science': 20})
Here, missing keys start with default value 0.

## Common Default Types

| Default Type | Default Value | Common Use |
|---|---|---|
| int | 0 | Counting |
| list | [] | Grouping values |
| set | set() | Grouping unique values |
| str | "" | Empty text |

## Example 2: Using list

```
PYTHON CODE
from collections import defaultdict

students = defaultdict(list)
students["Python"].append("Aman")
students["Python"].append("Riya")
students["Java"].append("Kabir")
print(students)
```

**Output:** defaultdict(<class 'list'>, {'Python': ['Aman', 'Riya'], 'Java': ['Kabir']})

## Normal Dictionary vs defaultdict

| Point | Normal Dictionary | defaultdict |
|---|---|---|
| Missing key | Gives KeyError | Creates default value |
| Best for | Simple key-value data | Grouping/counting |
| Needs manual checking | Yes | No |

CodeWithNishchal

# 9.5.3 OrderedDict

OrderedDict is a dictionary that remembers and manages the order of items.

## Syntax

```
PYTHON CODE
from collections import OrderedDict
dictionary_name = OrderedDict()
```

## Explanation

1. OrderedDict stores key-value pairs in order.
2. Normal dictionaries also preserve insertion order in modern Python.
3. OrderedDict is still useful because it has extra order-related methods.
4. It can move items to the beginning or end.
5. It can remove items from either side.

**Example 1**

```
PYTHON CODE
from collections import OrderedDict
student = OrderedDict()
student["name"] = "Aman"
student["age"] = 21
student["course"] = "Python"
print(student)
```

**Output:** OrderedDict([('name', 'Aman'), ('age', 21), ('course', 'Python')])

## Useful OrderedDict Methods

| Method | Meaning | Example |
|---|---|---|
| move_to_end(key) | Moves key to the end | data.move_to_end("name") |
| move_to_end(key, last=False) | Moves key to the beginning | data.move_to_end("name", last=False) |
| popitem() | Removes last item | data.popitem() |
| popitem(last=False) | Removes first item | data.popitem(last=False) |

Code with Nishchal

## Normal Dictionary vs OrderedDict

| Point | Normal Dictionary | OrderedDict |
|---|---|---|
| Preserves insertion order | Yes, in modern Python | Yes |
| Move key to end | Not directly | Yes |
| Remove first item easily | Not directly | Yes |
| Best for | General dictionary use | Order-sensitive dictionary logic |

# 9.5.4 deque

deque means **double-ended queue**.

It allows fast adding and removing from both ends.

## Syntax

from collections import deque

deque_name = deque(iterable)

1. deque is used when we need fast insert/remove from left and right.
2. It works like a queue and stack.
3. Lists are slower when removing from the beginning.
4. deque is useful for queues, recent history, undo operations, and sliding windows.

Example 1

```
PYTHON CODE
from collections import deque
numbers = deque([10, 20, 30])
numbers.append(40)
numbers.appendleft(5)
print(numbers)
```

**Output:** deque([5, 10, 20, 30, 40])

## Common deque Methods

deWithNishchal

| Method | Meaning | Example |
|---|---|---|
| append() | Adds item at right end | d.append(10) |
| appendleft() | Adds item at left end | d.appendleft(10) |
| pop() | Removes item from right end | d.pop() |
| popleft() | Removes item from left end | d.popleft() |
| extend() | Adds multiple items at right | d.extend([1, 2]) |
| extendleft() | Adds multiple items at left | d.extendleft([1, 2]) |
| rotate(n) | Rotates items | d.rotate(1) |
| clear() | Removes all items | d.clear() |

## List vs deque

| Point | List | deque |
|---|---|---|
| Add at end | Fast | Fast |
| Remove from end | Fast | Fast |
| Add at beginning | Slower | Fast |
| Remove from beginning | Slower | Fast |
| Best for | General storage | Queue operations |

# 9.5.5 ChainMap

ChainMap combines multiple dictionaries into one view.

## Syntax

```
from collections import ChainMap
chain_name = ChainMap(dict1, dict2, dict3)
```

1. ChainMap groups multiple dictionaries together.
2. It does not merge them permanently.
3. It creates a combined view.
4. When searching for a key, Python checks dictionaries from left to right.
5. If the same key exists in multiple dictionaries, the first one is used.

CodeWithNishchal

**Example 1**

```python
from collections import ChainMap
defaults = {
    "theme": "light",
    "language": "English"
}
user_settings = {
    "theme": "dark"
}
settings = ChainMap(user_settings, defaults)
print(settings["theme"])
print(settings["language"])
```

Output:
dark
English
Explanation:

1. "theme" exists in user_settings, so "dark" is used.
2. "language" is not in user_settings, so Python checks defaults.

## Useful ChainMap Features

| Feature | Meaning | Example |
|---|---|---|
| maps | Shows all dictionaries | settings.maps |
| new_child() | Adds new dictionary in front | settings.new_child({...}) |
| parents | Removes first dictionary from view | settings.parents |

CodeWithNishchal

# Chapter 10. Functions in Python

A **function** is a reusable block of code that performs a specific task. Instead of writing the same code again and again, we write it once inside a function and call it whenever needed.

## Basic idea

Function = reusable block of code

## Simple flow

Define function
  |
  v
Call function
  |
  v
Function code runs
  |
  v
Result/output is produced

# 10.1 What is a Function?

## Syntax

```python
def function_name():
    statement
```

1. A function groups related code together.
2. A function runs only when it is called.
3. Functions help avoid repeated code.
4. Functions make code clean, reusable, and easy to understand.
5. Python has built-in functions and user-defined functions.

## Built-in vs User-defined Functions

| Type | Meaning | Example |
|---|---|---|
| Built-in function | Already provided by Python | print(), len(), type() |
| User-defined function | Created by programmer | def greet(): |

CodeWithNishchal

## Flow Chart

Start
 |
 v
Function is defined
 |
 v
Function is called
 |
 v
Function body executes
 |
 v
Program continues

PYTHON CODE

```
def greet():
    print("Hello, welcome to Python")


greet()
```

**Output:** Hello, welcome to Python

# 10.2 Defining Functions

## Syntax

```
Python

def function_name():
    statement
```

1. def is used to define a function.
2. The function name comes after def.
3. Parentheses () are required.
4. A colon : marks the start of the function body.
5. The function body must be indented.

```
def       function_name      ()       :
 |            |             |        |
keyword     name         parentheses  colon
```

CodeWithNishchal

```python
def say_hello():
    print("Hello")
```

This only defines the function. It does not run yet. To run it, we must call it.

say_hello()

**Output:** Hello

## Empty Function

If we want to create a function but write logic later, use pass.

```python
def future_function():
    pass
```

# 10.3 Calling Functions

## Syntax

function_name()

1. Calling a function means executing it.
2. A function can be called once or many times.
3. The function body runs every time the function is called.
4. If a function is defined but never called, its code will not execute.

## Example 1

```python
def greet():
    print("Hello")


greet()
greet()
greet()
```

Output:Hello
Hello
Hello

CodeWithNishchal

# 10.4 Parameters

A **parameter** is a variable written inside the function definition. It receives values when the function is called.

## Syntax

```
PYTHON CODE
def function_name(parameter):
    statement
```

## Explanation

1. Parameters make functions flexible.
2. Parameters allow us to send data into a function.
3. A function can have one or more parameters.
4. Parameters are written inside parentheses during function definition.

## Flow Chart

Function definition has parameter
  |
  v
Function call sends value
  |
  v
Parameter receives value
  |
  v
Function uses that value

## Example 1

```
PYTHON CODE
def greet(name):
    print("Hello", name)
greet("Aman")
```

**Output:**
Hello Aman
Here, name is a parameter.

deWithNishchal

# 10.5 Arguments

An **argument** is the actual value passed to a function during function call.

## Syntax

```
function_name(argument)
```

## Explanation

1. Parameter is written in function definition.
2. Argument is passed during function call.
3. Arguments provide real values to parameters.

## Parameter vs Argument

| Term | Where It Appears | Meaning |
|---|---|---|
| Parameter | Function definition | Variable that receives value |
| Argument | Function call | Actual value passed |

## Example 1

PYTHON CODE

```
def greet(name):
    print("Hello", name)
greet("Riya")
```

**Output:** Hello Riya

| Code | Role |
|---|---|
| name | Parameter |
| "Riya" | Argument |

# 10.6 Positional Arguments

Positional arguments are matched based on their position.

## Syntax

```
function_name(argument1, argument2)
```

Codewith Nishchal

## Explanation

1. Python passes arguments in the same order as parameters.
2. The first argument goes to the first parameter.
3. The second argument goes to the second parameter.
4. Order matters in positional arguments.

## Example 1

```
PYTHON CODE
def student_info(name, age):
    print("Name:", name)
    print("Age:", age)

student_info("Aman", 21)
```

**Output:**
Name: Aman
Age: 21

## Flow Chart

student_info("Aman", 21)

| |

v v

name age

# 10.7 Return Values

A **return value** is the result sent back by a function.

## Syntax

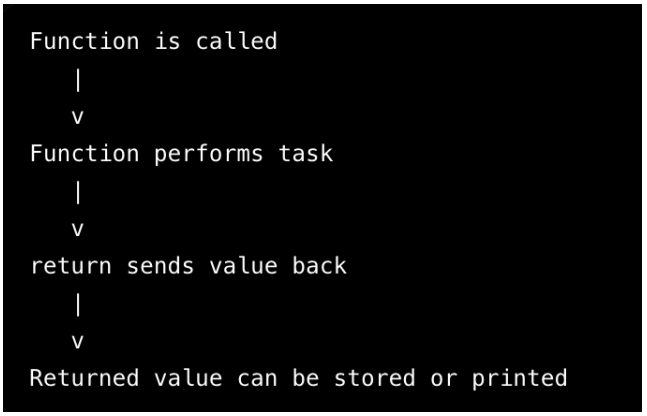
**def function_name():**

**return value**

1. return sends a value back to the place where the function was called.
2. A function can return one value or multiple values.
3. After return, the function stops executing.
4. If there is no return, Python returns None automatically.

CodeWithNishchal

## Flow Chart




## Example 1

PYTHON CODE

```
def add(a, b):
    result = a + b
    return result

answer = add(10, 20)
print(answer)
```

**Output:**

30

## Example 2: Function without return

PYTHON CODE

```
def greet():
    print("Hello")

result = greet()
print(result)
```

**Output:** Hello
None

Because greet() does not return any value, Python returns None.

deWithNishchal

# 10.8 Returning Multiple Values

Python functions can return multiple values.

**Syntax:**

```python
def function_name():
    return value1, value2
```

1. Multiple values can be returned using commas.
2. Python returns them as a tuple.
3. Returned values can be unpacked into variables.

## Example 1

PYTHON CODE

```
def calculate(a, b):
    total = a + b
    difference = a - b
    return total, difference
sum_result, diff_result = calculate(20, 10)
print(sum_result)
print(diff_result)
```

**Output:**

30
10

# 10.9 Default Parameters

A default parameter has a predefined value. If no argument is passed, the default value is used.

## Syntax

```python
def function_name(parameter=value):
    statement
```

1. Default parameters make arguments optional.
2. If an argument is provided, Python uses the given value.
3. If no argument is provided, Python uses the default value.
4. Non-default parameters must come before default parameters.

CodeWithNishchal

# Example 1

PYTHON CODE

```
def greet(name="Guest"):
    print("Hello", name)
greet("Aman")
greet()
```

**Output:**

Hello Aman

Hello Guest

# Correct and Incorrect Order

| Code | Valid? | Reason |
|---|---|---|
| def show(name, age=18): | Yes | Default parameter comes after normal parameter |
| def show(name="Guest", age=18): | Yes | Both have defaults |
| def show(name="Guest", age): | No | Non-default parameter cannot come after default parameter |

# **Important Warning:** Avoid Mutable Default Values

Do not use mutable objects like list or dictionary as default values.

**Wrong style:**

```
def add_item(item, items=[]):
    items.append(item)
    return items

print(add_item("A"))
print(add_item("B"))
```

**Output:** ['A']

['A', 'B']

The same list is reused between function calls.

**Better style:**

Python

```
def add_item(item, items=None):
    if items is None:
        items = []

    items.append(item)
    return items

print(add_item("A"))
print(add_item("B"))
```

Output: ['A']

['B']

Coded with Kushal

# 10.10 Keyword Arguments

Keyword arguments pass values using parameter names.

## Syntax

```
function_name(parameter_name=value)
```

1. Keyword arguments use names while calling a function.
2. Order does not matter when keyword arguments are used.
3. They make function calls more readable.
4. Positional arguments must come before keyword arguments.

## Example 1

PYTHON CODE

```
def student_info(name, age, course):
    print("Name:", name)
    print("Age:", age)
    print("Course:", course)
student_info(age=21, course="Python", name="Aman")
```

**Output:** Name: Aman
Age: 21
Course: Python

### Positional vs Keyword Arguments

| Type | Example | Order Matters? |
|---|---|---|
| Positional argument | student_info("Aman", 21, "Python") | Yes |
| Keyword argument | student_info(age=21, name="Aman", course="Python") | No |

## Important Rule

**Correct:**

```
student_info("Aman", age=21, course="Python")
```

**Incorrect:**

```
student_info(name="Aman", 21, "Python")
```

Positional arguments cannot come after keyword arguments.

CodeWithNishchal

# 10.11 *args

*args is used when we do not know how many positional arguments will be passed.

## Syntax

```
def function_name(*args):
    statement
```

1. *args collects extra positional arguments.
2. The collected values are stored as a tuple.
3. The name args is a convention; the * is important.
4. Use *args when the number of arguments is flexible.

## Flow Chart

Function call has many positional arguments
  |
  v
*args collects them
  |
  v
Values are stored as a tuple

## Example 1

PYTHON CODE

```
def add_numbers(*numbers):
    total = 0
    for number in numbers:
        total += number
    return total
print(add_numbers(10, 20, 30))
print(add_numbers(5, 15))
```

Output:
60
20
Here, numbers behaves like a tuple.

WithNishchal

# 10.12 **kwargs

**kwargs is used when we do not know how many keyword arguments will be passed.

## Syntax

```
def function_name(**kwargs):
    statement
```

## Explanation

1. **kwargs collects extra keyword arguments.
2. The collected values are stored as a dictionary.
3. Keys are argument names.
4. Values are argument values.
5. The name kwargs is a convention; the ** is important.

## Flow Chart

Function call has many keyword arguments
  |
  v
**kwargs collects them
  |
  v
Values are stored as a dictionary

## Example 1

PYTHON CODE

```
def show_profile(**details):
    for key, value in details.items():
        print(key, value)


show_profile(name="Aman", age=21,
course="Python")
```

Output:
name Aman
age 21
course Python

Here, details behaves like a dictionary.

deWithNishchal

# 10.13 *args vs **kwargs

| Point | *args | **kwargs |
|---|---|---|
| Collects | Positional arguments | Keyword arguments |
| Stored as | Tuple | Dictionary |
| Symbol | Single star * | Double star ** |
| Example call | func(10, 20, 30) | func(name="Aman", age=21) |

## Example 1

PYTHON CODE
```
def show_data(*args, **kwargs):
    print(args)
    print(kwargs)

show_data(10, 20, name="Aman", age=21)
```

**Output:**
(10, 20)
{'name': 'Aman', 'age': 21}

# 10.14 Function Parameter Order

When using different types of parameters together, the order matters.

## Syntax

```
def function_name(normal_parameter, default_parameter=value, *args, **kwargs):
    statement
```

## Explanation

1. Normal parameters come first.
2. Default parameters come after normal parameters.
3. *args comes after normal/default parameters.
4. **kwargs comes last.
5. This order keeps function calls clear and valid.

CodeWithNishchal

## Parameter Order Table

| Order | Parameter Type | Example |
|---|---|---|
| 1 | Normal parameter | name |
| 2 | Default parameter | age=18 |
| 3 | *args | *marks |
| 4 | **kwargs | **details |

## Example 1

```
def student_report(name, age=18, *marks, **details):
    print("Name:", name)
    print("Age:", age)
    print("Marks:", marks)
    print("Details:", details)

student_report("Aman", 21, 80, 90, 85, course="Python", city="Delhi")
```

**Output:**

Name: Aman
Age: 21
Marks: (80, 90, 85)
Details: {'course': 'Python', 'city': 'Delhi'}

# 10.15 Docstrings

A **docstring** is a string written inside a function to explain what the function does.

## Syntax

```
def function_name():
    """Function description."""
    statement
```

1. A docstring is written using triple quotes.
2. It is usually written as the first statement inside a function.
3. It explains the purpose of the function.
4. It helps other programmers understand the function.
5. It can be viewed using help() or .__doc__.

CodeWithNishchal

## Example 1

```
def add(a, b):
    """Return the sum of two numbers."""
    return a + b

print(add(10, 20))
print(add.__doc__)
```

**Output:** 30

Return the sum of two numbers.

## Good Docstring Style

```
Python

def calculate_area(length, width):
    """
    Return the area of a rectangle.

    length: rectangle length
    width: rectangle width
    """
    return length * width
```

# 10.16 Function Naming Rules

Function names follow the same basic rules as variable names.

## Syntax

```
def function_name():
    statement
```

1. Function names should be meaningful.
2. Use snake_case for function names.
3. Function names should usually describe an action.
4. Avoid using Python keywords as function names.
5. Avoid unclear names like x(), abc(), or test1().

CodeWithNishchal

# Chapter 11. Advanced Functions

Advanced functions help us write more flexible, reusable, and compact code.

Before starting, remember:

In Python, functions are objects. This means:

1. A function can be stored in a variable.
2. A function can be passed as an argument.
3. A function can be returned from another function.
4. A function can be written inside another function.

# 11.1 Functions as First-Class Objects

## Syntax

```
def function_name():
    statement

new_name = function_name
```

In Python, functions behave like normal objects. We can store a function in a variable and call it using that variable.

## Flow Chart

Create function
  |
  v
Assign function to variable
  |
  v
Call function using new variable

## Example 1

```
PYTHON CODE
def greet():
    print("Hello Python")
message = greet
message()
```

Output:
Hello Python
Here, message refers to the same function as greet.

CodeWithNishchal

# 11.2 Lambda Functions

A **lambda function** is a small anonymous function. Anonymous means it does not need a normal function name.

## Syntax

**lambda arguments: expression**

1. Lambda functions are used for small one-line functions.
2. They can take any number of arguments.
3. They can contain only one expression.
4. They automatically return the result of the expression.
5. Lambda functions are commonly used with map(), filter(), and sorting.

## Example 1

**Normal function:**

```python
def square(number):
    return number * number

print(square(5))
```

**Output:**
25
Same logic using lambda:

```python
square = lambda number: number * number
```

**Output:** 25

## Example 2

```python
add = lambda a, b: a + b

print(add(10, 20))
```

**Output:**30

Lambda is best for simple logic. For complex logic, use normal def functions.

CodeWithNishchal

# 11.3 map()

map() applies a function to every item of an iterable.

## Syntax

map(function, iterable)

## Explanation

1. map() takes a function and an iterable.
2. It applies the function to each item.
3. In Python 3, map() returns a map object, which is an iterator.
4. To display all results at once, convert it using list().

## Example 1

```
PYTHON CODE
numbers = [1, 2, 3, 4]
squares = map(lambda number: number * number, numbers)
print(list(squares))
```

**Output:** [1, 4, 9, 16]

## Example 2

```
names = ["aman", "riya", "kabir"]
upper_names = map(str.upper, names)
print(list(upper_names))
```

**Output:** ['AMAN', 'RIYA', 'KABIR']

## map() Table

| Part | Meaning |
|---|---|
| lambda number: number * number | Function to apply |
| numbers | Iterable |
| map() | Applies function to every item |
| list() | Converts result into a list |

CodeWithPanchal

# 11.4 filter()

filter() is used to select items from an iterable based on a condition.

## Syntax

filter(function, iterable)

## Explanation

1. filter() takes a function and an iterable.
2. The function must return True or False.
3. Items that return True are kept.
4. Items that return False are removed.
5. In Python 3, filter() returns a filter object, which is an iterator.

## Flow Chart

Iterable values
  |
  v
filter() checks condition
  |
  ├── True  -> keep item
  |
  └── False -> remove item
  |
  v
Filtered result

## Example 1

PYTHON CODE

```
numbers = [1, 2, 3, 4, 5, 6]
even_numbers = filter(lambda number: number % 2 == 0, numbers)
print(list(even_numbers))
```

**Output:** [2, 4, 6]

## map() vs filter()

| Point | map() | filter() |
|---|---|---|
| Purpose | Transforms every item | Selects matching items |
| Output length | Usually same as input | Can be smaller |
| Function returns | New value | True or False |
| Example use | Square every number | Keep even numbers |

CodeWithNishchal

# 11.5 reduce()

reduce() combines all items of an iterable into a single final value. reduce() is not directly available like map() and filter(). It must be imported from functools.

## Syntax

from functools import reduce

reduce(function, iterable)

## Explanation

1. reduce() takes a function and an iterable.
2. It combines values step by step.
3. It returns one final result.
4. It is useful for cumulative calculations.
5. For simple addition, sum() is usually better.

## Example 1

PYTHON CODE

```
from functools import reduce
numbers = [1, 2, 3, 4]
total = reduce(lambda a, b: a + b, numbers)
print(total)
```

**Output:** 10
Working idea:
1 + 2 = 3
3 + 3 = 6
6 + 4 = 10

## map() vs filter() vs reduce()

| Function | Purpose | Final Result |
|---|---|---|
| map() | Transform items | Iterator of transformed items |
| filter() | Select items | Iterator of selected items |
| reduce() | Combine items | Single value |

Code Nishchal

# 11.6 Recursion

Recursion means a function calling itself.

## Syntax

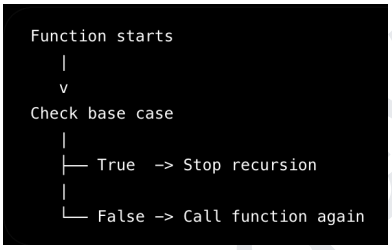
```python
def function_name():
    function_name()
```

A correct recursive function must have a stopping condition.

1. Recursion is used when a problem can be broken into smaller versions of the same problem.
2. A recursive function calls itself.
3. Every recursion must have a base case.
4. The base case stops the recursion.
5. Without a base case, recursion continues until Python raises RecursionError.

| Part | Meaning |
|---|---|
| Base case | Condition that stops recursion |
| Recursive case | Function calling itself |

## Flow Chart




## Example 1: Factorial

Factorial means: 5! = 5 × 4 × 3 × 2 × 1

PYTHON CODE

```python
def factorial(number):
    if number == 1:
        return 1
    return number * factorial(number - 1)
print(factorial(5))
```

Output: 120

factorial(5) = 5 * factorial(4) = 5 * 4 * factorial(3) = 5 * 4 * 3 * factorial(2) = 5 * 4 * 3 * 2 * factorial(1) = 5 * 4 * 3 * 2 * 1 = 120

CodeWithNishchal

# 11.7 Nested Functions

A nested function is a function defined inside another function.

## Syntax

```
PYTHON CODE
def outer_function():
    def inner_function():
        statement

    inner_function()
```

1. A nested function is created inside another function.
2. The inner function can be used only inside the outer function.
3. Nested functions are useful for hiding helper logic.
4. They are also used in closures and decorators.

## Example 1

```
PYTHON CODE
def outer():
    print("Outer function started")

    def inner():
        print("Inner function executed")

    inner()

outer()
```

**Output:** Outer function started
Inner function executed

**Important point:** inner() cannot be called directly outside outer().

# 11.8 Closures

A closure is created when an inner function remembers variables from its outer function, even after the outer function has finished.

## Syntax

```python
def outer_function():
    variable = value

    def inner_function():
        use variable

    return inner_function
```

Code With Nishchal

1. A closure needs a nested function.
2. The inner function uses a variable from the outer function.
3. The outer function returns the inner function.
4. The inner function remembers the outer variable.
5. Closures are useful for creating customized functions.

```
def multiplier(number):
    def multiply(value):
        return value * number

    return multiply

double = multiplier(2)

print(double(5))
```

**Output:** 10

# 11.9 Function Annotations

Function annotations are used to add type hints to function parameters and return values.

## Syntax

def function_name(parameter: type) -> return_type:
    statement

1. Function annotations describe expected data types. They make code easier to understand.
2. They help editors and tools detect possible mistakes.
3. Python does not automatically enforce these types at runtime.
4. Annotations are stored in the function's __annotations__ attribute.

## Example 1

PYTHON CODE

```
def add(a: int, b: int) -> int:
    return a + b
print(add(10, 20))
```

Output: 30

Python

```
def add(a: int, b: int) -> int:
    return a + b

print(add.__annotations__)
```

Output: {'a': <class 'int'>, 'b': <class 'int'>, 'return': <class 'int'>}
**Important point:** Type hints are hints. Python does not stop wrong types automatically.

deVuthNishchal

# 11.10 Higher-Order Functions

A higher-order function is a function that does at least one of these:

1. Takes another function as an argument.
2. Returns another function.

## Syntax

```python
def outer():
    def inner():
        statement

    return inner
```

OR

```
def outer(function):
    function()
```

## Explanation

1. Higher-order functions are possible because Python functions are objects.
2. They are used in map(), filter(), reduce(), decorators, and callbacks.
3. They make code flexible and reusable.
4. They are common in functional programming.

## Example 1: Function as Argument

PYTHON CODE

```
def shout(text):
    return text.upper()

def process_text(function, text):
    return function(text)

result = process_text(shout, "hello")
print(result)
```

**Output:** HELLO

Here, process_text() is a higher-order function because it accepts another function.

Vitthal Vishnu Nischal

# Chapter 12. Scope and Namespaces in Python

**Scope** means the area of a program where a variable can be accessed.

**Namespace** means a place where names are stored and mapped to objects.

name -> object/value

**Example:** x = 10

Here, Python stores the name x and connects it to the value 10.

# 12.1 Local Scope

A **local scope** is created inside a function. Variables created inside a function are local variables.

## Syntax

```python
def function_name():
    variable_name = value
```

## Explanation

1. A local variable is created inside a function.
2. It can be used only inside that function.
3. It cannot be accessed directly outside the function.
4. Local variables are created when the function is called.
5. They are destroyed after the function finishes.

## Example 1

```
PYTHON CODE
def show_name():
    name = "Aman"
    print(name)
show_name()
```

**Output:** Aman

Invalid outside access:

```
PYTHON CODE
print(name)
```

Output: NameError: name 'name' is not defined.

deWithNishchal

# 12.2 Global Scope

A **global scope** is the main area of the program. Variables created outside all functions are global variables.

## Syntax

```
PYTHON CODE
variable_name = value


def function_name():
    statement
```

## Explanation

1. A global variable is created outside functions.
2. It can be accessed inside functions.
3. It can also be accessed outside functions.
4. Reading a global variable inside a function does not require the global keyword.
5. To modify a global variable inside a function, we need the global keyword.

## Example 1

```
PYTHON CODE
course = "Python"


def show_course():
    print(course)


show_course()
print(course)
```

Output:
Python
Python

NOTE: Global variables are accessible throughout the file after they are defined.

---

# 12.3 Local vs Global Scope

| Point | Local Scope | Global Scope |
|---|---|---|
| Created where? | Inside function | Outside functions |
| Accessible where? | Only inside that function | Inside and outside functions |
| Lifetime | Exists during function call | Exists while program runs |
| Example | name = "Aman" inside function | course = "Python" outside function |

# 12.4 Variable Shadowing

Variable shadowing happens when a local variable has the same name as a global variable.

## Explanation

1. If a local and global variable have the same name, Python uses the local variable inside the function.
2. The global variable is not changed.
3. This is called shadowing.

## Example 1

```
PYTHON CODE
name = "Global Aman"

def show_name():
    name = "Local Aman"
    print(name)

show_name()
print(name)
```

**Output:**

Local Aman
Global Aman

**NOTE:** Inside the function, the local variable gets priority over the global variable.

WithNishchal

# 12.5 global Keyword

The global keyword is used to modify a global variable inside a function.

## Syntax

global variable_name

## Explanation

1. Use global when you want to assign a new value to a global variable inside a function.
2. Without global, assignment inside a function creates a local variable.
3. Reading a global variable does not need global. Using too many global variables is not recommended.

## Example 1

PYTHON CODE

```
count = 0

def increase_count():
    global count
    count += 1

increase_count()
print(count)
```

**output:** 1

## Without global

PYTHON CODE

```
count = 0

def increase_count():
    count += 1

increase_count()
```

**Output:** UnboundLocalError: cannot access local variable 'count' where it is not associated with a value

**NOTE:** Use global only when you really need to modify a global variable. Better style is usually to return a value:

Nishchal

```
PYTHON CODE
def increase_count(count):
    return count + 1

count = 0
count = increase_count(count)

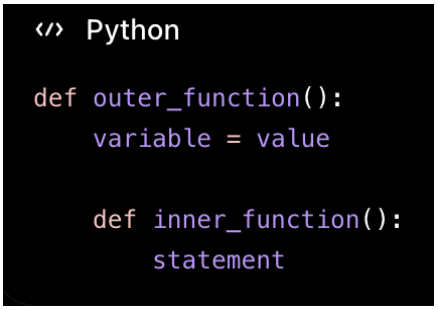
print(count)
```

Output: 1

# 12.6 Enclosing Scope

An **enclosing scope** exists when a function is defined inside another function.

## Syntax




1. Enclosing scope belongs to the outer function.
2. Inner functions can access variables from the outer function.
3. This scope is between local and global scope.
4. Enclosing scope is important for nested functions and closures.

## Example 1

```
PYTHON CODE
def outer():
    message = "Hello"

    def inner():
        print(message)

    inner()
outer()
```

Output: Hello

CodeWithNishchal

# 12.7 nonlocal Keyword

The nonlocal keyword is used to modify a variable from the nearest enclosing function scope.

## Syntax

nonlocal variable_name

1. nonlocal is used inside nested functions. It allows the inner function to modify a variable from the outer function.
2. It does not work with global variables.
3. The variable must already exist in the enclosing function. nonlocal is commonly used in closures.

## Example 1

PYTHON CODE

```
def outer():
    count = 0
    def inner():
        nonlocal count
        count += 1
        print(count)

    inner()
outer()
```

Output:1

## Without nonlocal

PYTHON CODE

```
def outer():
    count = 0

    def inner():
        nonlocal count
        count += 1
        print(count)

    inner()

outer()
```

Output: UnboundLocalError: cannot access local variable 'count' where it is not associated with a value

leWithNishchal

## global vs nonlocal

| Keyword | Used For | Scope Affected |
|---|---|---|
| global | Modify global variable | Global scope |
| nonlocal | Modify enclosing function variable | Enclosing scope |
| Neither | Normal local variable | Local scope |

# 12.8 Built-in Scope

Built-in scope contains names already provided by Python.
Examples: print, len, type, range, sum, max, min

## Explanation

1. Built-in names are available automatically.
2. We do not need to define them.
3. Python searches built-in scope last in the LEGB rule.
4. Avoid using built-in names as variable names.

## Example 1

```
numbers = [10, 20, 30]

print(len(numbers))
```

Output: 3

## Bad Practice

```
list = [10, 20, 30]
```

This is bad because the list is already a built-in name. Do not use names like list, dict, str, int, sum, or max as variable names.

# 12.9 LEGB Rule

LEGB is the order Python follows while searching for a variable name.
L -> Local
E -> Enclosing
G -> Global
B -> Built-in

CodeWithNishchal

When Python sees a variable name, it searches in this order:

1. **Local scope**: inside the current function.
2. **Enclosing scope**: inside outer functions.
3. **Global scope**: main program/file.
4. **Built-in scope**: built-in Python names.

Example 1

```python
x = "global"
def outer():
    x = "enclosing"
    def inner():
        x = "local"
        print(x)

    inner()
outer()
```

**Output:** local ⇒ Python finds x in the local scope first, so it does not search further.

## LEGB Search Table

| Search Order | Scope | Example |
|---|---|---|
| 1 | Local | Variable inside current function |
| 2 | Enclosing | Variable inside outer function |
| 3 | Global | Variable outside functions |
| 4 | Built-in | print, len, type |

# 12.10 Namespace Concept

A **namespace** is a system that stores names and their related objects.

## Syntax Idea

name -> object

1. A namespace maps names to values or objects.
2. Python uses namespaces to avoid name conflicts.
3. Different scopes have different namespaces.
4. Local, global, and built-in scopes each have their own namespaces.
5. The same name can exist in different namespaces without conflict.

CodeWithNishchal

## Example 1

```
PYTHON CODE
x = 100
def show():
    x = 50
    print(x)

show()
print(x)
```

**Output:** 50
100

1. The global namespace has x = 100. The local namespace inside show() has x = 50.
2. Both names are x, but they belong to different namespaces.

## Namespace Table

| Namespace | Contains |
|---|---|
| Local namespace | Names inside a function |
| Global namespace | Names created at file/program level |
| Built-in namespace | Python built-in names |
| Enclosing namespace | Names inside outer functions |

# 12.11 locals() and globals()

Python provides two built-in functions to inspect namespaces.

## Syntax

```
Python

locals()
globals()
```

1. locals() returns the current local namespace as a dictionary.
2. globals() returns the global namespace as a dictionary.
3. These are mainly used for debugging and learning.
4. Usually, we should not modify program logic using them.

codewithlishchal

## Example 1

```
PYTHON CODE
course = "Python"

def show():
    name = "Aman"
    print(locals())

show()
```

**Output:** {'name': 'Aman'}

# 12.12 NameError & UnboundLocalError

These errors are common in scope-related topics.

## NameError

NameError occurs when Python cannot find a name in LEGB search.

```
PYTHON CODE
print(age)
```

Output: NameError: name 'age' is not defined

## UnboundLocalError

UnboundLocalError occurs when Python treats a variable as local because it is assigned inside a function, but it is used before assignment.

```
PYTHON CODE
x = 10
def show():
    print(x)
    x = 20

show()
```

**output:** UnboundLocalError: cannot access local variable 'x' where it is not associated with a value

| Error | Meaning |
|---|---|
| NameError | Name not found in any scope |
| UnboundLocalError | Local variable used before assignment |

With Nishchal

# 12.13 Important Scope Summary Table

| Concept | Meaning |
|---|---|
| Local scope | Variable inside current function |
| Global scope | Variable outside all functions |
| Enclosing scope | Variable inside outer function |
| Built-in scope | Python built-in names |
| global | Modifies global variable inside function |
| nonlocal | Modifies enclosing function variable |
| LEGB | Search order for names |
| Namespace | Mapping of names to objects |
| locals() | Shows local namespace |
| globals() | Shows global namespace |
| NameError | Name not found |
| UnboundLocalError | Local variable used before assignment |

CodeWithNishchal

# Chapter 13. Modules and Packages in Python

A **module** is a Python file containing reusable code.

A **package** is a folder that contains multiple modules.

Module  -> single .py file

Package -> folder containing modules

Example:

math.py  -> module

my_package/  -> package

## Why Modules and Packages Are Used

1. To organize large programs.
2. To reuse code.
3. To avoid writing the same logic again.
4. To separate code into meaningful files.
5. To use built-in Python features from the standard library.

# 13.1 Importing Modules

Importing means using code from another module.

## Syntax

import module_name

1. import loads a module.
2. After importing, we can use functions, classes, and variables from that module.
3. We access module members using dot . notation.
4. Import statements are usually written at the top of the file.

## Example 1

PYTHON CODE

```
import math

print(math.sqrt(25))
```

Output: 5.0 => Here, math is a module and sqrt() is a function inside it.

CodeWithNishchal

# 13.2 Different Ways to Import

Python provides different import styles.

## Import Styles Table

| Import Style | Syntax | Example | How to Use |
|---|---|---|---|
| Normal import | import module | import math | math.sqrt(25) |
| Import with alias | import module as alias | import math as m | m.sqrt(25) |
| Import specific item | from module import item | from math import sqrt | sqrt(25) |
| Import multiple items | from module import a, b | from math import sqrt, pow | sqrt(25) |
| Import all | from module import * | from math import * | sqrt(25) |

## Example 1

PYTHON CODE

```
from math import sqrt


print(sqrt(36))
```

Output: 6.0

## Important Point

Avoid using from module import * in large programs. It can make code unclear and may cause name conflicts.

# 13.3 Standard Library Modules

The **standard library** is a collection of modules that come with Python. We do not need to install them separately.

## Explanation

1. Standard library modules are built into Python.
2. They help with math, dates, files, operating system tasks, random values, JSON, and more.
3. We can use them by importing them.

CodeWithNishchal

## Common Standard Library Modules

| Module | Purpose | Example Use |
|---|---|---|
| math | Mathematical operations | math.sqrt(25) |
| random | Random values | random.randint(1, 10) |
| datetime | Date and time | datetime.datetime.now() |
| os | Operating system tasks | os.getcwd() |
| sys | Python runtime information | sys.version |
| json | Work with JSON data | json.dumps(data) |
| statistics | Basic statistics | statistics.mean(data) |
| collections | Advanced data structures | Counter, deque |
| itertools | Iterator tools | itertools.count() |
| pathlib | Work with file paths | Path("file.txt") |

## Example 1

```
PYTHON CODE
import random
number = random.randint(1, 10)
print(number)
```

Output: A random number between 1 and 10

Actual output changes every time because it is random.

# 13.4 Creating Your Own Modules

A Python file can be used as a module.

## File Structure

project/
│
├── calculator.py
└── main.py

### calculator.py

Code with Ishchal

```python
def add(a, b):
    return a + b
```

## main.py

```python
import calculator

result = calculator.add(10, 20)
print(result)
```

Output: 30

## Explanation

1. calculator.py is a module.
2. main.py imports the calculator module.
3. The function add() is accessed using calculator.add().
4. Both files should be in the same folder for this simple import to work.

# 13.5 Importing Specific Code from Your Own Module

## Syntax

from module_name import function_name

**File structure:**

```
project/
│
├── calculator.py
└── main.py
```

### Example 1

calculator.py

```python
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b
```

main.py\

```python
from calculator import add
print(add(10, 20))
```

Output: 30

CodeWithNishchal

# 13.6 Module Search Path

When we import a module, Python searches for it in specific locations.

## Explanation

Python searches in this general order:

1. Current working directory.
2. Paths listed in PYTHONPATH, if set.
3. Standard library directories.
4. Installed third-party package directories.

## Syntax

```python
import sys

print(sys.path)
```

# 13.7 Exploring a Module with dir()

dir() shows the names available inside a module.

## Syntax

dir(module_name)

## Explanation

1. dir() helps us inspect a module.
2. It shows functions, variables, classes, and special names inside the module.
3. It is useful while learning or debugging.

## Example 1

PYTHON CODE

```
import math

print(dir(math))
```

Output:
List of names available inside the math module
NOTE: dir() does not explain what each name does. For explanation, use help().

CodeWithNishchal

# 13.8 __name__ == "__main__"

This is used to control whether code should run directly or only when imported.

## Syntax

```python
if __name__ == "__main__":
    statement
```

1. Every Python file has a special variable called __name__.
2. If the file is run directly, __name__ becomes "__main__".
3. If the file is imported, __name__ becomes the module name.
4. This is useful for testing module code safely.
5. It prevents some code from running during import.

## Example 1

calculator.py

```python
def add(a, b):
    return a + b

if __name__ == "__main__":
    print(add(10, 20))
```

When running calculator.py directly: 30

When importing calculator.py into another file, the function is available, but the test print does not run automatically.

## Flow Chart

Python file runs
  |
  v
Is file run directly?
  |
  ├── Yes -> __name__ is "__main__"
  |
  └── No  -> __name__ is module name

CodeWithNishchal

# 13.9 Package Structure

A package is a folder containing Python modules.

## Basic Package Structure

```
project/
│
├── main.py
│
└── mypackage/
    ├── __init__.py
    ├── calculator.py
    └── messages.py
```

## Explanation

1. mypackage is a package.
2. calculator.py and messages.py are modules inside the package.
3. __init__.py marks the folder as a regular Python package.
4. main.py can import modules from the package.

## Example 1

mypackage/calculator.py

```
PYTHON CODE
def add(a, b):
    return a + b
```

## main.py

```
PYTHON CODE
from mypackage.calculator import add


print(add(10, 20))
```

Output: 30

Code With Nishchal

# 13.10 __init__.py

__init__.py is a special file used in Python packages.

## Explanation

1. __init__.py is placed inside a package folder.
2. It tells Python that the folder is a regular package.
3. It can be empty.
4. It can also contain package-level initialization code.
5. It can control what is exposed when importing from the package.

mypackage/

├── __init__.py

├── calculator.py

└── messages.py

## Example 1

mypackage/__init__.py

```
PYTHON CODE
from .calculator import add
```

### main.py

```
PYTHON CODE
from mypackage import add

print(add(10, 20))
```

Output: 30

CodeWithNishchal

# 13.11 Absolute Imports

An absolute import uses the full path from the project/package root.

## Syntax

from package.module import name

## Explanation

1. Absolute imports are clear and easy to understand.
2. They show the full location of the imported module.
3. They are generally preferred in larger projects.
4. They reduce confusion compared to complex relative imports.

## Example 1

Inside main.py:

from app.calculator import add

**This means:**

From package app,
inside module calculator,
import add

# 13.12 Relative Imports

A relative imports code based on the current module's location inside a package.

## Syntax

from .module import name

from ..package import module

## Explanation

1. Relative imports use dots.
2. One dot . means current package.
3. Two dots .. means parent package.
4. Relative imports are used inside packages.
5. They are not meant for simple standalone scripts.

CodeWithNishchal

| Syntax | Meaning |
|---|---|
| . | Current package |
| .. | Parent package |
| ... | Grandparent package |

## Example Structure

```
project/
│
└── app/
    ├── __init__.py
    ├── calculator.py
    └── reports.py
```

Inside reports.py:

```
from .calculator import add
```

**This means:** Import add from calculator.py in the same package

## Important Point

Relative imports work properly when the module is part of a package. They may fail if the file is run directly as a script.
Better way to run a package module:
python -m app.reports

# 13.13 Package Management Basics

Package management means installing, updating, removing, and tracking external Python packages. Python commonly uses pip for package management.

## Explanation

1. The Python standard library comes with Python.
2. Third-party packages must be installed separately.
3. pip is used to install third-party packages.
4. Virtual environments are used to keep project dependencies separate.
5. requirements.txt is used to record project dependencies.

---

CodeWithNishchal

# 13.14 pip Basics

pip is Python's package installer.

## Common pip Commands

| Task | Command |
|---|---|
| Check pip version | python -m pip --version |
| Install package | python -m pip install package_name |
| Install specific version | python -m pip install package_name==1.2.3 |
| Upgrade package | python -m pip install --upgrade package_name |
| Uninstall package | python -m pip uninstall package_name |
| Show installed packages | python -m pip list |
| Show package details | python -m pip show package_name |

**NOTE:** Use python -m pip instead of only pip. This makes sure pip belongs to the Python version you are using.

# 13.15 Virtual Environment Basics

A virtual environment is an isolated Python environment for a project.

1. Different projects may need different package versions.
2. It prevents package conflicts.
3. It keeps the global Python installation clean.
4. It makes projects easier to share and manage.

## Create Virtual Environment

| System | Command |
|---|---|
| Windows | python -m venv venv |
| macOS/Linux | python3 -m venv venv |

## Activate Virtual Environment

CodeWithNishchal

| System | Command |
|---|---|
| Windows PowerShell | venv\Scripts\Activate.ps1 |
| Windows CMD | venv\Scripts\activate.bat |
| macOS/Linux | source venv/bin/activate |

### Deactivate

deactivate

# 13.16 requirements.txt

requirements.txt stores a list of packages needed for a project.

### Syntax

package_name==version

### Example

requests==2.32.3

numpy==2.0.0

| Task | Command |
|---|---|
| Save installed packages | python -m pip freeze > requirements.txt |
| Install from file | python -m pip install -r requirements.txt |

### Explanation

1. requirements.txt helps share project dependencies.
2. Another user can install the same packages using one command.
3. It is commonly used in Python projects.

CodeWithNishchal

# 13.17 Third-Party Packages

Third-party packages are packages created by other developers.

They are not part of the Python standard library.

| Package | Common Use |
|---|---|
| requests | Sending HTTP requests |
| numpy | Numerical computing |
| pandas | Data analysis |
| flask | Web development |
| django | Web applications |
| pytest | Testing |

# 13.18 Circular Import Warning

A circular import happens when two modules import each other.

## Example Structure

a.py imports b.py

b.py imports a.py

1. Circular imports can cause errors or incomplete imports.
2. They usually happen when modules depend on each other too much.
3. Good project structure helps avoid circular imports.
4. Shared code can be moved into a separate module.

## Example Problem

a.py -> imports b.py

b.py -> imports a.py

Python may not finish loading one module before the other needs it.

## Better Structure

common.py -> shared logic

a.py imports common.py

b.py imports common.py

CodeWithNishchal

# Chapter 14. Object-Oriented Programming in Python

Object-Oriented Programming, or **OOP**, is a programming style where code is organized using **classes** and **objects**.

# 14.1.1 OOPS Fundamentals

## Basic idea

Class  -> blueprint
Object -> real item created from class

**Example idea:**

**Class:** Student;
**Objects:**
student1
student2
student3

## Why OOP is Used

1. To organize large programs.
2. To reuse code.
3. To group data and behavior together.
4. To model real-world entities.
5. To make code easier to maintain.

# 14.1.1 Classes

A **class** is a blueprint for creating objects.

## Syntax

```python
class ClassName:
    statement
```

1. A class defines the structure of an object.
2. A class can contain attributes and methods.
3. Class names usually use PascalCase.
4. A class does not represent one real object by itself.
5. Objects are created from classes.

CodeWithNishchal

## Example 1

```
PYTHON CODE
class Student:
    pass

student1 = Student()
print(type(student1))
```

Output: <class '__main__.Student'>

## Class Naming Style

| Good Class Name | Reason |
|---|---|
| Student | Clear class name |
| BankAccount | Uses PascalCase |
| EmployeeRecord | Meaningful name |

# 14.1.2 Objects

An **object** is an instance created from a class.

## Syntax

object_name = ClassName()

1. An object is created from a class.
2. One class can create many objects.
3. Each object can have its own data.
4. Objects are also called instances.

## Example 1

```
PYTHON CODE
class Student:
    pass
student1 = Student()
student2 = Student()
print(student1)
print(student2)
```

CodeWithNishchal

Output:
<__main__.Student object at ...>
<__main__.Student object at ...>
The memory address may be different in every run.

## Class vs Object

| Point | Class | Object |
|---|---|---|
| Meaning | Blueprint | Real instance |
| Created using | class keyword | Class name with () |
| Example | Student | student1 = Student() |
| Memory | No object data yet | Stores actual object data |

# 14.1.3 Attributes

Attributes are variables that belong to an object or class.

## Syntax

object_name.attribute_name = value

## Explanation

1. Attributes store data about an object.
2. Each object can have different attribute values.
3. Attributes are accessed using dot . notation.

## Example 1

PYTHON CODE

```
class Student:
    pass


student1 = Student()
student1.name = "Aman"
student1.age = 21
print(student1.name)
print(student1.age)
```

Output: Aman
21

devWithNishchal

## Attribute Table

| Code | Meaning |
|---|---|
| student1.name | Accesses name attribute |
| student1.age | Accesses age attribute |
| student1.name = "Aman" | Creates/updates attribute |

# 14.1.4 Methods

A **method** is a function defined inside a class.

## Syntax

```python
class ClassName:
    def method_name(self):
        statement
```

1. Methods define object behavior.
2. Methods are functions inside a class.
3. Instance methods usually take self as the first parameter.
4. Methods are called using an object.

## Example 1

```
PYTHON CODE
class Student:
    def greet(self):
        print("Hello")

student1 = Student()
student1.greet()
```

Output: Hello

# 14.1.5 self

deWithNishchal

self refers to the current object.

## Syntax

```
def method_name(self):
    statement
```

1. self represents the object that is calling the method.
2. It is used to access object attributes and methods.
3. Python automatically passes the object as self.
4. self is not a keyword, but it is the standard convention.
5. Always use self for instance methods.

## Example 1

PYTHON CODE

```
class Student:
    def show_name(self):
        print(self.name)

student1 = Student()
student1.name = "Aman"
student1.show_name()
```

Output: Aman

# 14.1.6 __init__

__init__ is a special method used to initialize object data.

## Syntax

```
Python

class ClassName:
    def __init__(self):
        statement
```

1. __init__ runs automatically when an object is created.
2. It is used to set initial attribute values.
3. It is commonly called a constructor.
4. Technically, __init__ initializes the object after it is created.
5. The actual object creation is handled by __new__, which is advanced and usually not needed in core notes.

## Example 1

CodeWithNishchal

```
PYTHON CODE
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

student1 = Student("Aman", 21)
print(student1.name)
print(student1.age)
```

**Output:** Aman
21

# 14.1.7 Instance Variables

Instance variables are variables that belong to a specific object.

## Syntax

self.variable_name = value

1. Instance variables are usually created inside __init__.
2. Each object gets its own copy.
3. Changing one object's instance variable does not affect another object.
4. They are accessed using object.variable_name.

## Example 1

```
PYTHON CODE
class Student:
    def __init__(self, name):
        self.name = name


student1 = Student("Aman")
student2 = Student("Riya")
print(student1.name)
print(student2.name)
```

Output: Aman
Riya

**NOTE:** student1.name and student2.name are separate instance variables.

# 14.1.8 Class Variables

WithNishchal

Class variables are variables shared by all objects of a class.

## Syntax

```python
class ClassName:
    class_variable = value
```

## Explanation

1. Class variables are defined directly inside the class.
2. They are shared by all objects.
3. They are useful for common data.
4. They can be accessed using the class name or object name.
5. Prefer accessing class variables using the class name.

## Example 1

```
PYTHON CODE
class Student:
    school_name = "ABC School"

    def __init__(self, name):
        self.name = name

student1 = Student("Aman")
student2 = Student("Riya")
print(student1.school_name)
print(student2.school_name)
print(Student.school_name)
```

**Output:**
ABC School
ABC School
ABC School

## Instance Variable vs Class Variable

hNishchal

| Point | Instance Variable | Class Variable |
|---|---|---|
| Belongs to | Object | Class |
| Created where | Usually inside __init__ | Inside class, outside methods |
| Shared? | No | Yes |
| Access | self.name | ClassName.variable |
| Example | self.name | school_name |

## Important Warning

Avoid mutable class variables unless shared data is intentional.

```python
class Student:
    subjects = []
```

This list is shared by all objects, which can cause unexpected behavior.

# 14.1.9 Methods vs Functions

A function is independent.
A method belongs to a class or object.

## Comparison Table

| Point | Function | Method |
|---|---|---|
| Defined where? | Outside class | Inside class |
| Called using | Function name | Object or class |
| First parameter | No automatic self | Instance method gets self |
| Example | greet() | student.greet() |

# 14.2 OOPS Principles

# 14.2.1 Encapsulation

Code With Nishchal

Encapsulation means **binding data and methods together inside a class** and controlling how data is accessed or modified.

# Basic Idea

**Data + Methods = Encapsulation**

1. Encapsulation keeps related data and behavior inside one class.
2. It helps protect data from direct unwanted changes.
3. Python does not have strict private variables like Java or C++.
4. Python uses naming conventions to show access level.
5. Encapsulation is commonly handled using:
    1. Public attributes
    2. Protected attributes
    3. Private/name-mangled attributes
    4. Getter and setter methods
    5. @property

PYTHON CODE

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def get_balance(self):
        return self.__balance

account = BankAccount(1000)
account.deposit(500)
print(account.get_balance())
```

Output: 1500

Here, __balance is not directly accessed from outside. It is accessed using methods.

# Public, Protected, and Private Members

Python uses naming conventions for access control.

## Access Level Table

Nishchal

| Type | Syntax | Meaning | Example |
|---|---|---|---|
| Public | name | Can be accessed normally | self.name |
| Protected | _name | Meant for internal use | self._salary |
| Private / Name mangling | __name | Avoid accidental outside access | self.__pin |

## Example 1

```
PYTHON CODE
class Student:
    def __init__(self):
        self.name = "Aman"
        self._marks = 85
        self.__grade = "A"

student = Student()
print(student.name)
print(student._marks)
```

**Output:** Aman
85
Direct access to __grade will fail:
print(student.__grade)
**Output:** AttributeError: 'Student' object has no attribute '__grade'

## Important Point

Double underscore does not make data fully private.
It performs name mangling to avoid accidental access.
Internally, Python changes:
__grade -> _Student__grade
So technically it can still be accessed, but it should not be used directly.

# Getter and Setter Methods

Getter and setter methods are used to read and update private data safely.

CodeWithNishchal

```python
def get_value(self):
    return self.__value

def set_value(self, value):
    self.__value = value
```

1. Getter method reads private data.
2. Setter method updates private data.
3. Setter can validate data before updating.
4. This protects the object from invalid values.

## Example 1

PYTHON CODE

```python
class Student:
    def __init__(self, marks):
        self.__marks = marks

    def get_marks(self):
        return self.__marks

    def set_marks(self, marks):
        if marks >= 0:
            self.__marks = marks
        else:
            print("Marks cannot be negative")
student = Student(80)
student.set_marks(90)
print(student.get_marks())
```

Output: 90

# Encapsulation Using @property

Nishchal

@property allows a method to behave like an attribute.

Syntax

```python
@property
def attribute_name(self):
    return value
```

1. @property is a clean way to control access to data.
2. It allows validation before changing data.
3. It supports getter and setter behavior.
4. It makes code look simple while still protecting data.

## Example 1

PYTHON CODE

```
class Student:
    def __init__(self, marks):
        self.marks = marks

    @property
    def marks(self):
        return self._marks

    @marks.setter
    def marks(self, value):
        if value < 0:
            raise ValueError("Marks cannot be
negative")
        self._marks = value

student = Student(85)
student.marks = 95
print(student.marks)
```

Output: 95

# 14.2.2 Inheritance

Nishchal

Inheritance allows one class to reuse the properties and methods of another class.

## Syntax

```python
class ChildClass(ParentClass):
    statement
```

## Explanation

1. Inheritance supports code reuse.
2. The parent class is also called base class or superclass.
3. The child class is also called derived class or subclass.
4. The child class can use parent class attributes and methods.
5. The child class can also define its own attributes and methods.
6. The child class can override parent methods.
7. Inheritance represents an **is-a** relationship.

## Basic Example

PYTHON CODE

```python
class Animal:
    def eat(self):
        print("Eating")

class Dog(Animal):
    def bark(self):
        print("Barking")
dog = Dog()
dog.eat()
dog.bark()
```

Output: Eating
Barking

## Inheritance Flow

Parent class

hNishchal

|
v
Child class
|
v
Child class can reuse parent features

# Why Inheritance is Used

| Use | Meaning |
|---|---|
| Code reuse | Child class reuses parent code |
| Extensibility | Child class can add new features |
| Maintainability | Common logic stays in parent class |
| Polymorphism | Same method can behave differently |
| Real-world modeling | Represents is-a relationship |

Example:
Dog is an Animal
Car is a Vehicle
Student is a Person

# Types of Inheritance in Python

Python supports these major types of inheritance:

| Type | Meaning |
|---|---|
| Single inheritance | One child inherits from one parent |
| Multilevel inheritance | Child inherits from parent, and another child inherits from that child |
| Hierarchical inheritance | Multiple child classes inherit from one parent |
| Multiple inheritance | One child inherits from multiple parents |
| Hybrid inheritance | Combination of two or more inheritance types |

CodeWithNishchal

# Single Inheritance

Single inheritance means one child class inherits from one parent class.

## Syntax

```python
class Parent:
    pass

class Child(Parent):
    pass
```

**Example 1**

```
PYTHON CODE
class Animal:
    def eat(self):
        print("Eating")

class Dog(Animal):
    def bark(self):
        print("Barking")

dog = Dog()
dog.eat()
dog.bark()
```

**Output:** Eating
Barking

CodeWithNishchal

# Multilevel Inheritance

Multilevel inheritance means a class inherits from a child class, forming a chain.

## Syntax

```python
class A:
    pass

class B(A):
    pass

class C(B):
    pass
```

Example 1

```python
class Animal:
    def eat(self):
        print("Eating")

class Dog(Animal):
    def bark(self):
        print("Barking")

class Puppy(Dog):
    def weep(self):
        print("Weeping")

puppy = Puppy()
puppy.eat()
puppy.bark()
puppy.weep()
```

Output:
Eating
Barking
Weeping

# Hierarchical Inheritance

Hierarchical inheritance means multiple child classes inherit from one parent class.

Nishchal

## Syntax

```python
class Parent:
    pass

class Child1(Parent):
    pass

class Child2(Parent):
    pass
```

Example 1

```
PYTHON CODE
class Animal:
    def eat(self):
        print("Eating")

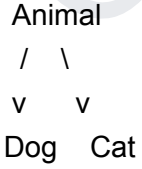
class Dog(Animal):
    def bark(self):
        print("Barking")

class Cat(Animal):
    def meow(self):
        print("Meowing")

dog = Dog()
cat = Cat()
dog.eat()
dog.bark()
cat.eat()
cat.meow()
```

Output: Eating
Barking
Eating
Meowing

## Diagram




# Multiple Inheritance

Multiple inheritance means one child class inherits from more than one parent class.

Code Nishchal

# Syntax

```
PYTHON CODE
class Parent1:
    pass

class Parent2:
    pass

class Child(Parent1, Parent2):
    pass
```

1. Python supports multiple inheritance.
2. A child class can use methods from multiple parent classes.
3. If parents have methods with the same name, Python uses MRO.
4. MRO means Method Resolution Order.

# Example 1

```
PYTHON CODE
class Father:
    def father_skill(self):
        print("Gardening")

class Mother:
    def mother_skill(self):
        print("Painting")

class Child(Father, Mother):
    def child_skill(self):
        print("Coding")

child = Child()

child.father_skill()
child.mother_skill()
child.child_skill()
```

**Output:**

Gardening
Painting
Coding

# Diagram

Father  Mother
  \  /

Nishchal

v v

Child

# Hybrid Inheritance

Hybrid inheritance is a combination of two or more types of inheritance.

1. Hybrid inheritance mixes different inheritance types.
2. It can include multiple, multilevel, or hierarchical inheritance together.
3. It is powerful but can become complex.
4. MRO is important in hybrid inheritance.

## Example 1

PYTHON CODE

```
class Person:
    def show_person(self):
        print("Person")

class Student(Person):
    def show_student(self):
        print("Student")

class Employee(Person):
    def show_employee(self):
        print("Employee")

class TeachingAssistant(Student, Employee):
    def show_ta(self):
        print("Teaching Assistant")

ta = TeachingAssistant()
ta.show_person()
ta.show_student()
ta.show_employee()
ta.show_ta()
```

Output:

Person

Student

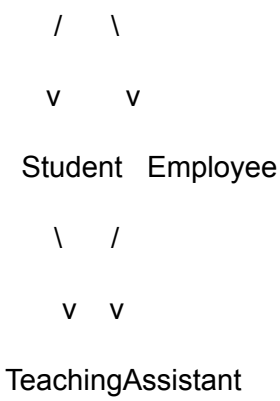
Employee

Teaching Assistant

## Diagram

Person

Nishchal

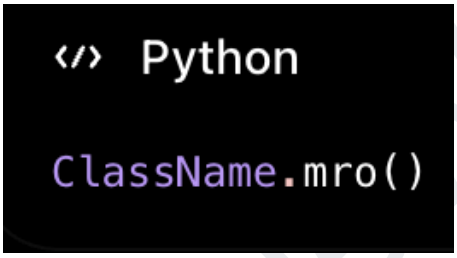


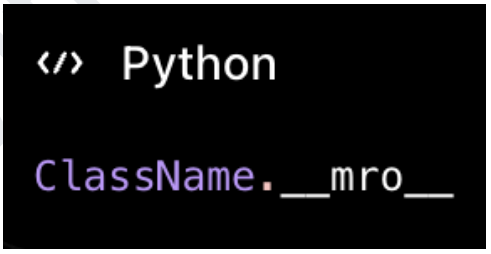

---

# 14.17 Method Resolution Order - MRO

MRO is the order Python follows while searching for methods in inheritance.

## Syntax



OR



1. MRO decides which method is called first.
2. It is important in multiple inheritance.
3. Python uses the C3 linearization algorithm for MRO.
4. The search starts from the child class.
5. Then Python checks parent classes according to MRO.
6. Finally, it checks the base object class.

## Example 1

PYTHON CODE
```
class A:
```

CodeWithNishchal

```
    def show(self):
        print("A")

class B(A):
    def show(self):
        print("B")

class C(A):
    def show(self):
        print("C")

class D(B, C):
    pass

obj = D()
obj.show()
print(D.mro())
```

Output: B

[<class '__main__.D'>, <class '__main__.B'>, <class '__main__.C'>, <class '__main__.A'>, <class 'object'>]

## MRO Flow

D -> B -> C -> A -> object

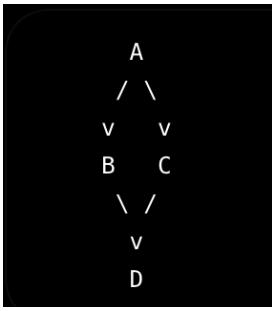
Since B comes before C, B.show() runs.

---

# Diamond Problem

CodeWithNishchal

The diamond problem happens when a child class inherits from two classes that both inherit from the same parent.




1. Class B and class C both inherit from A.
2. Class D inherits from both B and C.
3. If the same method exists in multiple classes, Python uses MRO to decide.
4. Python handles this safely using MRO.
5. super() also follows MRO.

## Example 1

PYTHON CODE

```
class A:
    def show(self):
        print("A")

class B(A):
    def show(self):
        print("B")

class C(A):
    def show(self):
        print("C")

class D(B, C):
    pass
d = D()
d.show()
```

**Output:** B
Because the MRO is:
D -> B -> C -> A -> object

# Constructor in Inheritance

Nishchal

When a child object is created, Python runs the child class __init__ method.
If the child class does not have __init__, Python uses the parent class __init__.

## Case 1: Child Has No __init__

PYTHON CODE

```
class Person:
    def __init__(self, name):
        self.name = name

class Student(Person):
    pass

student = Student("Aman")
print(student.name)
```

Output: Aman

The child class uses the parent class constructor.

## Case 2: Child Has Its Own __init__

PYTHON CODE

```
class Person:
    def __init__(self, name):
        self.name = name

class Student(Person):
    def __init__(self, course):
        self.course = course

student = Student("Python")
print(student.course)
```

Output:

Python

Here, parent __init__ does not run automatically because the child has its own __init__.

Nishchal

# super()

super() is used to call the next method in the MRO, usually from the parent class.

## Syntax

super().method_name()

## Explanation

1. super() is commonly used to call parent class methods.
2. It is commonly used inside __init__.
3. It avoids directly writing the parent class name.
4. In multiple inheritance, super() follows MRO.
5. It helps avoid repeating parent class logic.

## Example 1

PYTHON CODE

```
class Person:
    def __init__(self, name):
        self.name = name

class Student(Person):
    def __init__(self, name, course):
        super().__init__(name)
        self.course = course

student = Student("Aman", "Python")

print(student.name)
print(student.course)
```

Output:

Aman

Python

---

Nishchal

# 14.2.3 Method Overriding

Method overriding means a child class defines a method with the same name as a parent class method.

## Syntax

```python
class Parent:
    def method(self):
        statement

class Child(Parent):
    def method(self):
        new_statement
```

## Explanation

1. The method name is the same in parent and child.
2. The child class provides its own version.
3. When called on child object, child method runs.
4. This supports polymorphism.
5. Parent method can still be called using super().

## Example 1

PYTHON CODE

```
class Animal:
    def sound(self):
        print("Animal sound")

class Dog(Animal):
    def sound(self):
        print("Bark")

dog = Dog()
dog.sound()
```

Output:Bark

WithNishchal

## Overriding with super()

```
PYTHON CODE
class Animal:
    def sound(self):
        print("Animal sound"
class Dog(Animal):
    def sound(self):
        super().sound()
        print("Bark")
dog = Dog()
dog.sound()
```

Output: Animal sound
Bark

# 14.2.4 Polymorphism

Polymorphism means the same method or operation behaves differently for different objects.

## Explanation

1. Polymorphism means “many forms”.
2. The same method name can work differently in different classes.
3. Python supports polymorphism through:
    1. **Method overriding**
    2. **Duck typing**
    3. **Operator overloading**

```
PYTHON CODE
class Dog:
    def sound(self):
        print("Bark")


class Cat:
    def sound(self):
        print("Meow")
animals = [Dog(), Cat()]
for animal in animals:
    animal.sound()
```

Output: Bark

Meow

WithNishchal

## Polymorphism Types in Python

| Type | Meaning | Example |
|---|---|---|
| Method overriding | Child changes parent method | Dog.sound() |
| Duck typing | Object is accepted if it has required method | animal.sound() |
| Operator overloading | Operator works differently by class | obj1 + obj2 |

# 14.2.5 Duck Typing

Duck typing means Python focuses on what an object can do, not its exact type.

## Basic Idea

If an object has the needed method, Python can use it.

## Example 1

PYTHON CODE

```
class Duck:
    def speak(self):
        print("Quack")

class Person:
    def speak(self):
        print("Hello")

def call_speak(obj):
    obj.speak()

call_speak(Duck())
call_speak(Person())
```

Output:
Quack
Hello

Python does not care whether the object is Duck or Person. It only checks whether speak() exists.

Nishchal

# 14.2.6 Abstraction

Abstraction means hiding internal implementation and showing only essential features.

## Explanation

1. Abstraction focuses on what an object does.
2. It hides how the object does it internally.
3. Python supports abstraction using abstract base classes.
4. Abstract base classes are created using the abc module.
5. A class with abstract methods cannot be instantiated directly.
6. Child classes must implement abstract methods.

## Example 1

PYTHON CODE

```
from abc import ABC, abstractmethod

class Payment(ABC):
    @abstractmethod
    def pay(self, amount):
        pass

class UpiPayment(Payment):
    def pay(self, amount):
        print("Paid", amount, "using UPI")

payment = UpiPayment()

payment.pay(500)
```

Output: Paid 500 using UPI

## Important Point

Payment cannot be used directly because it has an abstract method.

This gives an error:

payment = Payment()

**Output:**

TypeError: Can't instantiate abstract class Payment with abstract method pay

Nishchal

# Encapsulation vs Abstraction

| Point | Encapsulation | Abstraction |
|---|---|---|
| Meaning | Protecting and controlling data | Hiding implementation details |
| Focus | Data access | Essential behavior |
| Achieved by | Private variables, methods, @property | Abstract classes, interfaces |
| Example | Hide __balance | Define pay() without showing payment logic |

**Encapsulation** = How data is protected
**Abstraction** = How unnecessary details are hidden

# 14.2.7 Composition

Composition means one class contains an object of another class.

## Basic Idea

Inheritance = is-a relationship
Composition = has-a relationship

1. Composition is used when one object is made of another object.
2. It represents a **has-a** relationship.
3. It is often preferred over inheritance when there is no true is-a relationship.

## Example 1

```
PYTHON CODE
class Engine:
    def start(self):
        print("Engine started")

class Car:
    def __init__(self):
        self.engine = Engine()

    def start(self):
        self.engine.start()
car = Car()
car.start()
```

Output:Engine started
**Explanation:**
Car has an Engine.
So this is composition.

WithNishchal

# Association, Aggregation, & Composition

| Relationship | Meaning | Example |
|---|---|---|
| Association | One class uses another class | Teacher teaches Student |
| Aggregation | One class has another, but both can exist independently | Department has Teachers |
| Composition | One class owns another strongly | House has Rooms |

## Important Difference

| Point | Aggregation | Composition |
|---|---|---|
| Relationship | Weak has-a | Strong has-a |
| Child object can exist alone? | Yes | Usually no |
| Example | Team has Players | Car has Engine |

# isinstance() and issubclass()

These functions are useful when working with inheritance.

## Syntax

isinstance(object, ClassName)
issubclass(ChildClass, ParentClass)

1. isinstance() checks whether an object belongs to a class.
2. It also returns True if the object belongs to a child class.
3. issubclass() checks whether one class inherits from another class.
4. Both return True or False.

## Example 1

PYTHON CODE
```
class Animal:
    pass

class Dog(Animal):
    pass

dog = Dog()
print(isinstance(dog, Dog))
print(isinstance(dog, Animal))
print(issubclass(Dog, Animal))
```

Output: True
True
True

devWithNishchal

# Inheritance vs Composition

| Point | Inheritance | Composition |
|---|---|---|
| **Relationship** | Is-a | Has-a |
| **Code style** | Child class extends parent | Class contains another object |
| **Example** | Dog is an Animal | Car has an Engine |
| **Reuse method** | Inherit methods | Use contained object methods |
| **Flexibility** | Can become tightly connected | Usually more flexible |

## Use Inheritance When

Child really is a type of parent.

Example:Dog is an Animal

Student is a Person

## Use Composition When

One object has or uses another object.

Example:

Car has an Engine

Computer has a Processor

# Complete Inheritance Types Summary

| Type | Structure | Example |
|---|---|---|
| Single | A -> B | Animal -> Dog |
| Multilevel | A -> B -> C | Animal -> Dog -> Puppy |
| Hierarchical | A -> B, A -> C | Animal -> Dog, Animal -> Cat |
| Multiple | A + B -> C | Father + Mother -> Child |
| Hybrid | Combination | Person -> Student/Employee -> TeachingAssistant |

# OOP Principles Summary

CodeWithNishchal

| Principle | Meaning | Python Feature |
|---|---|---|
| Encapsulation | Control access to data | _name, __name, getter/setter, @property |
| Inheritance | Reuse parent class code | class Child(Parent) |
| Polymorphism | Same method, different behavior | Overriding, duck typing |
| Abstraction | Hide implementation details | ABC, @abstractmethod |
| Composition | Build one object using another | Object inside object |

# Important OOP Scenario Table

| Scenario | Use |
|---|---|
| Need to protect data | Encapsulation |
| Need to reuse common code | Inheritance |
| Need same method with different behavior | Polymorphism |
| Need to force child class method structure | Abstraction |
| Need one object inside another | Composition |
| Need multiple parent classes | Multiple inheritance |
| Need to resolve method search order | MRO |
| Need parent constructor logic | super() |
| Need to check object/class relationship | isinstance(), issubclass() |

CodeWithNishchal

## 14.3 Special Methods / Magic Methods / Dunder Methods

Special methods are predefined methods in Python with double underscores before and after their names.

They are also called:

- Special methods
- Magic methods
- Dunder methods

dunder means **double underscore**.

Syntax

```python
def __method_name__(self):
    statement
```

# Explanation

1. Special methods allow objects to work with Python's built-in operations.
2. They are called automatically by Python.
3. They usually start and end with double underscores.
4. We normally do not call them directly.
5. Instead, we use operators or built-in functions.

# Example Idea

print(obj)  -> calls obj.__str__()

len(obj)  -> calls obj.__len__()

obj[0]  -> calls obj.__getitem__(0)

obj1 + obj2  -> calls obj1.__add__(obj2)

# Common Special Methods Table

CodeWithNishchal

| Special Method | Triggered By | Purpose |
|---|---|---|
| __init__ | ClassName() | Initializes object |
| __str__ | str(obj), print(obj) | User-friendly string |
| __repr__ | repr(obj) | Developer-friendly string |
| __len__ | len(obj) | Returns length |
| __getitem__ | obj[index] | Indexing/slicing |
| __add__ | obj1 + obj2 | Addition |
| __sub__ | obj1 - obj2 | Subtraction |
| __call__ | obj() | Makes object callable |
| __enter__ | with obj: | Starts context manager |
| __exit__ | End of with block | Exits context manager |

# 14.3.1 __str__

__str__ returns a user-friendly string representation of an object.

## Syntax

```python
def __str__(self):
    return "text"
```

## Explanation

1. __str__ is called by str(obj).
2. It is also called by print(obj).
3. It should return a string.
4. It is mainly for users.
5. It should be readable and simple.

## Example 1

CodeWithNishchal

PYTHON CODE

```
class Student:
    def __init__(self, name, course):
        self.name = name
        self.course = course

    def __str__(self):
        return f"{self.name} is studying {self.course}"

student = Student("Aman", "Python")
print(student)
```

**Output:** Aman is studying Python

# 14.3.2 __repr__

__repr__ returns a developer-friendly string representation of an object.

## Syntax

```
Python

def __repr__(self):
    return "text"
```

1. __repr__ is called by repr(obj). It is mainly used for debugging.
2. It should return a string. A good __repr__ often looks like valid Python code.
3. If __str__ is not defined, Python may use __repr__ while printing.

PYTHON CODE

```
class Student:
    def __init__(self, name, course):
        self.name = name
        self.course = course

    def __repr__(self):
        return f"Student(name={self.name!r}, course={self.course!r})"
student = Student("Aman", "Python")
print(repr(student))
```

Output:Student(name='Aman', course='Python')

# 14.3.3 __len__

CodeWithNishchal

__len__ defines the behavior of len(obj).

## Syntax

```
Python

def __len__(self):
    return integer
```

## Explanation

1. __len__ is called by len(obj).
2. It must return a non-negative integer.
3. It is useful for custom container-like classes.
4. If __bool__ is not defined, Python may use __len__ to decide truthiness.
5. If __len__ returns 0, the object is considered False in Boolean context.

## Example 1

PYTHON CODE

```
class Team:
    def __init__(self, members):
        self.members = members

    def __len__(self):
        return len(self.members)

team = Team(["Aman", "Riya", "Kabir"])
print(len(team))
```

Output:3

# 14.3.4 __getitem__

__getitem__ defines indexing and slicing behavior.

Nishchal

## Syntax

```
def __getitem__(self, index):
    return value
```

1. __getitem__ is called when we use obj[index].
2. It allows custom objects to support indexing.
3. It can also support slicing. It is useful for custom sequence-like classes.
4. If implemented carefully, it can also help an object work in loops.

PYTHON CODE

```
class Team:
    def __init__(self, members):
        self.members = members

    def __getitem__(self, index):
        return self.members[index]
team = Team(["Aman", "Riya", "Kabir"])
print(team[0])
print(team[1])
print(team[0:2])
```

**Output:**

```
Aman
Riya
['Aman', 'Riya']
```

# 14.3.5 __add__

__add__ defines custom behavior for the + operator.

## Syntax

```
def __add__(self, other):
    return result
```

1. __add__ is called when obj1 + obj2 is used.
2. It is used for operator overloading.
3. It should return a new result.
4. If the other object type is not supported, return NotImplemented.
5. It should be used only when addition makes logical sense.

## Example 1

CodeWithNishchal

PYTHON CODE

```
class Money:
    def __init__(self, amount):
        self.amount = amount

    def __add__(self, other):
        if not isinstance(other, Money):
            return NotImplemented
        return Money(self.amount + other.amount)

    def __str__(self):
        return f"Amount: {self.amount}"

money1 = Money(100)
money2 = Money(50)
result = money1 + money2
print(result)
```

Output: Amount: 150

# 14.3.6 __sub__

__sub__ defines custom behavior for the - operator.

## Syntax

```
def __sub__(self, other):
    return result
```

1. __sub__ is called when obj1 - obj2 is used.
2. It is also part of operator overloading.
3. It should return a meaningful result.
4. Return NotImplemented if the other type is unsupported.

## Example 1

CodeWithPanchal

PYTHON CODE

```
class Money:
    def __init__(self, amount):
        self.amount = amount

    def __sub__(self, other):
        if not isinstance(other, Money):
            return NotImplemented
        return Money(self.amount - other.amount)

    def __str__(self):
        return f"Amount: {self.amount}"

money1 = Money(100)
money2 = Money(40)
result = money1 - money2
print(result)
```

Output: Amount: 60

---

# 14.3.7 __call__

__call__ allows an object to be called like a function.

## Syntax

```
def __call__(self):
    statement
```

## Explanation

1. __call__ runs when an object is called using parentheses.
2. It makes an object callable.
3. It is useful when an object stores data and also performs an action.
4. It is commonly used in decorators, callbacks, and callable classes.

CodeWithHarshal

## Example 1

```
PYTHON CODE
class Greeter:
    def __init__(self, name):
        self.name = name

    def __call__(self):
        print("Hello", self.name)

greet_aman = Greeter("Aman")
greet_aman()
```

Output: Hello Aman

# 14.3.8 __enter__ and __exit__

__enter__ and __exit__ are used to create context managers.Context managers work with the with statement.

## Syntax

```
PYTHON CODE
def __enter__(self):
    return self

def __exit__(self, exc_type, exc_value,
traceback):
    statement
```

## Explanation

1. __enter__ runs at the start of the with block.
2. __exit__ runs when the with block ends.
3. __exit__ runs even if an error occurs inside the with block.
4. __exit__ receives exception details if an error occurs.
5. If __exit__ returns True, the exception is suppressed.
6. If __exit__ returns False or None, the exception continues.

Courtesy Nishchal

PYTHON CODE

```
class SimpleContext:
    def __enter__(self):
        print("Entering")
        return self

    def __exit__(self, exc_type, exc_value, traceback):
        print("Exiting")

with SimpleContext():
    print("Inside with block")
```

Output:
Entering
Inside with block
Exiting

# 14.3.9 Context Manager Real Use

A common real-world example of a context manager is file handling.

## Example 1

```python
with open("data.txt", "w") as file:
    file.write("Hello Python")
```

## Explanation

1. open() returns a file object.
2. The file object works as a context manager.
3. The file opens at the start of the with block.
4. The file closes automatically at the end.
5. This is safer than manually closing the file.

---

Code With Nishchal

# 14.3.10 Operator Overloading

Operator overloading means giving custom behavior to operators for user-defined classes.

## Explanation

1. Operators like +, -, *, ==, <, and [] can be customized.
2. This is done using special methods.
3. Operator overloading should be logical.
4. Do not overload operators in a confusing way.
5. Operators internally call matching dunder methods.

## Operator Overloading Table

| Operator / Operation | Special Method | Example |
|---|---|---|
| + | __add__ | obj1 + obj2 |
| - | __sub__ | obj1 - obj2 |
| * | __mul__ | obj1 * obj2 |
| / | __truediv__ | obj1 / obj2 |
| // | __floordiv__ | obj1 // obj2 |
| % | __mod__ | obj1 % obj2 |
| ** | __pow__ | obj1 ** obj2 |
| == | __eq__ | obj1 == obj2 |
| != | __ne__ | obj1 != obj2 |
| < | __lt__ | obj1 < obj2 |
| <= | __le__ | obj1 <= obj2 |
| > | __gt__ | obj1 > obj2 |
| >= | __ge__ | obj1 >= obj2 |
| [] | __getitem__ | obj[index] |
| () | __call__ | obj() |
| len() | __len__ | len(obj) |
| str() | __str__ | str(obj) |
| repr() | __repr__ | repr(obj) |

CodeWithVishchal

# 14.3.11 Comparison Special Methods

Comparison methods allow objects to be compared.

## Syntax

```python
def __eq__(self, other):
    return result
```

## Explanation

1. __eq__ defines equality using ==.
2. __lt__ defines less than using <.
3. Other comparison methods work similarly.
4. These methods should return True or False.
5. Return NotImplemented if comparison with the other type is unsupported.

## Example 1

PYTHON CODE

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def __eq__(self, other):
        if not isinstance(other, Student):
            return NotImplemented
        return self.marks == other.marks

student1 = Student("Aman", 85)
student2 = Student("Riya", 85)
print(student1 == student2)
```

Output: True

Here, two students are considered equal because their marks are equal.

Prabhat Nishchal

## Comparison Methods Table

| Method | Operator |
|---|---|
| __eq__ | == |
| __ne__ | != |
| __lt__ | < |
| __le__ | <= |
| __gt__ | > |
| __ge__ | >= |

# 14.3.12 Reverse and In-place Operator Methods

Python also supports reverse and in-place operator methods.

## Explanation

1. Reverse methods are used when the left object does not support the operation.
2. In-place methods are used for operators like +=, -=, *=.
3. These are advanced but useful to know.

## Table

| Type | Example Operator | Method |
|---|---|---|
| Normal addition | obj + other | __add__ |
| Reverse addition | other + obj | __radd__ |
| In-place addition | obj += other | __iadd__ |
| Normal subtraction | obj - other | __sub__ |
| Reverse subtraction | other - obj | __rsub__ |
| In-place subtraction | obj -= other | __isub__ |

CodeWithNishchal

# 14.3.13 __bool__

__bool__ defines truth value behavior of an object.

## Syntax

```python
def __bool__(self):
    return True_or_False
```

## Explanation

1. __bool__ is called by bool(obj).
2. It is also used in if obj: conditions.
3. It must return True or False.
4. If __bool__ is not defined, Python may use __len__.
5. If both are missing, most objects are considered True.

## Example 1

PYTHON CODE

```python
class Cart:
    def __init__(self, items):
        self.items = items

    def __bool__(self):
        return len(self.items) > 0

cart = Cart(["Book"])

if cart:
    print("Cart has items")
```

Output: Cart has items

Prithi Vishchal

# 14.3.14 __iter__ and __next__

These methods make an object iterable.

## Syntax

```
PYTHON CODE
def __iter__(self):
    return self

def __next__(self):
    return next_value
```

1. __iter__ returns an iterator object. __next__ returns the next value.
2. When no value is left, __next__ should raise StopIteration.
3. These methods are used by loops. They are part of the iterator protocol.

```
PYTHON CODE
class CountUpTo:
    def __init__(self, limit):
        self.current = 1
        self.limit = limit

    def __iter__(self):
        return self

    def __next__(self):
        if self.current > self.limit:
            raise StopIteration

        value = self.current
        self.current += 1
        return value
counter = CountUpTo(3)
for number in counter:
    print(number)
```

Output:
1
2
3

Nishchal

# 14.3.15 __contains__

__contains__ defines behavior for the in operator.

## Syntax

```
def __contains__(self, item):

    return True_or_False
```

## Explanation

1. __contains__ is called by item in obj.
2. It should return True or False.
3. It is useful for custom container-like classes.

## Example 1

PYTHON CODE

```
class Team:
    def __init__(self, members):
        self.members = members

    def __contains__(self, member):
        return member in self.members

team = Team(["Aman", "Riya", "Kabir"])

print("Aman" in team)
print("Neha" in team)
```

Output:

True

False

---

Nishchal

# Important Rules for Special Methods

| Rule | Explanation |
|---|---|
| Return correct type | __str__ and __repr__ must return string |
| __len__ must return integer | It should return non-negative integer |
| Use NotImplemented | For unsupported operator types |
| Avoid confusing behavior | Operators should behave logically |
| Do not call directly | Use len(obj), not obj.__len__() |

# Special Methods Summary

| Method | Meaning | Common Use |
|---|---|---|
| __str__ | User-friendly string | print(obj) |
| __repr__ | Developer-friendly string | repr(obj) |
| __len__ | Defines length | len(obj) |
| __getitem__ | Defines indexing | obj[index] |
| __add__ | Defines addition | obj1 + obj2 |
| __sub__ | Defines subtraction | obj1 - obj2 |
| __call__ | Makes object callable | obj() |
| __enter__ | Starts context manager | with obj: |
| __exit__ | Ends context manager | End of with |
| __eq__ | Equality comparison | obj1 == obj2 |
| __lt__ | Less-than comparison | obj1 < obj2 |
| __bool__ | Truth value | if obj: |
| __iter__ | Returns iterator | for item in obj: |
| __next__ | Returns next item | Iterator protocol |
| __contains__ | Membership test | item in obj |

CodeWithNishchal

# Chapter 15: Advanced OOP in Python

Advanced OOP topics help us write cleaner, safer, and more reusable class-based code.

This section includes:

1. @classmethod
2. @staticmethod
3. @property
4. Abstract base classes
5. Mixins
6. Metaclasses
7. Data classes

---

## 15.1 @classmethod

A **class method** is a method that receives the class as its first argument.

The first parameter is usually named cls.

### Syntax

```
class ClassName:
    @classmethod
    def method_name(cls):
        statement
```

### Explanation

1. @classmethod is used to create a class method.
2. A class method receives the class automatically as cls.
3. It can access class variables.
4. It can modify class variables.
5. It can be called using the class name or object name.
6. It is commonly used to create alternate constructors.

### Example 1

CodeWithNishchal

PYTHON CODE

```
class Student:
    school_name = "ABC School"

    def __init__(self, name):
        self.name = name

    @classmethod
    def change_school(cls, new_school):
        cls.school_name = new_school

Student.change_school("XYZ School")

student1 = Student("Aman")

print(student1.school_name)
print(Student.school_name)
```

Output:

XYZ School

XYZ School

## Important Points

| Point | Explanation |
|---|---|
| First parameter | cls |
| Receives | Class |
| Can access class variables | Yes |
| Can access instance variables directly | No |
| Can be called by class | Yes |
| Can be called by object | Yes |

CodeWithNishchal

# 15.2 Class Method as Alternate Constructor

An alternate constructor means creating an object in a different way.

## Syntax

```python
@classmethod
def method_name(cls, data):
    return cls(...)
```

1. A class normally creates objects using __init__.
2. Sometimes data comes in a different format.
3. A class method can convert that data and return an object.
4. cls(...) creates an object of the current class.
5. This is useful for flexible object creation.

## Example 1

```python
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    @classmethod
    def from_string(cls, data):
        name, age = data.split("-")
        return cls(name, int(age))

student = Student.from_string("Aman-21")
print(student.name)
print(student.age)
```

Output:Aman

21

Nishchal

# 15.3 @staticmethod

A **static method** is a method inside a class that does not receive self or cls.

## Syntax

```
class ClassName:
    @staticmethod
    def method_name():
        statement
```

1. @staticmethod creates a static method.
2. It does not receive the object as self.
3. It does not receive the class as cls.
4. It behaves like a normal function placed inside a class.
5. It is used when the method is logically related to the class but does not need object or class data.

## Example 1

PYTHON CODE

```
class MathHelper:
    @staticmethod
    def add(a, b):
        return a + b
print(MathHelper.add(10, 20))
```

Output: 30

## Important Points

| Point | Explanation |
|---|---|
| First parameter | No automatic first parameter |
| Receives object? | No |
| Receives class? | No |
| Can access instance variables directly? | No |
| Can access class variables directly? | No |

Codewith Nishchal

## 15.4 Instance Method vs Class Method vs Static Method

| Point | Instance Method | Class Method | Static Method |
|---|---|---|---|
| Decorator | No decorator | @classmethod | @staticmethod |
| First parameter | self | cls | No automatic parameter |
| Receives | Object | Class | Nothing automatically |
| Access instance data | Yes | No direct access | No |
| Access class data | Yes | Yes | No direct access |
| Common use | Object behavior | Class-level behavior | Helper logic |
| Call style | obj.method() | Class.method() | Class.method() |

# 15.5 @property

@property allows a method to be accessed like an attribute.

## Syntax

```
class ClassName:
    @property
    def attribute_name(self):
        return value
```

## Explanation

1. @property is used to create managed attributes.
2. It allows method logic to run when accessing an attribute.
3. It helps with encapsulation.
4. It can be used for validation.
5. It makes code cleaner than normal getter methods.

CodeWithNishchal

## Example 1

PYTHON CODE

```
class Student:
    def __init__(self, marks):
        self._marks = marks

    @property
    def marks(self):
        return self._marks

student = Student(85)
print(student.marks)
```

Output: 85

**Important point:**

student.marks looks like an attribute, but internally it calls the marks() method.

# 15.6 @property Setter

A setter is used to control how a property value is updated.

## Syntax

```
@property
def name(self):
    return self._name

@name.setter
def name(self, value):
    self._name = value
```

## Explanation

1. A getter returns the value.
2. A setter updates the value.
3. The setter can validate the value before saving it.
4. This protects objects from invalid data.
5. The property name and setter name must match.

CodeWithNishchal

## Example 1

```python
class Student:
    def __init__(self, marks):
        self.marks = marks

    @property
    def marks(self):
        return self._marks

    @marks.setter
    def marks(self, value):
        if value < 0:
            raise ValueError("Marks cannot be negative")
        self._marks = value

student = Student(80)
student.marks = 95
print(student.marks)
```

Output: 95

## Getter and Setter Table

| Part | Purpose |
|---|---|
| @property | Reads value |
| @marks.setter | Updates value |
| self._marks | Internal storage attribute |
| student.marks | Public property access |

CodeWithNishchal

# 15.7 Read-only Property

A property becomes read-only if we define only getter and no setter.

## Syntax

@property

def property_name(self):

    return value

## Explanation

1. If no setter is defined, the property cannot be assigned directly.
2. This is useful for calculated values.
3. It protects values from direct modification.

PYTHON CODE

```
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width

    @property
    def area(self):
        return self.length * self.width

rectangle = Rectangle(10, 5)

print(rectangle.area)
```

Output:

50

Invalid update:

rectangle.area = 100

Output:

AttributeError: property 'area' of 'Rectangle' object has no setter

Nishchal

# 15.8 Abstract Base Classes

An **abstract base class** defines a common structure that child classes must follow.

## Syntax

```python
from abc import ABC, abstractmethod

class ClassName(ABC):
    @abstractmethod
    def method_name(self):
        pass
```

1. Abstract base classes are created using ABC.
2. Abstract methods are created using @abstractmethod.
3. A class with abstract methods cannot be instantiated directly.
4. Child classes must implement all abstract methods.
5. Abstract base classes are useful when many classes should follow the same structure.

**Abstract class**
**|**
**v**
**Defines required method**
**|**
**v**
**Child class must implement method**
**|**
**v**
**Child object can be created**

PYTHON CODE

```python
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Square(Shape):
    def __init__(self, side):
        self.side = side

    def area(self):
        return self.side * self.side

square = Square(5)
print(square.area())
```

Output: 25

WithNishchal

# 15.9 Why Abstract Base Classes Are Used

Abstract base classes are used when we want to force child classes to implement required methods.

Every payment class must have pay() method.
UPI payment -> pay()
Card payment -> pay()
Cash payment -> pay()

| Benefit | Explanation |
|---|---|
| Common structure | All child classes follow same design |
| Prevents incomplete classes | Child must implement abstract methods |
| Improves readability | Required methods are clear |
| Supports polymorphism | Same method name works across classes |

# 15.10 Mixins

A **mixin** is a small class that provides extra reusable behavior to another class.

## Syntax

```
class MixinName:
    def method_name(self):
        statement

class MainClass(MixinName):
    pass
```

1. A mixin is used to add extra features.
2. A mixin is usually not meant to be used alone.
3. Mixins are commonly used with multiple inheritance.
4. A mixin should be small and focused.
5. Mixin class names often end with Mixin.

CodeWithNishchal

## Example 1

```python
class JsonMixin:
    def to_json(self):
        return self.__dict__

class Student(JsonMixin):
    def __init__(self, name, age):
        self.name = name
        self.age = age

student = Student("Aman", 21)

print(student.to_json())
```

Output: {'name': 'Aman', 'age': 21}

## Important Points

| Point | Explanation |
|---|---|
| Purpose | Add reusable behavior |
| Usually used alone? | No |
| Common with | Multiple inheritance |
| Naming style | Ends with Mixin |
| Best design | Small and focused |

# 15.11 Mixin vs Normal Parent Class

| Point | Normal Parent Class | Mixin |
|---|---|---|
| Purpose | Represents main inheritance relationship | Adds extra behavior |
| Relationship | Usually is-a | Usually feature-based |
| Used alone | Often yes | Usually no |
| Example | Dog inherits Animal | Student uses JsonMixin |
| Size | Can be large | Should be small |

CodeWithNishchal

# 15.12 Metaclasses

A **metaclass** is the class of a class.

## Basic Idea

Object is created from class. Class is created from metaclass.

1. In Python, classes are also objects.
2. The default metaclass in Python is type.
3. A metaclass controls how a class is created.
4. Metaclasses are advanced.
5. Most normal Python programs do not need custom metaclasses.
6. For Core Python, understanding the basic idea is enough.

```python
class Student:
    pass

student = Student()

print(type(student))
print(type(Student))
```

Output: <class '__main__.Student'>
<class 'type'>

| Code | Meaning |
|---|---|
| type(student) | student is an object of Student |
| type(Student) | Student is an object of type |

# 15.13 Data Classes

A **data class** is a class mainly used to store data. It reduces repeated code like __init__, __repr__, and comparison methods.

## Syntax

```
from dataclasses import dataclass
@dataclass
class ClassName:
    field_name: type
```

CodeWithNishchal

## Explanation

1. Data classes are created using @dataclass.
2. They are useful for classes that mainly store data.
3. Fields are declared using type annotations.
4. Python automatically creates __init__.
5. Python automatically creates useful __repr__.
6. Data classes were introduced in Python 3.7.

## Example 1

PYTHON CODE

```
from dataclasses import dataclass


@dataclass
class Student:
    name: str
    age: int
    course: str


student = Student("Aman", 21, "Python")
print(student)
```

# 15.14 Normal Class vs Data Class

## Comparison Table

| Point | Normal Class | Data Class |
|---|---|---|
| Need to write __init__ | Yes | Automatically generated |
| Useful __repr__ | Need to write manually | Automatically generated |
| Type annotations | Optional | Used for fields |
| Best for | Behavior-heavy classes | Data-storing classes |
| Less repeated code | No | Yes |

---

CodeWithNishchal

# 15.15 Default Values in Data Classes

Data class fields can have default values.

## Syntax

@dataclass

class ClassName:

field1: type

field2: type = default_value

1. Fields can have default values.
2. Fields without default values must come before fields with default values.
3. This rule is similar to function parameters.

# 15.16 Mutable Defaults in Data Classes

Do not directly use mutable default values like lists in data classes.

## Wrong Style

```python
from dataclasses import dataclass

@dataclass
class Student:
    name: str
    subjects: list = []
```

This is not allowed in modern Python data classes.

## Correct Style : Use field(default_factory=list).

```python
from dataclasses import dataclass, field

@dataclass
class Student:
    name: str
    subjects: list = field(default_factory=list)

student1 = Student("Aman")
student2 = Student("Riya")

student1.subjects.append("Python")

print(student1.subjects)
print(student2.subjects)
```

Output: ['Python']

[]

1. Mutable defaults can accidentally be shared.
2. default_factory=list creates a new list for each object.
3. This avoids shared mutable data problems.

NishchalNishchal

# 15.17 Frozen Data Classes

A frozen data class creates objects that cannot be modified after creation.

## Syntax

```python
@dataclass(frozen=True)
class ClassName:
    field_name: type
```

## Explanation

1. frozen=True makes data class objects immutable-like.
2. After object creation, fields cannot be reassigned normally. It is useful for fixed data.
3. It is similar in idea to immutability, but internal mutable fields can still be modified if they exist.

## Example 1

PYTHON CODE

```
from dataclasses import dataclass


@dataclass(frozen=True)
class Point:
    x: int
    y: int


point = Point(10, 20)
print(point)
```

**Output:**
Point(x=10, y=20)
**Invalid update:**
point.x = 50
Output: dataclasses.FrozenInstanceError: cannot assign to field 'x'

hNishchal

# Chapter 16. File Handling and Error Management

This chapter has three main parts:

1. **File Operations**
2. **Exception Handling**
3. **File Formats**

File handling is used to store and read data from files.
Exception handling is used to handle runtime errors safely.

# 16.1 File Operations

## 16.1.1 File Handling

File handling means working with files using Python.

Python can:

1. Open files
2. Read files
3. Write files
4. Append data
5. Close files
6. Work with text files, binary files, CSV, JSON, and XML

# 16.1.2 Opening Files

A file is opened using the open() function.

## Syntax

file = open("file_name", "mode")

**With encoding:**

file = open("file_name", "mode", encoding="utf-8")

## Explanation

1. open() opens a file.
2. The first argument is the file name or path.
3. The second argument is the file mode.
4. Encoding is commonly used for text files.
5. utf-8 is a common and recommended encoding.

## Example 1

CodeWithNishchal

file = open("data.txt", "r", encoding="utf-8")
This opens data.txt in read mode.

# 16.1.3 Closing Files

## Syntax

file.close()

1. Closing a file releases system resources. If a file is not closed, data may not be saved properly.
2. The with statement is preferred because it closes the file automatically.

## Example 1

file = open("data.txt", "r", encoding="utf-8")
file.close()

# 16.1.4 Reading Files

Reading means getting data from a file.

| Method | Meaning |
|---|---|
| read() | Reads the entire file |
| readline() | Reads one line |
| readlines() | Reads all lines into a list |

## Example 1

file = open("data.txt", "r", encoding="utf-8")
content = file.read()
print(content)
file.close()
**Output:** Hello Python
Welcome to file handling

CodeWithNishchal

# 16.1.5 Reading Line by Line

Reading line by line is useful for large files.

## Syntax

```
for line in file:
    statement
```

1. This reads one line at a time.
2. It is memory-friendly.
3. It is better than reading a very large file at once.

## Example 1

```
with open("data.txt", "r", encoding="utf-8") as file:
    for line in file:
        print(line.strip())
```

Output: Hello Python

Welcome to file handling

strip() removes extra newline characters from the line.

# 16.1.6 Writing Files

Writing means saving data into a file.

## Syntax

```
open("file_name", "w")
```

1. "w" means write mode.If the file does not exist, Python creates it.
2. If the file already exists, old content is removed.
3. New content is written from the beginning.

## Example 1

```
with open("data.txt", "w", encoding="utf-8") as file:
    file.write("Hello Python")
```

This writes text into data.txt.
**NOTE:** Write mode overwrites old file content.

CodeWithNishchal

# 16.1.7 Appending Files

Appending means adding new data at the end of an existing file.

## Syntax

open("file_name", "a")

## Explanation

1. "a" means append mode. New data is added at the end.
2. Old content is not removed. If the file does not exist, Python creates it.

## Example 1

```
with open("data.txt", "a", encoding="utf-8") as file:
    file.write("\nNew line added")
```

# 16.1.8 File Modes

File modes tell Python how the file should be opened.

| Mode | Meaning | File Must Exist? | Old Content |
|---|---|---|---|
| "r" | Read text file | Yes | Not changed |
| "w" | Write text file | No | Removed if file exists |
| "a" | Append text file | No | Kept |
| "x" | Create new file | No | Error if file exists |
| "b" | Binary mode | Depends | Depends |
| "t" | Text mode | Depends | Depends |
| "+" | Read and write | Depends | Depends |

CodeWithNishchal

## Common Mode Combinations

| Mode | Meaning |
|---|---|
| "rt" | Read text file |
| "wt" | Write text file |
| "at" | Append text file |
| "rb" | Read binary file |
| "wb" | Write binary file |
| "r+" | Read and write existing file |
| "w+" | Write and read, overwrites file |
| "a+" | Append and read |

**Important point:** Default mode is "rt", which means read text.

# 16.1.9 with Statement

The with statement is the recommended way to work with files.

## Syntax

```
with open("file_name", "mode") as file:
    statement
```

1. with automatically closes the file. It is safer than manually using close().
2. It works even if an error happens inside the block. It makes file handling cleaner.

## Example 1

```
with open("data.txt", "r", encoding="utf-8") as file:
    content = file.read()
    print(content)
```

**No need to write:**

```
file.close()
```

CodeWithNishchal

# 16.1.10 File Object Methods

| Method | Meaning |
|---|---|
| read() | Reads entire file |
| read(size) | Reads given number of characters/bytes |
| readline() | Reads one line |
| readlines() | Reads all lines into a list |
| write(text) | Writes text |
| writelines(list) | Writes multiple strings |
| seek(position) | Moves file pointer |
| tell() | Returns current file pointer position |
| close() | Closes file |
| flush() | Forces buffered data to be written |

# 16.1.11 Binary Files

Binary files store data in bytes.

- Images
- Videos
- Audio files
- PDF files
- Executable files

## Syntax

open("file_name", "rb")
open("file_name", "wb")

## Explanation

1. "rb" means read binary."wb" means write binary.
2. Binary mode works with bytes, not normal strings.
3. Encoding is not used in binary mode.Binary files are useful for non-text data.

CodeWithNishchal

# 16.1.12 File Paths

| Type | Meaning | Example |
|---|---|---|
| Relative path | Path from current folder | "data.txt" |
| Absolute path | Full path from root/drive | "C:/Users/Aman/data.txt" |
| Folder path | File inside folder | "files/data.txt" |
| Parent folder path | File in parent folder | "../data.txt" |

# 16.1.13 Basic os Module

The os module helps work with the operating system.

## Syntax

import os

## Common os Functions

| Function | Meaning |
|---|---|
| os.getcwd() | Gets current working directory |
| os.listdir() | Lists files and folders |
| os.mkdir("folder") | Creates folder |
| os.remove("file.txt") | Deletes file |
| os.rename("old", "new") | Renames file/folder |
| os.path.exists(path) | Checks if path exists |
| os.path.isfile(path) | Checks if path is a file |
| os.path.isdir(path) | Checks if path is a folder |
| os.path.join(a, b) | Joins paths safely |

# 16.1.14 Basic pathlib Module

CodeWithNishchal

pathlib is a modern way to work with file paths.

## Syntax

from pathlib import Path

1. pathlib works with paths as objects.It is cleaner than manually joining strings.
2. It is recommended for modern Python code.It works across operating systems.

## Common pathlib Methods

| Code | Meaning |
|---|---|
| Path("data.txt") | Creates path object |
| path.exists() | Checks if path exists |
| path.is_file() | Checks if path is file |
| path.is_dir() | Checks if path is folder |
| path.read_text() | Reads text file |
| path.write_text() | Writes text file |
| path.mkdir() | Creates folder |
| path.unlink() | Deletes file |

# 16.1.15 Encoding

Encoding decides how text is stored in bytes.

**Syntex:** open("file.txt", "r", encoding="utf-8")

1. Text files store text using an encoding.
2. utf-8 supports most common characters.
3. Always mention encoding when working with text files.
4. Encoding problems can cause UnicodeDecodeError.

## Example 1

```
with open("data.txt", "w", encoding="utf-8") as file:
    file.write("Python is easy")
```

# 16.2 Exception Handling

CodeWithNishchal

An exception is an error that happens while the program is running. Exception handling allows us to handle errors safely without crashing the whole program.

## Example Runtime Error

number = int("abc")

**Output:** ValueError: invalid literal for int() with base 10: 'abc'

## Why Exception Handling is Used

1. To prevent sudden program crashes.
2. To handle risky code safely.
3. To clean up resources properly.
4. To continue program execution when possible.

# 16.2.1 try-except

try-except is used to handle exceptions.

## Syntax

```
try:
    risky_code
except ExceptionType:
    handling_code
```

1. Code that may cause an error is written inside try.
2. Error handling code is written inside except.
3. If an exception occurs, Python jumps to the matching except block.
4. If no exception occurs, the except block is skipped.

## Flow Chart

```
try block runs
  |
  v
Error occurs?
  |
  ├── Yes -> except block runs
  |
  └── No  -> except block skipped
```

## Example 1

CodeWithNishchal

```python
try:
    number = int("abc")
except ValueError:
    print("Invalid number")
```

**Output:** Invalid number

# 16.2.2 Catching Exception Object

We can store the exception object using as.

## Syntax

```
except ExceptionType as error:
    statement
```

## Example 1

```python
try:
    number = int("abc")
except ValueError as error:
    print(error)
```

**Output:** invalid literal for int() with base 10: 'abc'

# 16.2.3 Multiple except Blocks

Multiple except blocks are used to handle different errors differently.

## Syntax

```python
try:
    risky_code
except ErrorType1:
    handling_code
except ErrorType2:
    handling_code
```

CodeWithNishchal

## Explanation

1. Different exceptions can need different handling.
2. Specific exceptions should come before general exceptions.
3. Python runs only the first matching except block.
4. Exception should usually come last.

## Example 1

PYTHON CODE

```
try:
    numbers = [10, 20, 30]
    print(numbers[5])
except IndexError:
    print("Invalid index")
except ValueError:
    print("Invalid value")
except Exception:
    print("Some other error occurred")
```

**Output:** Invalid index

# 16.2.4 Handling Multiple Exceptions Together

Multiple exception types can be handled in one except block.

## Syntax

PYTHON CODE

```
except (ErrorType1, ErrorType2):
    statement
```

## Example 1

PYTHON CODE

```
try:
    value = int("abc")
except (ValueError, TypeError):
    print("Invalid conversion")
```

**Output:** Invalid conversion

CodeWithNishchal

# 16.2.5 else in Exception Handling

The else block runs only when no exception occurs.

## Syntax

```
PYTHON CODE
try:
    risky_code
except ExceptionType:
    handling_code
else:
    code_if_no_error
```

1. else runs only if the try block has no exception. It is useful for success logic.
2. It keeps error handling separate from normal code.

```
PYTHON CODE
try:
    number = int("100")
except ValueError:
    print("Invalid number")
else:
    print("Conversion successful:", number)
```

**Output:** Conversion successful: 100

# 16.2.6 finally

The finally block always runs.

## Syntax

```
PYTHON CODE
try:
    risky_code
except ExceptionType:
    handling_code
finally:
    cleanup_code
```

## Explanation

1. finally runs whether an exception occurs or not.
2. It is used for cleanup operations.
3. It is useful for closing files, network connections, or database connections.
4. With file handling, with is usually cleaner than manually using finally.

deWithNishchal

## Example 1

```python
try:
    file = open("data.txt", "r", encoding="utf-8")
    print(file.read())
except FileNotFoundError:
    print("File not found")
finally:
    print("Finally block executed")
```

**Output:** Finally block executed
The file output depends on whether the file exists.

# 16.2.7 try-except-else-finally Flow

```
try
 |
 v
Error occurs?
 |
 ├── Yes -> except -> finally
 |
 └── No  -> else   -> finally
```

## Structure

```python
try:
    risky_code
except ExceptionType:
    error_handling
else:
    success_code
finally:
    cleanup_code
```

# 16.2.8 Raising Exceptions

Raising an exception means creating an error manually.

## Syntax

raise ExceptionType("message")

WithNishchal

1. raise is used to manually trigger an exception.
2. It is useful for validation.
3. It stops normal flow and sends control to exception handling.
4. We can raise built-in or custom exceptions.

## Example 1

```
PYTHON CODE
age = -5
if age < 0:
    raise ValueError("Age cannot be negative")
```

Output: ValueError: Age cannot be negative

# 16.2.9 Re-raising Exceptions

Re-raising means raising the same exception again after catching it.

## Syntax

raise

1. Plain raise is used inside an except block.
2. It re-raises the current exception.
3. It is useful when we want to log an error but still allow it to continue upward.

## Example 1

```
PYTHON CODE
try:
    number = int("abc")
except ValueError:
    print("Logging error")
    raise
```

**Output:** Logging error
ValueError: invalid literal for int() with base 10: 'abc'

WithNishchal

# 16.2.10 Custom Exceptions

A custom exception is a user-defined exception class.

## Syntax

```
PYTHON CODE
class CustomError(Exception):
    pass
```

## Explanation

1. Custom exceptions are created by inheriting from Exception.
2. They make errors more meaningful.
3. They are useful in larger projects.
4. Custom exception names usually end with Error.

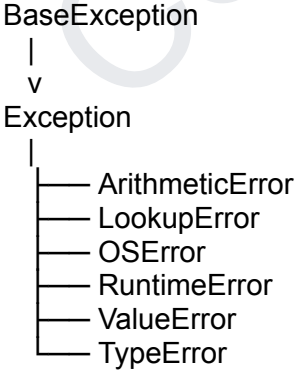
## Example 1

```
PYTHON CODE
class InsufficientBalanceError(Exception):
    pass

balance = 500
withdraw_amount = 1000
if withdraw_amount > balance:
    raise InsufficientBalanceError("Not enough
balance")
```

**Output:** InsufficientBalanceError: Not enough balance

# 16.2.11 Exception Hierarchy




Code Nishchal

| Exception | When It Happens |
|---|---|
| ValueError | Correct type, invalid value |
| TypeError | Operation with wrong type |
| NameError | Name not defined |
| IndexError | Invalid list/tuple index |
| KeyError | Missing dictionary key |
| FileNotFoundError | File does not exist |
| ZeroDivisionError | Division by zero |
| ImportError | Import fails |
| ModuleNotFoundError | Module not found |
| PermissionError | No permission for file operation |
| UnicodeDecodeError | Text decoding fails |

# 16.2.12 Bare except Warning

A bare except catches almost everything and should usually be avoided.

**Bad Style**

```
PYTHON CODE
try:
    number = int("abc")
except:
    print("Error")
```

**Better Style**

```
PYTHON CODE
try:
    number = int("abc")
except ValueError:
    print("Invalid number")
```

## Why Bare except is Bad

| Problem | Explanation |
|---|---|
| Too broad | Catches errors you did not expect |
| Hides bugs | Makes debugging difficult |
| Poor readability | Does not show what error is handled |
| Can catch system-exit style errors | Not safe for normal use |

Nishchal

# 16.2.13 File Handling with Exception Handling

| Error | Meaning |
|---|---|
| FileNotFoundError | File does not exist |
| PermissionError | No permission |
| IsADirectoryError | Expected file but got folder |
| UnicodeDecodeError | Encoding issue |
| OSError | General operating system error |

## Example 1

PYTHON CODE

```
try:
    with open("data.txt", "r", encoding="utf-8")
as file:
        content = file.read()
except FileNotFoundError:
    print("File not found")
else:
    print(content)
```

# 16.3 File Formats

## 16.3.1 Text Files

A text file stores human-readable text.

Examples:

- .txt
- .log
- .md
- .csv
- .json
- .xml

## Explanation

1. Text files store characters.
2. Text files should be opened with encoding.
3. Common modes are "r", "w", and "a".
4. Text mode is default in Python.

CodeWithNishchal

# 16.3.2 CSV Files

CSV means **Comma-Separated Values**. CSV files store table-like data.

### Example CSV Data

name,age,course
Aman,21,Python

1. CSV files store rows and columns. Values are commonly separated by commas.
2. Python provides the built-in csv module. CSV is commonly used for spreadsheets and data export. Use newline="" when opening CSV files for writing.

### Reading CSV File

```
PYTHON CODE
import csv

with open("students.csv", "r",
encoding="utf-8") as file:
    reader = csv.reader(file)

    for row in reader:
        print(row)
```

**Possible output:**['name', 'age', 'course']
['Aman', '21', 'Python']
['Riya', '20', 'Java']

### Writing CSV File

```
PYTHON CODE
import csv

with open("students.csv", "w",
encoding="utf-8", newline="") as file:
    writer = csv.writer(file)

    writer.writerow(["name", "age", "course"])
    writer.writerow(["Aman", 21, "Python"])
```

# 16.3.3 csv.reader vs csv.DictReader

| Tool | Output Type | Best Use |
|---|---|---|
| csv.reader() | List for each row | Simple row-based reading |
| csv.DictReader() | Dictionary for each row | Column-name based reading |
| csv.writer() | Writes rows | Simple CSV writing |
| csv.DictWriter() | Writes dictionaries | Column-name based writing |

ithNishchal

# 16.3.4 JSON Files

JSON means **JavaScript Object Notation**. JSON is commonly used for APIs and configuration files.

## Example JSON Data

```
PYTHON CODE
{
  "name": "Aman",
  "age": 21,
  "course": "Python"
}
```

## Explanation

1. JSON stores data in key-value format. It looks similar to Python dictionaries.
2. Python provides the built-in json module.
3. JSON is very common in web development and APIs. JSON keys must be strings.

## Python and JSON Conversion

| Python | JSON |
|---|---|
| dict | object |
| list | array |
| str | string |
| int, float | number |
| True | true |
| False | false |
| None | null |

## Reading JSON File

```
PYTHON CODE
import json

with open("student.json", "r",
encoding="utf-8") as file:
    data = json.load(file)
print(data["name"])
```

CodeWithNishchal

## Writing JSON File

```
PYTHON CODE
import json

student = {
    "name": "Aman",
    "age": 21,
    "course": "Python"
}

with open("student.json", "w",
encoding="utf-8") as file:
    json.dump(student, file, indent=4)
```

# 16.3.5 json.load() vs json.loads()

| Function | Meaning | Input |
|---|---|---|
| json.load() | Reads JSON from file | File object |
| json.loads() | Reads JSON from string | JSON string |
| json.dump() | Writes JSON to file | File object |
| json.dumps() | Converts Python object to JSON string | Python object |

# 16.3.6 XML Files

XML means **Extensible Markup Language**.

XML stores data using tags.

## Example XML Data

```
<student>
    <name>Aman</name>
    <age>21</age>
    <course>Python</course>
</student>
```

CodeWithNishchal

# Chapter 17. Advanced Python Concepts

Advanced Python concepts help us write cleaner, memory-efficient, reusable, and professional Python code.
This chapter covers:

1. Decorators
2. Generators and Iterators
3. Context Managers
4. Regular Expressions

# 17.1 Decorators

## 17.1.1 Decorators

A **decorator** is a function that takes another function and adds extra behavior to it without changing the original function code.

## Basic Idea

Decorator = function that modifies/enhances another function

## Syntax

```python
@decorator_name
def function_name():
    statement
```

**This=>**

```python
@decorator_name
def function_name():
    statement
```

**is equal to:**

```python
def function_name():
    statement

function_name = decorator_name(function_name)
```

Code With Nishchal

1. Decorators are based on functions being first-class objects. A decorator takes a function as input.
2. It usually defines an inner wrapper function. It returns the wrapper function.
3. The wrapper adds extra behavior before or after the original function.

## Flow Chart

Original function
  |
  v
Decorator receives function
  |
  v
Wrapper adds extra behavior
  |
  v
Decorated function is returned

# 17.1.2 Function Decorators

A function decorator is used to add extra behavior to a function.

## Syntax

```python
def decorator_name(func):
    def wrapper():
        extra_code
        func()
        extra_code
    return wrapper
```

PYTHON CODE

```
def my_decorator(func):
    def wrapper():
        print("Before function")
        func()
        print("After function")
    return wrapper

@my_decorator
def greet():
    print("Hello")
greet()
```

Output: Before function
Hello
After function

1. my_decorator receives greet. wrapper adds extra behavior.
2. greet() now actually calls wrapper(). Inside wrapper, the original greet() is called.

WithNishchal

# 17.1.3 Decorators with Function Arguments

If the decorated function has arguments, the wrapper should accept *args and **kwargs.

## Syntax

PYTHON CODE
```
def decorator_name(func):
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper
```

## Example 1

PYTHON CODE
```
def show_call(func):
    def wrapper(*args, **kwargs):
        print("Function is being called")
        return func(*args, **kwargs)
    return wrapper

@show_call
def add(a, b):
    return a + b

print(add(10, 20))
```

Output: Function is being called
30

# 17.1.4 functools.wraps

functools.wraps preserves the original function's metadata. Without wraps, the decorated function may lose its original name and docstring.

## Syntax

Python
```
from functools import wraps

def decorator_name(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper
```

CodeWithNishchal

1. Decorators replace the original function with a wrapper.
2. Because of this, metadata like function name and docstring can be lost.
3. @wraps(func) copies metadata from the original function to the wrapper.
4. Professional decorators should usually use functools.wraps.

PYTHON CODE
```
from functools import wraps

def my_decorator(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper

@my_decorator
def greet():
    """This function greets the user."""
    print("Hello")

print(greet.__name__)
print(greet.__doc__)
```

**Output:** greet

This function greets the user.

# 17.1.5 Decorators with Arguments

Decorators can also accept their own arguments. This requires one extra outer function.

## Syntax

PYTHON CODE
```
def decorator_with_args(value):
    def actual_decorator(func):
        def wrapper(*args, **kwargs):
            statement
            return func(*args, **kwargs)
        return wrapper
    return actual_decorator
```

1. The outer function receives decorator arguments. The middle function receives the original function.
2. The inner wrapper runs extra logic.This is useful when decorator behavior needs customization.

eWithNishchal

```python
from functools import wraps

def repeat(times):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            for _ in range(times):
                func(*args, **kwargs)
        return wrapper
    return decorator

@repeat(3)
def greet():
    print("Hello")

greet()
```

**Output:** Hello
Hello
Hello

# 17.1.6 Multiple Decorators

A function can have more than one decorator.

## Syntax

```python
@decorator1
@decorator2
def function_name():
    statement
```

**NOTE:** Decorators are applied from bottom to top.

```python
@decorator1
@decorator2
def greet():
    pass
```

**is equal to:**

```python
greet = decorator1(decorator2(greet))
```

Nishchal

# 17.1.7 Class Decorators

A class decorator modifies or enhances a class.

## Syntax

```
PYTHON CODE
@decorator_name
class ClassName:
    statement
```

1. A class decorator receives a class as input. It can add or modify class behavior.
2. It returns the modified class. Class decorators are less common than function decorators.
3. They are useful for logging, registration, validation, and configuration.

# 17.1.8 Built-in and Standard Decorators

| Decorator | From | Purpose |
|---|---|---|
| **@staticmethod** | Built-in | Creates static method |
| **@classmethod** | Built-in | Creates class method |
| **@property** | Built-in | Creates managed attribute |
| **@property_name.setter** | Built-in | Adds setter to property |
| **@dataclass** | dataclasses | Generates data class methods |
| **@abstractmethod** | abc | Defines abstract method |
| **@wraps** | functools | Preserves function metadata |
| **@lru_cache** | functools | Caches function results |

# 17.2 Generators and Iterators

Generators and iterators are used to work with values one at a time. They are memory-efficient because they do not need to store all values at once.

CodeWithNischal

# 17.2.1 Iterable vs Iterator

## Explanation

An iterable is an object we can loop over. An iterator is an object that gives values one by one using next().

| Term | Meaning | Example |
|---|---|---|
| **Iterable** | Can be looped over | list, tuple, string, dictionary, set, range |
| **Iterator** | Gives next value using next() | object from iter() |

## Example 1

PYTHON CODE
```
numbers = [10, 20, 30]


iterator = iter(numbers)
print(next(iterator))
print(next(iterator))
print(next(iterator))
```

**Output:**
10
20
30

# 17.2.2 Iterator Protocol

The iterator protocol uses two methods:

1. __iter__()
2. __next__()

## Syntax

PYTHON CODE
```
def __iter__(self):
    return self

def __next__(self):
    return next_value
```

CodeWithNishchal

## Explanation

1. __iter__() returns an iterator object.
2. __next__() returns the next value.
3. When no values are left, __next__() raises StopIteration.
4. for loops use this protocol internally.

## Example 1

PYTHON CODE

```
class CountUpTo:
    def __init__(self, limit):
        self.current = 1
        self.limit = limit

    def __iter__(self):
        return self

    def __next__(self):
        if self.current > self.limit:
            raise StopIteration

        value = self.current
        self.current += 1
        return value

counter = CountUpTo(3)

for number in counter:
    print(number)
```

Output:

```
1
2
3
```

Nishchal

# 17.2.3 __iter__

__iter__ returns an iterator object.

## Syntax

```
PYTHON CODE
def __iter__(self):
    return self
```

## Explanation

1. __iter__() is called by iter(object).
2. It is also used automatically for loops.
3. If the object itself is the iterator, it returns self.
4. If the object is only iterable, it can return a separate iterator object.

## Example 1

```
PYTHON CODE
numbers = [10, 20, 30]


iterator = iter(numbers)

print(iterator)
```

Output: A list_iterator object

# 17.2.4 __next__

__next__ returns the next value from an iterator.

## Syntax

```
PYTHON CODE
def __next__(self):
    return value
```

1. __next__() is called by next(iterator).
2. It returns one value at a time.
3. It should raise StopIteration when there are no values left.
4. Without StopIteration, the iteration may not stop correctly.

## Example 1

CodeWithNishchal

```python
numbers = iter([10, 20])


print(next(numbers))
print(next(numbers))
print(next(numbers))
```

Output:
10
20
StopIteration
The third next() raises StopIteration because no values are left.

# 17.2.5 Generator Functions

A generator function is a function that uses yield.

## Syntax

```python
def generator_name():
    yield value
```

## Explanation

1. A generator function returns a generator object.
2. It does not run fully at once.
3. It pauses at yield.
4. When next() is called again, it continues from where it paused.
5. Generators are memory-efficient.

## Example 1

```python
def count_up_to_three:
    yield 1
    yield 2
    yield 3
```

**Correct code:**

CodeWithNishchal

```python
def count_up_to_three():
    yield 1
    yield 2
    yield 3

counter = count_up_to_three()

print(next(counter))
print(next(counter))
print(next(counter))
```

Output:
1
2
3

# 17.2.6 yield

yield is used to return a value from a generator without ending the function permanently.

## Syntax

yield value

1. yield gives one value at a time. It pauses the function.
2. The function state is saved.
3. On the next call, execution continues after the previous yield.
4. When the function finishes, Python raises StopIteration.

## return vs yield

| Point | return | yield |
|---|---|---|
| Used in | Normal functions | Generator functions |
| Gives | Final result | One value at a time |
| Function state | Ends function | Pauses function |
| Memory use | May store full result | Memory-efficient |
| Output | Normal value | Generator object |

CodeWithNishchal

# 17.2.7 Generator Expressions

A generator expression is a short way to create a generator.

## Syntax

(expression for item in iterable)

## Explanation

1. Generator expressions look like list comprehensions.
2. They use parentheses ().
3. They produce values lazily.
4. They do not create a full list in memory.
5. They are useful for large data.

## Example 1

```
PYTHON CODE
squares = (number * number for number in
range(1, 5))

for value in squares:
    print(value)
```

Output:
1
4
9
16

# 17.2.8 yield from

yield from is used to yield values from another iterable or generator.

## Syntax

yield from iterable

1. yield from simplifies nested loops in generators It passes values from another iterable directly.
2. It is useful when one generator uses another generator.
3. It makes generator code cleaner.

CodeWithNishchal

# 17.2.9 One-time Consumption of Iterators

Iterators and generators are usually consumed once.

## Explanation

1. Once a value is taken from an iterator, it is not repeated.
2. After all values are consumed, the iterator is exhausted.
3. To iterate again, create a new iterator or generator.

## Example 1

values = (x for x in range(3))

print(list(values))

print(list(values))

**Output:**

[0, 1, 2]

[]

The second list is empty because the generator was already consumed.

---

# 17.2.10 itertools

itertools is a standard library module for working with iterators.

## Syntax

import itertools

## Explanation

1. itertools provides memory-efficient iterator tools.
2. It is useful for combinations, permutations, counting, grouping, and chaining.
3. Many itertools functions return iterators.
4. Results may need to be converted using list() for display.

CodeWithNishchal

## Common itertools Tools

| Tool | Meaning |
|---|---|
| count() | Infinite counting |
| cycle() | Repeats iterable forever |
| repeat() | Repeats a value |
| chain() | Joins iterables lazily |
| islice() | Slices an iterator |
| combinations() | Unique combinations |
| permutations() | All possible arrangements |
| product() | Cartesian product |
| groupby() | Groups consecutive matching items |

# 17.2.11 Memory Efficiency

Generators are memory-efficient because they produce values only when needed.

1. Lists store all values in memory.
2. Generators produce one value at a time.
3. This is called lazy evaluation.
4. Generators are useful for large files, large ranges, and data streams.

## Example 1

```
PYTHON CODE
numbers = (number for number in
range(1000000))
print(next(numbers))
print(next(numbers))
```

Output:
0
1
Only needed values are produced.

CodeWithNishchal

# 17.3 Context Managers

A context manager is an object that manages setup and cleanup automatically.

The most common example is file handling using with.

## Syntax

with resource as name:

    statement

1. Context managers are used with the with statement.
2. They handle setup before the block starts.
3. They handle cleanup after the block ends.
4. Cleanup happens even if an error occurs.
5. They are useful for files, database connections, locks, and network connections.

## Flow Chart

with block starts
  |
  v
__enter__ runs
  |
  v
Block code runs
  |
  v
__exit__ runs

## Example 1

PYTHON CODE

```
with open("data.txt", "w", encoding="utf-8") as file:

    file.write("Hello Python")
```

The file closes automatically after the with block.

---

CodeWithNishchal

# 17.3.1 __enter__

__enter__ runs at the beginning of a with block.

## Syntax

```python
def __enter__(self):
    return self
```

1. __enter__() is called when the with block starts. It usually prepares or opens a resource.
2. The value returned by __enter__() is stored after as. It is part of the context manager protocol.

# 17.3.2 __exit__

__exit__ runs when the with block ends.

## Syntax

```python
def __exit__(self, exc_type, exc_value, traceback):
    statement
```

1. __exit__() is called at the end of the with block. It is used for cleanup.
2. It receives exception details if an error occurs. If it returns True, the exception is suppressed.
3. If it returns False or None, the exception continues normally.

## __exit__ Parameters

| Parameter | Meaning |
|---|---|
| exc_type | Exception class/type |
| exc_value | Exception object/message |
| traceback | Traceback object |
| None, None, None | Passed if no exception occurred |

CodeWithNishchal

# 17.3.3 Custom Context Manager Using Class

## Syntax

```
PYTHON CODE
class ClassName:
    def __enter__(self):
        return self

    def __exit__(self, exc_type, exc_value,
traceback):
        cleanup_code
```

## Example 1

```
PYTHON CODE
class SimpleContext:
    def __enter__(self):
        print("Entering")
        return self

    def __exit__(self, exc_type, exc_value,
traceback):
        print("Exiting")

with SimpleContext():
    print("Inside block")
```

**Output:** Entering
Inside block
Exiting

# 17.3.4 contextlib

contextlib is a standard library module for creating and working with context managers.

## Syntax

from contextlib import contextmanager

1. contextlib provides utilities for context managers.
2. It helps create context managers without writing a full class.
3. The most common tool is @contextmanager. It is useful for simple setup-cleanup logic.

CodeWithNishchal

# 17.3.5 @contextmanager

@contextmanager converts a generator function into a context manager.

## Syntax

```
from contextlib import contextmanager

@contextmanager
def manager_name():
    setup_code
    yield value
    cleanup_code
```

## Explanation

1. Code before yield works like __enter__. The yielded value is used after as.
2. Code after yield works like __exit__. Cleanup should be placed in finally for safety.
3. This is useful for simple context managers.

## Class Context Manager vs @contextmanager

| Point | Class-based | @contextmanager |
|---|---|---|
| Uses | __enter__, __exit__ | yield |
| Best for | Complex managers | Simple managers |
| Code length | More | Less |

# 16.3.6 Resource Management

Resource management means safely opening and closing resources.

| Resource | Why Context Manager Helps |
|---|---|
| File | Closes automatically |
| Database connection | Disconnects safely |
| Lock | Releases lock |
| Network connection | Closes connection |
| Temporary setup | Cleans up after use |

CodeWithNishchal

# 17.4 Regular Expressions

A regular expression, or **regex**, is a pattern used to search, match, extract, replace, or split text.

## Basic Idea

Regex = pattern matching for text

## Example Uses

1. Validate email format.
2. Extract phone numbers.
3. Find dates in text.
4. Replace unwanted characters.
5. Split text using complex patterns.

---

# 17.4.1 re Module

Python provides the built-in re module for regular expressions.

## Syntax

import re

## Explanation

1. re is Python's regular expression module.
2. It provides functions like search(), match(), findall(), sub(), and split().
3. Regex patterns are usually written as raw strings.
4. Raw strings use prefix r.

## Raw String Example

pattern = r"\d+"
This is preferred over:
pattern = "\\d+"

**NOTE:** Use raw strings for regex patterns to avoid escape-character confusion.

CodeWithNishchal

# 17.4.2 Pattern Matching

Pattern matching means checking whether text follows a specific pattern.

```
PYTHON CODE
import re


text = "My age is 21"
result = re.search(r"\d+", text)
print(result.group())
```

Output: 21

# 17.4.3 Regex Metacharacters

Metacharacters are special characters with special meaning in regex.

| Pattern | Meaning | Example Match |
|---|---|---|
| . | Any character except newline | a, 1, @ |
| ^ | Start of string | ^Hello |
| $ | End of string | world$ |
| * | Zero or more | a* |
| + | One or more | a+ |
| ? | Zero or one | a? |
| {n} | Exactly n times | a{3} |
| {n,} | n or more times | a{2,} |
| {n,m} | Between n and m times | a{2,4} |
| [] | Character set | [abc] |
| [^] | Not in set | [^0-9] |
| ` | ` | OR |
| () | Group | (ab)+ |
| \ | Escape character | \. |

CodeWithLischal

# 17.4.4 Common Regex Character Classes

| Pattern | Meaning |
|---|---|
| \d | Digit, same as [0-9] |
| \D | Non-digit |
| \w | Word character: letters, digits, underscore |
| \W | Non-word character |
| \s | Whitespace |
| \S | Non-whitespace |
| \b | Word boundary |
| \A | Start of string |
| \Z | End of string |

# 17.4.5 Groups and Capturing

Groups are created using parentheses ().

They are used to capture parts of a match.

## Syntax

(pattern)

## Explanation

1. Parentheses create groups.
2. Capturing groups store matched parts.
3. group(0) gives the full match.
4. group(1) gives the first captured group.
5. group(2) gives the second captured group.

CodeWithNishchal

# 17.4.6 Non-capturing Groups

A non-capturing group groups a pattern without storing it.

## Syntax

(?:pattern)

## Explanation

1. Non-capturing groups are used for grouping only.
2. They do not create a captured group number.
3. They are useful when we need grouping but do not need to extract that part.

# 17.4.7 search()

search() looks for the first match anywhere in the string.

## Syntax

re.search(pattern, text)

## Explanation

1. Searches the entire string. Returns the first match object.
2. Returns None if no match is found.
3. Use .group() to get matched text.

# 17.4.8 match()

match() checks for a match only at the beginning of the string.

## Syntax

re.match(pattern, text)

## Explanation

1. Checks only from the start of the string. Returns match object if pattern matches at the beginning.
2. Returns None if pattern appears later.

CodeWithNishchal

# 17.4.9 fullmatch()

fullmatch() checks whether the whole string matches the pattern.

## Syntax

re.fullmatch(pattern, text)

## Explanation

1. It checks the entire string.
2. It returns a match only if the full string matches.
3. It is useful for validation.

# 17.4.10 findall()

findall() returns all non-overlapping matches as a list.

## Syntax

re.findall(pattern, text)

## Explanation

1. Finds all matches.
2. Returns a list.
3. If the pattern has capturing groups, it returns captured groups.
4. If no match is found, it returns an empty list.

# 17.4.11 finditer()

finditer() returns an iterator of match objects.

## Syntax

re.finditer(pattern, text)

1. It finds all matches. It returns match objects one by one.
2. It is useful when we need match positions.
3. It is more memory-friendly for large text than findall().

CodeWithNishchal

# 17.4.12 sub()

sub() replaces matched text with new text.

## Syntax

re.sub(pattern, replacement, text)

## Explanation

1. Finds matches using pattern. Replaces them with replacement text.
2. Returns the modified string. Original string is not changed.

# 17.4.13 split()

split() splits a string using a regex pattern.

## Syntax

re.split(pattern, text)

1. Splits text wherever the pattern matches.
2. More powerful than normal string split().
3. Useful when separators are multiple or irregular.

# 17.4.14 re.compile()

re.compile() creates a reusable regex pattern object.

## Syntax

pattern = re.compile(r"pattern")

## Explanation

1. Compiled patterns can be reused.
2. This makes code cleaner when using the same pattern many times.
3. It can be useful for repeated matching.
4. Pattern objects have methods like search(), findall(), sub(), and split()

CodeWithNishchal

## Chapter 18. Concurrent and Asynchronous Programming

Concurrent programming means handling multiple tasks during the same time period. It does not always mean tasks are running at the exact same instant.

## Basic Terms

| Term | Meaning |
|---|---|
| **Concurrency** | Managing multiple tasks at once |
| **Parallelism** | Running multiple tasks at the same time |
| **I/O-bound task** | Task waiting for input/output, like file, network, database |
| **CPU-bound task** | Task doing heavy calculation |
| **Thread** | Lightweight unit of execution inside a process |
| **Process** | Independent program execution with separate memory |
| **Async programming** | Single-threaded cooperative concurrency using async and await |

## Choosing the Right Tool

| Task Type | Better Tool |
|---|---|
| Waiting for network/API/file/database | threading, ThreadPoolExecutor, or asyncio |
| Heavy CPU calculation | multiprocessing or ProcessPoolExecutor |
| Many async network operations | asyncio |
| Simple parallel task execution | concurrent.futures |

CodeWithNishchal

# 18.1 Multithreading

Multithreading means running multiple threads inside the same process.

Python provides the threading module for thread-based concurrency. The official docs describe threading as thread-based parallelism and also point to ThreadPoolExecutor as a higher-level interface.

## Syntax

import threading

1. A thread is a small unit of execution.
2. Multiple threads can run inside one process.
3. Threads share the same memory.
4. Threads are useful for I/O-bound tasks.
5. Threads can cause race conditions when shared data is modified.
6. Locks are used to protect shared data.

## Flow Chart

Main program
  |
  v
Create threads
  |
  v
Start threads
  |
  v
Threads run tasks
  |
  v
Join threads
  |
  v
Program continues

# 18.1.1 threading Module

The threading module provides classes and tools for working with threads.

| Tool | Purpose |
|---|---|
| Thread | Creates and manages a thread |
| Lock | Prevents multiple threads from changing shared data at the same time |
| RLock | Re-entrant lock; same thread can acquire it multiple times |
| Semaphore | Limits how many threads can access a resource |
| Event | Allows communication between threads |
| Condition | Allows threads to wait for a condition |
| current_thread() | Returns current thread object |

CodeWithNishchal

| | |
|---|---|
| active_count() | Returns number of active threads |

# 18.1.2 Creating Threads

## Syntax

thread = threading.Thread(target=function_name)
thread.start()
thread.join()

## Explanation

1. target is the function that the thread will run.
2. start() starts the thread.
3. join() waits for the thread to finish.
4. Without join(), the main program may continue while the thread is still running.

## Example 1

PYTHON CODE

```
import threading

def show_message():
    print("Thread is running")

thread =
threading.Thread(target=show_message)

thread.start()
thread.join()

print("Main program finished")
```

Output:

Thread is running

Main program finished

---

Nishchal

# 18.1.3 Managing Threads

| Method / Attribute | Meaning |
|---|---|
| start() | Starts thread execution |
| join() | Waits for thread to finish |
| is_alive() | Checks whether thread is still running |
| name | Thread name |
| daemon | If True, thread stops when main program exits |

# 18.1.4 Race Conditions

A race condition happens when multiple threads access and modify shared data at the same time, causing incorrect results.

## Explanation

1. Threads share memory.
2. If two threads change the same variable together, the result can become unpredictable.
3. This problem is called a race condition.
4. Locks are used to avoid race conditions.

## Example Idea

Thread 1 reads count = 0

Thread 2 reads count = 0

Thread 1 updates count to 1

Thread 2 updates count to 1

Expected result: 2

Actual result: 1

CodeWithNishchal

# 18.1.5 Locks

A lock allows only one thread to access a critical section at a time.

## Syntax

```
lock = threading.Lock()
with lock:
    shared_data_update
```

1. A lock protects shared data.
2. Only one thread can hold the lock at a time.
3. Other threads must wait.
4. Use with lock: because it releases the lock automatically.

## Example 1

PYTHON CODE

```
import threading

count = 0
lock = threading.Lock()

def increase():
    global count

    for _ in range(100000):
        with lock:
            count += 1

thread1 = threading.Thread(target=increase)
thread2 = threading.Thread(target=increase)

thread1.start()
thread2.start()
thread1.join()
thread2.join()
print(count)
```

Nishchal

Output: 200000

# 18.1.6 Synchronization

Synchronization means coordinating threads so they work safely together.

## Synchronization Tools Table

| Tool | Purpose |
|---|---|
| Lock | Allows one thread at a time |
| RLock | Allows same thread to acquire lock multiple times |
| Semaphore | Allows limited number of threads |
| Event | One thread signals another thread |
| Condition | Threads wait until a condition becomes true |
| Queue | Thread-safe data exchange between threads |

# 18.2 Multiprocessing

Multiprocessing means running multiple processes.

Python provides the multiprocessing module for process-based parallelism. ProcessPoolExecutor is also documented as a higher-level interface for running tasks in background processes.

## Syntax

import multiprocessing

## Explanation

1. A process has its own Python interpreter and memory space.
2. Processes do not share normal memory like threads.
3. Multiprocessing is useful for CPU-bound tasks.
4. It can use multiple CPU cores.
5. Communication between processes needs special tools like Queue, Pipe, Manager, or shared memory.

CodeWithNishchal

## Thread vs Process

| Point | Thread | Process |
|---|---|---|
| Memory | Shared memory | Separate memory |
| Creation cost | Lighter | Heavier |
| Best for | I/O-bound tasks | CPU-bound tasks |
| Communication | Easier but risky | Needs special tools |
| Race condition risk | Higher with shared data | Lower by default |

# 18.2.1 Managing Processes

| Method / Attribute | Meaning |
|---|---|
| start() | Starts process |
| join() | Waits for process to finish |
| is_alive() | Checks whether process is running |
| terminate() | Stops process forcefully |
| pid | Process ID |
| name | Process name |
| exitcode | Process exit status |

# 18.2.2 Process Pools

A process pool manages multiple worker processes.

## Syntax

from multiprocessing import Pool

1. A pool reuses worker processes. It is useful when many tasks need to be processed.

CodeWithNishchal

2. It avoids manually creating many process objects.
3. For many cases, ProcessPoolExecutor is simpler and more modern.

# 18.2.3 Memory Sharing Basics

Processes have separate memory by default.

1. Normal variables are not automatically shared between processes.
2. Each process gets its own memory space.
3. To share data, special multiprocessing tools are needed.
4. Shared memory should be used carefully.

## Memory Sharing Tools

| Tool | Purpose |
|---|---|
| multiprocessing.Queue | Send data between processes |
| multiprocessing.Pipe | Two-way communication |
| multiprocessing.Manager | Shared list/dict-like objects |
| multiprocessing.Value | Shared single value |
| multiprocessing.Array | Shared array |
| multiprocessing.shared_memory | Direct shared memory block |

# 18.3 Concurrent Futures

The concurrent.futures module provides a high-level interface for running callables asynchronously using threads, processes, or interpreters. The main executors are ThreadPoolExecutor and ProcessPoolExecutor.

## Syntax

from concurrent.futures import ThreadPoolExecutor

from concurrent.futures import ProcessPoolExecutor

## Explanation

1. concurrent.futures is easier than manually creating threads/processes.

CodeWithNischal

2. It uses executors to manage workers. It returns Future objects.
3. A Future represents a result that may not be ready yet.
4. It is useful for running many tasks concurrently.

# 18.3.1 ThreadPoolExecutor

ThreadPoolExecutor runs tasks using a pool of threads.  Best for I/O-bound tasks

## Example 1

PYTHON CODE

```
from concurrent.futures import
ThreadPoolExecutor


def square(number):
    return number * number

with ThreadPoolExecutor() as executor:
    results = executor.map(square, [1, 2, 3, 4])


print(list(results))
```

Output: [1, 4, 9, 16]

# 18.3.2 ProcessPoolExecutor

ProcessPoolExecutor runs tasks using a pool of processes. Best for CPU-bound tasks

PYTHON CODE

```
from concurrent.futures import ProcessPoolExecutor


def square(number):
    return number * number

if __name__ == "__main__":
    with ProcessPoolExecutor() as executor:
        results = executor.map(square, [1, 2, 3, 4])
    print(list(results))
```

odeWithNishchal

Output: [1, 4, 9, 16]

# 18.3.3 Futures

A Future represents a task that may complete later.

| Method | Meaning |
|---|---|
| result() | Gets result, waits if needed |
| done() | Checks whether task is completed |
| cancel() | Attempts to cancel task |
| cancelled() | Checks whether task was cancelled |
| exception() | Returns exception if task failed |
| add_done_callback() | Runs callback when task finishes |

## Example 1

PYTHON CODE

```
from concurrent.futures import
ThreadPoolExecutor

def square(number):
    return number * number

with ThreadPoolExecutor() as executor:
    future = executor.submit(square, 5)

print(future.result())
```

Output: 25

# 18.4 Async Programming

Async programming is used to write concurrent code using async and await.

Python's official asyncio documentation describes it as a library for writing concurrent code using async/await, commonly used for high-performance network servers, web servers,

hNishchal

database libraries, and distributed task queues. **Best for I/O-bound tasks with many waiting operations**

# 18.4.1 asyncio

asyncio is Python's standard library module for asynchronous programming.

## Syntax

import asyncio

1. asyncio runs asynchronous tasks.
2. It uses an event loop.
3. It is usually single-threaded cooperative concurrency.
4. Tasks pause when they reach await.
5. While one task is waiting, another task can run.

## Flow Chart

Event loop starts
  |
  v
Task 1 runs
  |
  v
Task 1 awaits
  |
  v
Task 2 runs
  |
  v
Tasks complete

# 18.4.2 async and Coroutines

A coroutine is created using async def.

## Syntax

```
async def function_name():
    statement
```

## Explanation

1. async def defines a coroutine function.
2. Calling a coroutine function returns a coroutine object.

CodeWithNishchal

3. The coroutine does not run immediately just because it is called.
4. It must be awaited or run by the event loop.
5. asyncio.run() is commonly used to start the main coroutine.

## Example 1

PYTHON CODE
```
import asyncio


async def greet():
    print("Hello async")


asyncio.run(greet())
```

Output: Hello async

# 18.4.3 await

await pauses a coroutine until an awaitable finishes.

## Syntax

await awaitable

## Explanation

1. await can be used only inside async def. It pauses the current coroutine.
2. It allows the event loop to run other tasks.
3. It is used with coroutines, tasks, and async operations.

## Example 1

PYTHON CODE
```
import asyncio


async def main():
    print("Start")
    await asyncio.sleep(1)
    print("End")
asyncio.run(main())
```

CodeWithNishchal

Output: Start

End

# 18.4.4 Event Loop

The event loop manages and runs async tasks.

1. The event loop schedules coroutines. It switches between tasks when they await.
2. asyncio.run() creates and manages the event loop for most programs.
3. Beginners should usually use asyncio.run() instead of manually creating loops.

# 18.4.5 Tasks

A task schedules a coroutine to run concurrently.

## Syntax

task = asyncio.create_task(coroutine())

1. A task wraps a coroutine. It schedules the coroutine to run on the event loop.
2. Multiple tasks can run concurrently. await task gets the final result.

## Example 1

```
PYTHON CODE
import asyncio

async def work(name):
    await asyncio.sleep(1)
    print(name, "done")

async def main():
    task1 = asyncio.create_task(work("Task 1"))
    task2 = asyncio.create_task(work("Task 2"))

    await task1
    await task2
asyncio.run(main())
```

**Output:**

With Nishchal

Task 1 done
Task 2 done
Both tasks wait concurrently.

# 18.4.6 asyncio.gather()

asyncio.gather() runs multiple awaitables concurrently and collects their results.

# 18.4.7 Async Futures

An asyncio.Future represents a result that may be available later.

1. asyncio.Future is a low-level awaitable object.
2. It is mainly used inside asyncio libraries and frameworks.
3. In normal application code, prefer coroutines and tasks.
4. Do not confuse asyncio.Future with concurrent.futures.Future.

Python's docs note that asyncio.Future is usually for low-level callback-based code and recommend not exposing Future objects in user-facing APIs.

## Future Comparison

| Type | Module | Used With |
|---|---|---|
| concurrent.futures.Future | concurrent.futures | Thread/process executors |
| asyncio.Future | asyncio | Event loop and async tasks |

# 18.4.8 Async Context Managers

Async context managers are used with async with. They use:

__aenter__
__aexit__
not normal __enter__ and __exit__.

## Syntax

PYTHON CODE
```
class ClassName:
    async def __aenter__(self):
        return self
```

CodeWithNischal

```
    async def __aexit__(self, exc_type,
exc_value, traceback):
        pass
```

# Explanation

1. async with is used for async resource management.
2. __aenter__ runs when the async context starts.
3. __aexit__ runs when the async context ends.
4. These methods can use await.
5. They are common in async database connections, HTTP clients, and network resources.

# Example 1

PYTHON CODE

```
import asyncio


class AsyncManager:
    async def __aenter__(self):
        print("Entering async context")
        return self


    async def __aexit__(self, exc_type,
exc_value, traceback):
        print("Exiting async context")


async def main():
    async with AsyncManager():
        print("Inside async context")


asyncio.run(main())
```

Output:
Entering async context
Inside async context
Exiting async context

Nishchal

# 18.4.9 Async vs Threading vs Multiprocessing

| Feature | Threading | Multiprocessing | Asyncio |
|---|---|---|---|
| Unit | Thread | Process | Coroutine/task |
| Memory | Shared | Separate | Shared in same thread |
| Best for | I/O-bound blocking work | CPU-bound work | I/O-bound non-blocking work |
| Runs in | Same process | Separate processes | Event loop |
| Communication | Shared data, queue | Queue, pipe, manager | Awaitables, tasks |
| Main risk | Race conditions | Process overhead | Blocking the event loop |
| Common tool | threading | multiprocessing | asyncio |

## Important Async Rule

Do not put long blocking code inside async functions.

If blocking code runs inside the event loop, it delays other async tasks. Python's asyncio development docs note that CPU-intensive work can block the event loop and delay other tasks, and executors can be used to run such work elsewhere.

CodeWithNishchal

SOME LOVE BY FOLLOWING **CodeWithNishchal on social media:**

- **Instagram:** https://www.instagram.com/codewithnishchal/
- **LinkedIn:** https://www.linkedin.com/in/nishchal-muradia/
- **YouTube:** https://www.youtube.com/@CodeWithNishchal
- **Telegram** https://t.me/codewithnishchal
- **Watch My Latest Time Complexity Series:** Time Complexity
- **Join My IG Community for Tech discussions :** CodeN' community